

Compositional Probabilistic Optimization

*A Markov-Categorical Backbone and Plugin Architecture for
Retrieval-Augmented Systems*

Volume III of the ragkit / ragopt architecture study

August 2026

Contents

Abstract
Preface
Principal contributions
Notation
1. Optimization is a domain, not a loop
2. The supplied optimization architecture
3. Requirements and non-goals
4. Four semantic layers
5. Categories of processes
6. Markov categories
7. Typed sampled kernels
8. Structured trace algebra
9. Randomness and couplings
10. Parametrized processes
11. Evaluation as a separate morphism
12. Observations and sufficient statistics
13. Exact paired experiments
14. Paired deltas and statistical interpretation
15. Decision semantics
16. Partial order and Pareto fronts
17. Adaptive campaigns as stochastic processes
18. Visibility and information-flow discipline
19. Statistical games and Bayesian lenses
20. Refinement and equivalence
21. The abstract optimization field
22. Denotational semantics of the backbone
23. Operational semantics
24. The plugin doctrine
25. Law certificates
26. Catalogs are not service locators
27. Dynamic plugins at explicit edges
28. Exact and sampled interpreters
29. Repository tour
30. Canonical specifications
31. Seeds, outcomes, and traces
32. Composition combinators
33. Exact finite stochastic processes
34. Experiment custody
35. Evidence and gates
36. Campaign package
37. Plugin implementation
38. Compatibility with current ragopt
39. RAG as a parametrized open process
40. Indexing plugins
41. Query plugins
42. Answer and agent plugins
43. RAG evaluator plugins
44. The toy RAG domain
45. Demonstration campaign
46. Indexing-querying joint optimization
47. Production optimization as open-loop and closed-loop control
48. ragopt: preserve custody, add semantics around it
49. ragkit: make components realizations of typed process specs
50. RAG-TTC: proving the composite architecture
51. GEC: authorization and administrative evaluation
52. Garden: interaction and presentation as evaluable processes
53. Plugin placement rules
54. The dependency graph as a free presentation
55. Migration plan
56. Law hierarchy
57. Property-based and differential testing
58. State-machine testing
59. Security and plugin trust
60. Statistical validity under adaptivity
61. Limits of the reference implementation
62. Research directions
63. Conclusion
Appendix A. Formal definitions and laws
Appendix B. Structural operational semantics
Appendix C. Core Go API blueprint
Appendix D. Plugin law catalog
Appendix E. RAG parameter, dependency, and fidelity catalog
Appendix F. Sandbox execution, extension, and verification
Appendix G. Empirical source map and current-to-target relation
Appendix H. Selected bibliography
Appendix I. Glossary

Abstract

Optimization infrastructure for retrieval-augmented generation is usually built as an accretion of runners, parameter dictionaries, callbacks, metric maps, search algorithms, and deployment scripts. These systems can execute experiments, but they rarely state what an optimizable system is, how stochastic components compose, what evidence means, which parts of an experiment are product-specific, or when a plugin can be substituted without invalidating the comparison. As a result, composition is syntactic rather than semantic. A chunker can be replaced through an interface, yet there is no general account of the artifacts it invalidates. A reranker can be registered by name, yet its disclosure effects and random coupling are not part of its contract. An optimizer can choose the next trial, yet the evidence it has observed and the holdout data it must not observe are not represented in the type or runtime state.

This thesis develops a compact mathematical and software backbone for compositional optimization, with retrieval-augmented systems as the principal application. The foundation is a symmetric monoidal category of typed stochastic processes, instantiated concretely by Markov kernels. Deterministic processes form a distinguished subcategory with lawful copying; stochastic processes compose sequentially and in parallel but cannot in general be copied without changing their meaning. Parametrized processes are represented through the categorical Para construction: an optimizable family from input X to output Y is a parameter object P together with a process $P \otimes X \rightarrow Y$. A candidate is a global element $I \rightarrow P$. This separates the structure of a system from the policy that searches its parameter space.

Real systems produce more than values. They can fail, cancel, disclose data, consume resources, choose fallbacks, and emit lineage. The thesis therefore lifts the process model to instrumented stochastic kernels

$$K : X \rightarrow D((Y + F + C) \times T \times R \times W),$$

where T is a structured semantic trace, R is an additive resource vector, W is a warning/fallback projection, and F and C represent attributable failure and cancellation. Extensional outcomes and intensional traces are retained separately. Two implementations that produce the same answer text need not be equivalent when one discloses unauthorized evidence, consumes a different release, falls back more often, or changes cost and latency.

Evaluation is modeled as a separate, product-owned morphism from cases and instrumented outcomes to observations. This is related to statistical games and Bayesian lenses but intentionally narrower: the shared core does not claim to know product utility. It provides exact experiment coordinates, hidden-label separation, common-random-number couplings, append-only custody, sufficient-statistic aggregation, ordered hard gates, noninferiority, and Pareto comparison. Adaptive search is another independent stochastic process over campaign histories. A proposer may be grid search, Bayesian optimization, an LLM, a human, or a hybrid; none is allowed to redefine system execution or evidence semantics.

The resulting architecture has four layers:

1. **system semantics**, describing typed deterministic and stochastic processes;
2. **evidence semantics**, describing workloads, couplings, evaluators, observations, and durable cells;
3. **decision semantics**, describing constraints, noninferiority, improvement, budgets, and partial orders;
4. **control semantics**, describing adaptive proposers and campaign state transitions.

Plugins are grafted at the boundaries of these layers. The preferred plugin is a typed factory with a pure, canonical $\text{Spec}()$ and a separate $\text{Bind}(\text{Environment})$ operation. The specification identifies meaning, effects, capabilities, and random namespace; binding supplies endpoints, credentials, stores, and clients. Executable plugin instances are not stored in an untyped service locator. A catalog stores descriptions and law certificates only. Out-of-process or dynamically discovered plugins are permitted at explicit schema-checked edges and are immediately adapted back into typed kernels.

A self-contained Go implementation accompanies the thesis. It uses only the standard library. The `finite` package implements exact rational finite stochastic matrices and checks category and Markov-category laws. The `core` package implements typed sampled kernels, canonical specifications, pathwise-stable splittable seeds, structured sequential/parallel traces, outcomes, and composition. `experiment` implements exact paired cells, common-random-number coupling, JSONL resume, and product-owned evaluators. `evidence` implements summaries, paired deltas, lexicographic gates, and Pareto fronts. `campaign` implements typed candidates, proposers, selectors, and adaptive history. `plugin` implements typed factories, effect policies, law certificates, and a deliberately explicit dynamic boundary. `ragtoy` grafts a complete miniature RAG domain onto the backbone, including chunking, representations,

lexical/vector retrieval, authorization-before-reranking, stochastic reranking, index/query parameters, evaluation, and resource trade-offs.

The executable demonstration certifies six exact finite-process laws, runs five paired RAG candidates over ten cases and five repeats, rejects an over-budget candidate, leaves a non-improving candidate undecided, computes a Pareto frontier, and promotes a bounded-rerank candidate. Unit tests, `go vet`, and the Go race detector pass. The implementation is not presented as a general measure-theoretic probability engine or a finished distributed optimization service. It is an executable reference model: small enough to understand and test, but strong enough to define the semantic contracts against which production adapters for `ragkit`, `ragopt`, GEC, RAG-TTC, and the TTC Garden assistant can be built.

Preface

This volume takes the optimization chapter of the preceding RAG semantics study and treats its mathematical premise as the main subject. The earlier chapter described optimization as a typed intervention over a dependency graph. That description corrected flat parameter sweeps, but it did not yet supply the abstract process theory needed to make arbitrary indexing, retrieval, answer, agent, and production evaluators compose under one small set of laws.

The objective here is not to replace ordinary Go interfaces with category-theory terminology. It is to identify the minimum structures that give interfaces meaning. The distinction matters. An interface can state that a reranker has a `Run` method. A semantic contract states its input and output objects, whether it is deterministic, which information it can disclose, how its randomness couples across arms, what trace it must preserve, and which laws a replacement must satisfy. The former is substitutable at compile time. The latter is substitutable in an experiment without destroying validity.

The thesis is both mathematical and constructive. Definitions are followed by API consequences. Laws are followed by executable checks. The finite exact interpreter is used where equality is decidable. The sampled interpreter is used where production adapters require effects and pseudorandomness. The RAG example is intentionally small but crosses the indexing/querying boundary so the implementation cannot evade the difficult part by optimizing only a ranking formula over frozen inputs.

Readers primarily interested in theory should read Parts II through V. Readers implementing a plugin system should begin with Parts VI and VII. Readers migrating the present repositories should read Parts VIII and IX. The appendices contain formal transition rules, laws, API listings, execution instructions, and the complete empirical mapping to the supplied source snapshot.

Principal contributions

1. A four-layer separation of system, evidence, decision, and adaptive-control semantics.
2. A Markov-categorical model of optimizable stochastic systems with deterministic copying and parallel composition.
3. A parametrized-process account of candidates and interventions that does not bind the architecture to gradient descent or any one search method.
4. An instrumented outcome semantics retaining failures, cancellation, traces, resources, warnings, and disclosure-relevant effects.
5. A coupled paired-experiment model that makes common random numbers and exact cell coordinates first-class.
6. A product-owned evaluator interface related to statistical games while preserving native artifacts as the diagnostic authority.
7. A constraint-first decision algebra with explicit noninferiority, budgets, and Pareto fronts instead of one universal scalar reward.
8. An adaptive campaign model as a stochastic state transition over evidence-visible histories.
9. A plugin doctrine separating pure specifications from runtime bindings, typed factories from dynamic edges, and catalogs from service locators.
10. An executable, standard-library-only Go reference implementation with exact and sampled interpreters and a complete toy RAG graft.
11. A migration path that strengthens the current **ragopt** experiment-custody model without moving RAG semantics into the generic optimization package.
12. A set of proof obligations, property tests, state-machine tests, and law certificates suitable for gradual formalization.

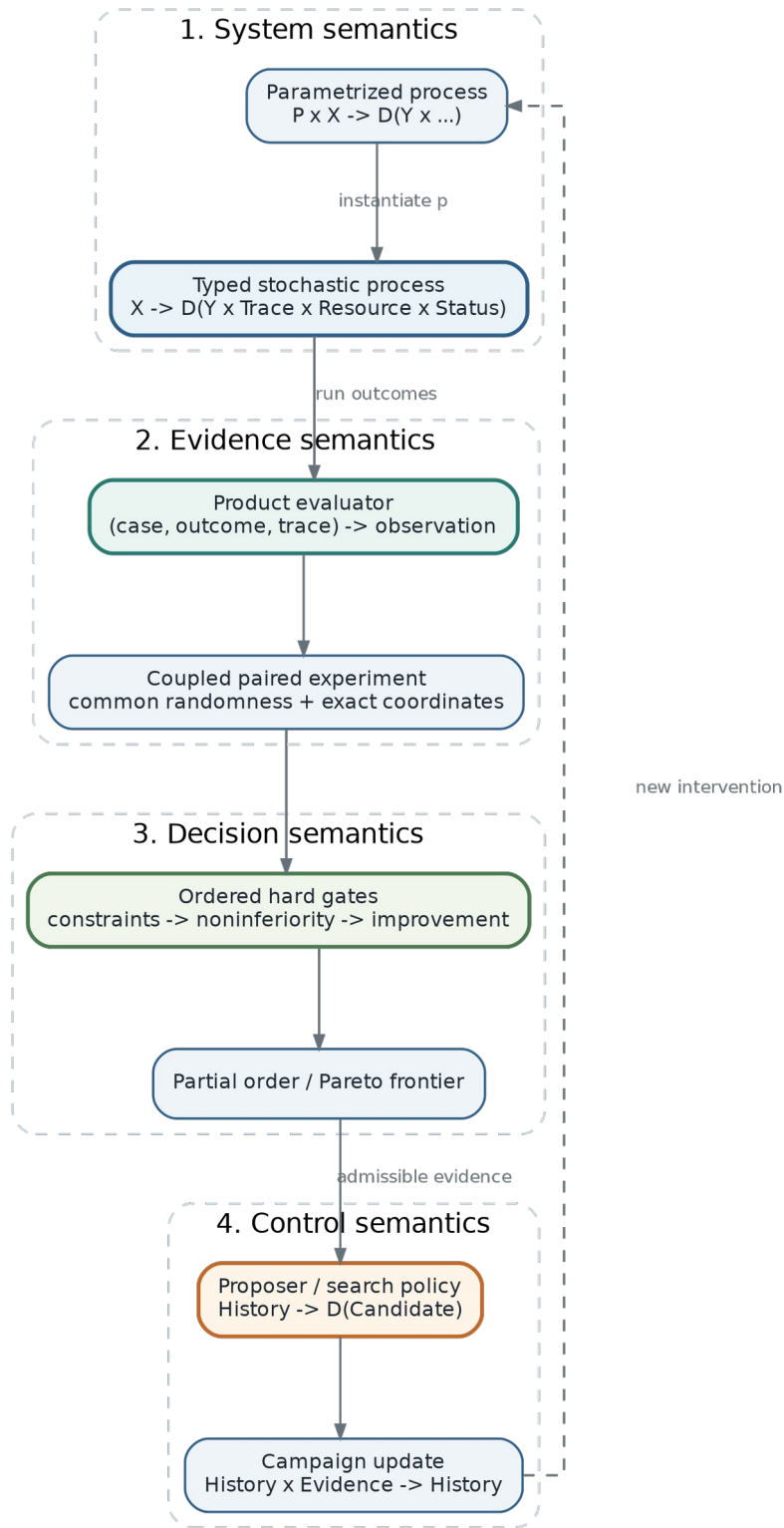
Notation

A symmetric monoidal category is written $(C, \otimes, I)$. Objects are types or interfaces. Morphisms are processes. Sequential composition is $g \circ f$ and parallel composition is $f \otimes g$. The category of finite sets and stochastic matrices is written $FinStoch$. The distribution object or finite distribution monad is written D . A deterministic map is distinguished from a general stochastic map.

A Markov category equips every object X with a copying morphism $\Delta_X: X \rightarrow X \otimes X$ and discarding morphism $!_X: X \rightarrow I$, satisfying commutative-comonoid laws. Discarding is natural for all processes in a causal Markov category. Copying is natural only for deterministic processes; demanding it for arbitrary stochastic maps would incorrectly duplicate a sample as though two independent samples were the same operation.

A parametrized process from X to Y is written (P, f) with $f: P \otimes X \rightarrow Y$. A concrete candidate $p: I \rightarrow P$ induces $f_p = f \circ (p \otimes id_X)$. A workload case is $c = (x, z, g)$, where x is visible input, z is hidden expected information, and g is a set of groups or tags. An observation is $o \in O$. A campaign history is $h \in H$.

The symbol $\approx$ denotes an explicitly declared observational equivalence, not necessarily equality of artifacts or execution schedules. A relation $\leq$ denotes a preorder or partial preference. Positive paired deltas are normalized to mean “better” after accounting for metric direction.



The proposed optimization backbone separates four semantic layers.

Part I. Problem statement and empirical setting

1. Optimization is a domain, not a loop

1.1 The usual abstraction is too small

A conventional optimization harness can be summarized as:

```
| candidate -> run -> score -> choose next candidate
```

This is sufficient when `run` is a pure deterministic function and `score` is a single total objective. Production RAG satisfies neither assumption. Its indexing path can call stochastic representation generators and embedding providers; its query path can execute channels concurrently, fail over, rerank remotely, filter by subject policy, and invoke a generator or agent; its evaluators can contain hidden relevance judgments, model judges, multi-turn interactions, and product-specific frontend assertions. Its outcomes are vectors with hard constraints rather than scalar rewards. Its corpus and release state can change between campaigns.

The problem is not simply that the loop needs more fields. The participating concerns have different algebraic roles. A stochastic component composes as a process. An evaluator consumes outcomes but should not become part of the system under test. A hard safety gate is not a differentiable loss. A proposer observes campaign history and chooses interventions, but it should not be able to reinterpret past cells. A runtime plugin can bind credentials, yet credentials should not change semantic identity unless they alter behavior.

When these roles are represented through one callback or registry, invalid substitutions become easy. A metric plugin can silently drop failures. A proposer can inspect holdout labels. A reranker wrapper can introduce an extra sample stream whose seeds depend on goroutine order. A cache can use an implementation name rather than a semantic specification. A dynamic plugin can claim compatible JSON while changing disclosure policy. Strong semantics begins by separating the roles.

1.2 Optimization as controlled intervention

An optimization field contains a family of systems, workloads, evaluators, decisions, and control policies. It should answer:

- What object is being varied?
- Which aspects of its behavior are stochastic?
- What does one candidate denote?
- Which comparisons are paired, and under what coupling?
- Which data is visible to the system, evaluator, and proposer?
- Which failures remain in the denominator?
- Which constraints precede preference?
- Which components may be substituted through plugins?
- What laws make such substitutions safe?
- How does an adaptive campaign change state over time?

The thesis treats these as semantic questions before they become orchestration questions.

1.3 The target architecture

The target is deliberately not a universal optimization framework. It is a small backbone with typed ports and multiple interpreters. Concrete fields—RAG, model prompting, ANN tuning, frontend calibration, job scheduling—graft domain objects and evaluators onto it. The common layer owns composition, identity, randomness, trace structure, exact experiment coordinates, evidence custody, and decision laws. It does not own the meaning of relevance, groundedness, customer value, or source authorization.

A good backbone should support simple static campaigns without ceremony and more sophisticated adaptive campaigns without changing the meaning of cells. It should support exact finite models for laws and sampled effectful models for production. It should be usable directly in Go rather than requiring a theorem prover at runtime. It should also leave a clear path to stronger formalization.

2. The supplied optimization architecture

2.1 Current `ragopt` strengths

The supplied `ragopt` package already establishes several unusually strong operational contracts. A candidate is an immutable parent/child snapshot pair with exactly one mutable asset changed. Locked assets and dimensions must remain identical. A paired run schedules one cell for every case, repeat, and arm. Product arms receive copied immutable inputs and write a native artifact. Ordinary arm failure is represented in the cell rather than dropped. The run store supports explicit resume and validates complete run identity. Comparison and gate packages remain separate from evaluation execution.

These properties should be preserved. They are not replaced by the categorical model; they become one operational realization of it.

2.2 Current semantic gaps

The present `Arm` interface is intentionally broad:

```
type Arm interface {
    Name() string
    Run(context.Context, Request) (Outcome, error)
}
```

Cases and product inputs are opaque JSON. Shared outcomes are a compact projection containing completion, contract validity, abstention, metrics, call counts, tokens, duration, and a native artifact. This is an effective integration boundary but not yet a compositional process model.

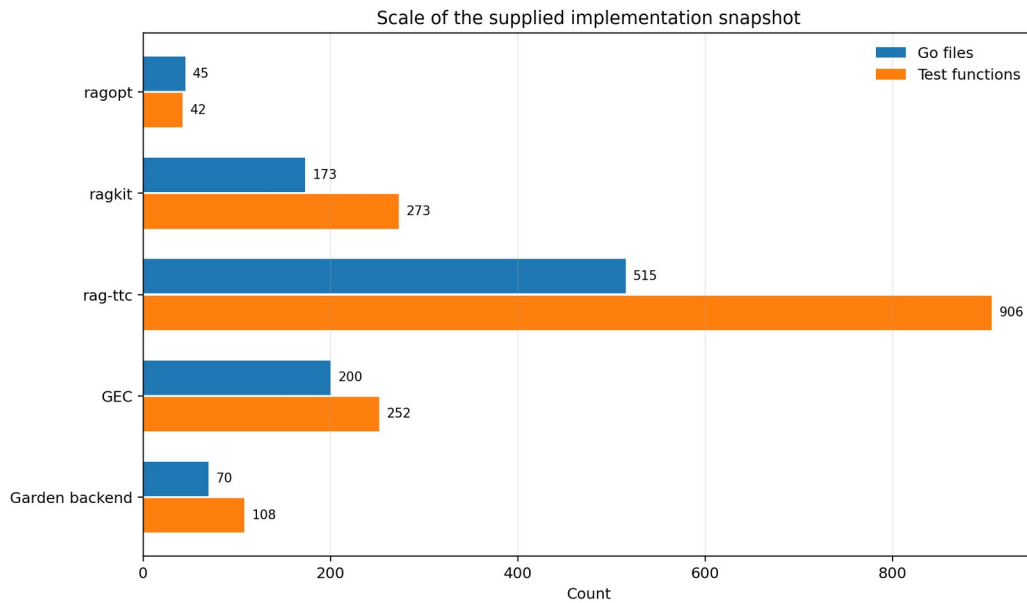
The interface does not declare:

- typed input and output ports;
- a pure semantic specification or canonical process identity;
- deterministic versus stochastic behavior;
- random namespaces or a baseline/candidate coupling;
- effects such as remote disclosure or state mutation;
- sequential and parallel composition laws;
- structured traces and fallback paths;
- capability requirements;
- an evaluator object distinct from the arm;
- a typed parameter space or dependency closure;
- an adaptive proposer state;
- metric schemas and direction beyond string maps.

The missing structures explain why `ragopt` is currently a disciplined *experiment custody* system rather than a complete optimization semantics. That is a sound boundary. The new backbone should be layered around it rather than turning `ragopt` into a RAG engine.

2.3 Empirical scale

The supplied snapshot contains 45 Go files in `ragopt`, 173 in `ragkit`, 515 in RAG-TTC, 200 in the GEC RAG scope, and 70 in the Garden backend scope. The corresponding test-function counts are 42, 273, 906, 252, and 108. The scale makes two conclusions relevant. First, a common architecture must remain smaller than the systems it coordinates. Second, plugin boundaries should follow semantics already demonstrated by multiple implementations rather than hypothetical generality.



Scale of the implementation snapshot reviewed for this study.

2.4 Chapter 21 as the starting point

The earlier RAG dynamics volume classified interventions across source admission, normalization, chunking, representation, embedding, indexing, rewriting, routing, fusion, reranking, context, answer policy, agent policy, serving, and presentation. It required every candidate to declare target nodes, dependency closure, semantic class, claimed invariants, and required evidence. That is the correct domain description.

The remaining question is how to make such a declaration compositional. A flat struct can list changed nodes, but it does not explain why two build stages compose, why an evaluator remains separate, why a paired trial is a valid coupling, or why a plugin replacement preserves a law. The rest of this thesis supplies that backbone.

3. Requirements and non-goals

3.1 Requirements

The architecture must satisfy the following requirements.

Typed boundaries. Internal composition should use ordinary static types. Dynamic schemas are restricted to explicit process boundaries.

Stochastic semantics. Randomness is part of meaning. Composition must be stable under reassociation and independent of goroutine schedule.

Intensional observability. Traces, fallbacks, disclosure, and resource usage remain visible rather than being compressed into a final score.

Product-owned evaluation. Domain evaluators retain native artifacts and hidden labels. The core aggregates a declared projection.

Exact pairing. Incumbent and candidate cells share case and repeat coordinates and may share an explicit coupling.

Constraint-first decisions. Safety, integrity, and budgets cannot be averaged away by quality gains.

Adaptive but auditable control. Search policies can be stochastic and stateful, while every proposed candidate and observed evidence remains immutable.

Plugin substitutability. Plugins declare effects, capabilities, and laws. Runtime binding does not erase semantic identity.

Dual interpretation. Small exact models and realistic sampled adapters share the same compositional vocabulary.

Incremental adoption. Existing ragopt arms and outcomes can be wrapped and projected during migration.

3.2 Non-goals

The reference implementation does not attempt to provide:

- arbitrary measurable-space probability or exact continuous distributions;
- gradients, automatic differentiation, or a tensor library;
- a distributed scheduler, worker lease service, or artifact store;
- a secure plugin sandbox for untrusted native code;
- universal metric definitions or product utility;
- automatic proof that a provider implements its advertised semantics;
- one scalar objective for every campaign;
- a general dependency-injection container;
- production-grade Bayesian optimization;
- a replacement for `ragkit`, `ragopt`, or product runtimes.

These omissions are architectural protections. They keep the core small and make its claims testable.

4. Four semantic layers

4.1 System semantics

System semantics answers what one component or composition does. Its central object is a typed stochastic process. It includes deterministic functions, random providers, stateful adapters represented through explicit state, sequential composition, parallel composition, failure, traces, and resources.

A RAG chunker, embedder, index builder, query rewriter, retriever, reranker, context builder, and agent step can each be represented at this layer. Product authorization policy can also be a typed process, but its decision meaning remains product-owned.

4.2 Evidence semantics

Evidence semantics answers how system behavior becomes experimental evidence. It introduces cases, hidden expected values, tags, seeds, couplings, evaluators, observations, cells, ledgers, aggregation, and paired deltas.

This layer is deliberately not the same as system semantics. If an evaluator is composed into the candidate system, a proposer might optimize against hidden labels through an accidental data path. Separating the evaluator makes visibility explicit.

4.3 Decision semantics

Decision semantics answers whether evidence makes a candidate eligible, rejected, undecided, preferred, or Pareto-nondominated. Hard constraints, noninferiority margins, improvement targets, and resource budgets have ordered roles. A decision policy is an auditable program, not a score formula hidden inside a search library.

4.4 Control semantics

Control semantics answers which candidate to evaluate next and how campaign history evolves. Static lists, grids, random search, Bayesian optimization, evolutionary search, LLM proposers, and human review are all control policies. They consume only the evidence and visibility allowed by the campaign protocol.

Separating control protects the meaning of completed trials. Changing the proposer does not change what a cell means, just as changing a scheduler should not change a pure function.

4.5 Why the split matters

A monolithic optimizer interface tends to collapse all four roles:

```
type Optimizer interface {
    Next(config, metrics) config
    Evaluate(config) metrics
}
```

Such an interface cannot state whether `metrics` include holdout information, whether `Evaluate` is stochastic, whether failure is missing data, or whether `Next` is allowed to change the workload. The four-layer model gives each responsibility its own composition and laws.

Part II. Categorical probability as the process backbone

5. Categories of processes

5.1 Objects and morphisms

A category consists of objects, morphisms, identity morphisms, and associative composition. For software architecture, the valuable move is to treat interfaces or value spaces as objects and executable transformations as morphisms. A morphism has one typed input boundary and one typed output boundary. Composition is permitted when the boundaries match.

This is not profound by itself. The strength comes from making the composition laws explicit. If $f: X \rightarrow Y$,

$g: Y \rightarrow Z$, and $h: Z \rightarrow W$, then

$$h \circ (g \circ f) = (h \circ g) \circ f.$$

The equality is semantic. An implementation may allocate different closures or generate different composite IDs, but it should preserve the denoted behavior under the declared observation relation. Identities must be neutral.

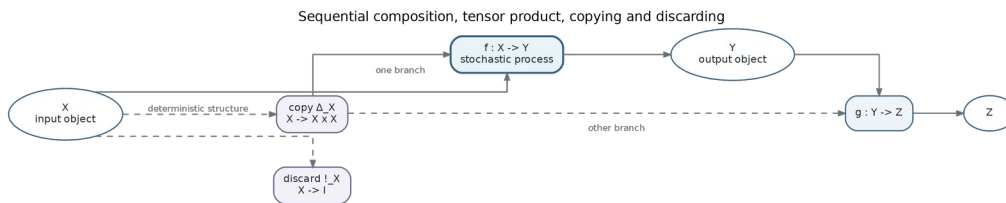
5.2 Symmetric monoidal composition

Optimization systems also combine independent components. A symmetric monoidal category supplies a tensor product:

$$f \otimes g: X \otimes X' \rightarrow Y \otimes Y'.$$

In typed software the tensor is commonly a product or record. Parallel composition does not necessarily require concurrent goroutines; it expresses independent wiring. An interpreter may execute the branches sequentially, concurrently, remotely, or symbolically while preserving the same semantic trace and result relation.

Symmetry allows the branches to be exchanged. Associativity allows nested products to be reassociated. These laws make diagrams and plugin graphs compositional without requiring one global pipeline type.



Sequential, parallel, copy, and discard structure in the process category.

5.3 Deterministic process category

Pure Go functions form the simplest process category. The reference implementation lifts functions into kernels with explicit domain failure. This deterministic subcategory is important because only deterministic values can be copied lawfully in the Markov sense.

A chunker with fixed specification and input document can be deterministic. A remote summary generator is not, even at temperature zero unless the provider gives a strong deterministic version contract. A cached retained summary can be treated as deterministic material after its artifact identity is fixed. The distinction belongs in the specification.

6. Markov categories

6.1 Why probability requires more than a monad in the API

A stochastic process from X to Y can be represented as a Markov kernel $X \rightarrow D(Y)$. Sequential composition integrates over intermediate outcomes. In finite systems, this is matrix multiplication of row-stochastic matrices. A probability monad gives one construction of this category through a Kleisli category.

The Markov-category perspective adds explicit copy and discard structure to each object and distinguishes deterministic morphisms. This is useful for optimization architecture because workloads, parameters, and observed values are copied and routed frequently, while stochastic effects must not be duplicated casually.

6.2 Copying a value versus resampling a process

Suppose $f : X \rightarrow Y$ is stochastic. Two constructions differ:

1. run f once, copy the resulting Y ;
2. copy X , run f independently on both copies.

The first yields perfectly correlated outputs. The second may yield independent outputs. Treating them as equal would destroy the meaning of coupling and repeats. Therefore copying is natural for deterministic maps but not for arbitrary stochastic maps.

This distinction directly informs the `Fanout` combinator in the sandbox. It copies the deterministic input and invokes two kernels. It does not claim to copy a stochastic sample invisibly.

6.3 Discard and causality

Discarding an output means ignoring it. For causal stochastic processes, discarding after the process is equivalent to discarding the input:

$$!_Y \circ f = !_X.$$

In exact finite probability this states that each row sums to one. A subprobability process that can disappear would violate the equation unless nontermination or failure is represented explicitly. The sandbox therefore represents failure and cancellation as outcome variants rather than silently losing probability mass.

6.4 Finite exact model

The `finite` package implements distributions using exact rational probabilities. A finite kernel is a function from an input value to an exact distribution. Composition multiplies and sums rational mass. Tensor forms product distributions. Copy and discard are deterministic kernels.

The exact interpreter checks:

- left identity;
- right identity;
- associativity;
- discard naturality;
- copy naturality for deterministic kernels;
- commutativity of copy.

The exact model is small but important. It distinguishes algebraic law failure from floating-point or sampling error. It also provides a canonical place to test future extensions before binding them to operational effects.

6.5 Generality and limits

Markov categories are not limited to finite stochastic matrices. The literature develops them as a uniform abstract language for discrete, measure-theoretic, Gaussian, and other probabilistic settings. The thesis uses that abstraction as design guidance but the implementation commits only to finite exact and seeded sampled interpreters. No claim is made that arbitrary measurable disintegration or conditioning is implemented.

7. Typed sampled kernels

7.1 Denotation

The primary operational interface is:

```
type Kernel[A, B any] interface {
  Spec() Spec
  Run(context.Context, Request[A]) (Outcome[B], error)
}
```

The intended denotation is not the Go method itself. It is a probability kernel

$$\llbracket K \rrbracket : A \rightarrow D((B + F + C) \times T \times R \times W).$$

Run returns one sample of that denotation, subject to the runtime environment and seed protocol. A Go error is reserved for failure of the interpreter contract—corrupt binding, impossible nonterminal output, serialization failure—not ordinary product failure. Product failure is data in Outcome and therefore enters evaluation.

7.2 Outcome structure

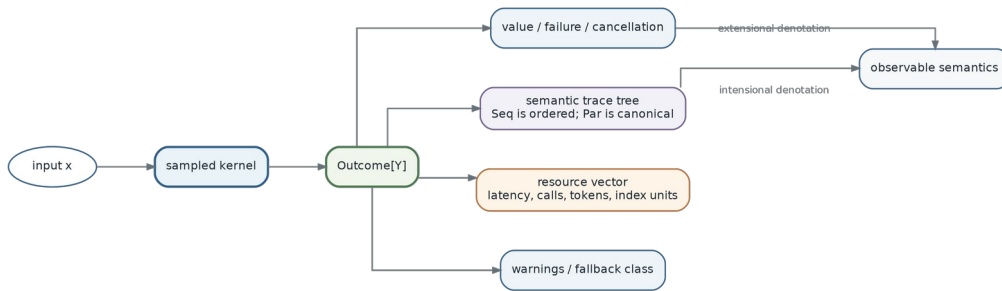
An outcome contains:

- an optional typed value;
- completed, failed, or cancelled status;
- attributable failure class and message;
- warnings;
- a structured trace;
- a finite resource vector.

The status is terminal. A completed outcome must have a value. Failed and cancelled outcomes do not. Resources must be finite and nonnegative under their declared semantics. This prevents NaN scores or negative token counts from silently contaminating summaries.

7.3 Instrumented composition

Sequential composition short-circuits domain failure but retains preceding trace and resources. Parallel composition combines branch resources additively and builds a canonical parallel trace. The semantic trace records process structure, not wall-clock scheduling.



An instrumented stochastic kernel retains extensional and intensional behavior.

7.4 Why trace is part of meaning

Consider two RAG candidates that return the same ranked chunks. One performs authorized local retrieval. The other sends a larger unfiltered pool to a remote reranker and removes restricted chunks afterward. Extensional result equality hides a security difference. Similarly, two answers can match while one used a fallback after a provider timeout. Optimization should be able to constrain such differences.

Trace need not expose sensitive payloads. It can contain typed event identities, counts, policy certificates, and artifact references. The key is that the trace schema and equality projection are declared rather than improvised from logs.

8. Structured trace algebra

8.1 Two forms of composition

The sandbox trace type has four constructors:

```
Empty
Event(label, spec, fields)
Seq(t1, ..., tn)
Par(t1, ..., tn)
```

Seq is associative with Empty as unit and preserves order. Par is associative, commutative after canonical sorting, and has Empty as unit. Nested nodes are flattened. This normalization makes reassociation invisible at the trace level.

The design intentionally does not assert a full interchange law between sequential and parallel trace operations. Real process graphs can have causal dependencies, and a naive interchange equation can identify distinct structures. A future implementation may use a richer free symmetric monoidal category or duoidal trace object; the reference model keeps only the laws it tests.

8.2 Semantic versus operational trace

Wall-clock timestamps, goroutine IDs, machine names, and retry backoff belong in operational artifacts. They are not part of schedule-independent semantic trace. This separation allows two executions to be semantically equivalent while having different scheduling details, yet keeps the operational evidence available for latency and incident analysis.

Trace.Digest() canonically hashes the semantic tree. A product can define a projection that ignores selected events for a particular equivalence claim, but the underlying trace is retained.

8.3 Trace laws as plugin obligations

A plugin must not discard the trace of its wrapped process. A fallback wrapper appends a fallback event. A cache wrapper records hit or miss under the same semantic operation identity and must prove that a hit returns an observationally equivalent value. A remote adapter records disclosure class and provider identity without embedding secret text.

These rules turn tracing from best-effort instrumentation into an intensional contract.

9. Randomness and couplings

9.1 The associativity trap

A naive sampled implementation splits one RNG at each composite node. Then the random seed reaching a leaf depends on parenthesization:

```
| (f >> g) >> h
| f >> (g >> h)
```

can produce different draws even though category associativity says they denote the same composite. The difference is not merely theoretical. Refactoring a pipeline can change experiment results.

The sandbox avoids this by treating the request seed as an immutable root and deriving each stochastic leaf stream from a stable random namespace plus local labels. Composite nodes pass the root through unchanged. Reassociation therefore leaves leaf draws pathwise equal.

9.2 Splittable seeds

A seed is a 256-bit immutable value. Derivation hashes the parent seed and length-delimited labels. A component-local stream is:

$$seed_{k,\ell} = H(\text{root}, \text{namespace}(k), \ell).$$

Distinct stochastic stages use distinct namespaces. Runtime scheduling cannot change the derived stream. The scheme is reproducibility infrastructure, not cryptographic randomness.

9.3 Common random numbers

Incumbent and candidate implementations can deliberately share a random namespace. When the same case/repeat seed reaches both, provider noise is coupled. This common-random-number coupling can reduce variance in paired differences.

The coupling is explicit. Sharing a namespace asserts that the stochastic source represents the same latent disturbance across arms. It is invalid when candidate behavior changes the interpretation or number of draws in a way that breaks alignment. Plugins should declare coupling compatibility rather than having a runner guess from names.

9.4 Couplings as first-class evidence design

For baseline kernel K_b and candidate kernel K_c , a paired experiment needs a joint kernel

$$\Gamma : X \rightarrow D(O_b \times O_c)$$

whose marginals are K_b and K_c . Independent execution uses the product coupling. Common random numbers use a correlated coupling. Replay against retained provider responses can use an even stronger deterministic coupling.

A paired delta has meaning only relative to Γ . The experiment artifact should identify it.

10. Parametrized processes

10.1 The *Para* idea

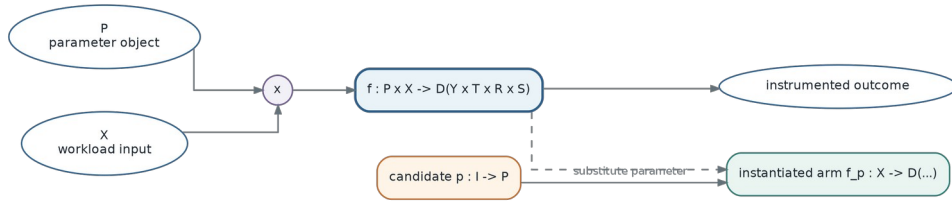
An optimizable system is not one arrow but a family. The categorical construction of parametrized maps represents a process from X to Y as a pair (P, f) :

$$f : P \otimes X \rightarrow Y.$$

Sequential composition tensors the parameter objects. If $(P, f) : X \rightarrow Y$ and $(Q, g) : Y \rightarrow Z$, the composite has parameters $Q \otimes P$ and process

$$Q \otimes P \otimes X \xrightarrow{id_Q \otimes f} Q \otimes Y \xrightarrow{g} Z.$$

The exact categorical presentation is often bicategorical because parameter reparametrizations are meaningful 2-cells. The software architecture needs only the practical consequence: each component owns a typed parameter object, and composite systems obtain a structured product of parameters rather than one flat map of strings.



A candidate instantiates a parametrized process.

10.2 Candidates as global elements

A candidate is a chosen parameter value $p : I \rightarrow P$. Substituting it yields a concrete arm:

$$f_p = f \circ (p \otimes id_X).$$

The parameter value is immutable and canonically identified. Its identity includes values that alter denotation and excludes execution-only details such as worker count when worker count is claimed not to alter behavior.

The current ragopt exactly-one-mutation rule is a restricted but valuable form of this idea. It constrains candidate transitions so causal interpretation remains tractable. The new model generalizes the parameter object while retaining the ability to require one declared intervention per campaign.

10.3 Structured RAG parameter object

A RAG release can factor its parameter space as:

$$P = P_{source} \otimes P_{normalize} \otimes P_{chunk} \otimes P_{repr} \otimes P_{embed} \otimes P_{index} \otimes P_{query} \otimes P_{rerank} \otimes P_{context} \otimes P_{answer} \otimes P_{agent} \otimes \dots$$

Not every campaign exposes every factor. A fusion campaign varies P_{query} while locking the others. A chunking campaign varies P_{chunk} and triggers a dependency closure through representations, embeddings, indexes, retrieval, answers, and sessions. Typed factorization makes those relationships inspectable.

10.4 Reparametrization

A user-facing search space can map into a lower-level parameter space. For example, a `quality_tier` enum may choose channel depths, rerank pool, and context budget. This is a reparametrization $r : P' \rightarrow P$. It is useful for plugin APIs because a product can expose a safe constrained space while the system implementation retains its full specification.

Reparametrization must be identified and versioned. Changing its mapping changes candidate meaning even when the high-level value is unchanged.

Part III. Evidence, decisions, and adaptive control

11. Evaluation as a separate morphism

11.1 Cases have visible and hidden components

A benchmark case should not be represented as one value passed indiscriminately to every component. Let

$$c = (x, z, g),$$

where x is the system-visible input, z is hidden expected information, and g is product-owned grouping metadata. The system under test receives x . The evaluator receives the complete case and the system outcome. The proposer normally receives only aggregated evidence permitted by the campaign visibility policy.

This separation prevents several forms of leakage. A retriever cannot inspect expected document IDs. A generation prompt cannot adapt to the judge rubric. A proposer cannot overfit a hidden holdout by reading per-case failures unless the protocol explicitly permits it.

The sandbox encodes Expected as opaque `json.RawMessage` inside a typed `Case[X]`. The system kernel is invoked only with Input x ; the evaluator receives the Case after execution. Product code decodes the expected data.

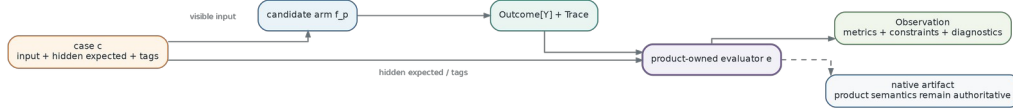
11.2 Evaluator denotation

For system output object Y , trace object T , and case object C , an evaluator has the conceptual form

$$e : C \otimes (Y + F + C) \otimes T \otimes R \rightarrow D(O \times A),$$

where O is the shared observation projection and A is a product-native artifact or artifact reference. The evaluator can itself be stochastic, as with a model judge. Its random namespace and repeated evaluations must therefore be represented explicitly.

The evaluator is not a utility function embedded in the system. It is an external observation process. This matches the intuition of statistical games: a model or process is paired with a loss or fitness context. The thesis uses the connection as guidance while retaining simpler software boundaries.



A product evaluator consumes hidden expectations and an instrumented outcome.

11.3 Native artifact authority

A shared observation may contain metrics such as recall, MRR, groundedness, latency, or provider calls. It cannot contain every diagnostic fact. Product evaluators should write a native artifact that retains channel rankings, citations, transcript, judge rationale, widget lineage, or source revisions as appropriate.

The shared projection is for comparison. The native artifact is for diagnosis and audit. A generic optimization package should never force product evidence into one universal schema and discard the remainder.

11.4 Evaluator plugins

An evaluator plugin differs from a system plugin:

- it may access hidden labels;
- it must identify its rubric, judge, prompt, and aggregation semantics;
- it may need blind arm labels to avoid bias;
- it emits observations rather than product outcomes;
- its stochastic repeats can differ from system repeats;
- it must not mutate the system or candidate.

These differences justify a distinct interface and law set.

12. Observations and sufficient statistics

12.1 Observation schema

The sandbox observation contains:

```
type Observation struct {
  Metrics      map[string]float64
  Constraints  map[string]bool
  Tags         []string
  Diagnostics  map[string]string
}
```

This is intentionally modest. Production code should attach a metric schema declaring direction, unit, boundedness, aggregation, missingness policy, and protected strata. The map is retained in the sandbox to make adapters easy and to demonstrate that stronger semantics can surround a pragmatic representation.

12.2 Metrics and constraints are not interchangeable

A metric is a measured quantity. A constraint is a proposition that must be evaluated under a declared quantifier. “No unauthorized disclosure” should not be encoded as a metric with a large negative weight. A single violation can be disqualifying even when average relevance improves.

Likewise, contract validity is often a constraint; latency is usually both a metric and a budget; relevance is usually a metric with noninferiority and improvement requirements. The decision layer interprets these roles.

12.3 Mergeable summaries

For each metric the sandbox records count, mean, sample standard deviation, minimum, and maximum. These can be computed from mergeable sufficient statistics $(n, \sum x, \sum x^2, \min x, \max x)$. Constraint summaries record seen, passed, and failed counts.

Mergeability matters for distributed evaluation and stratification. It does not solve statistical dependence. Repeated outcomes from one conversation, source, or user may need clustered analysis. The artifact must retain cell coordinates so a stronger analyzer can reconstruct the correct sampling unit.

12.4 Missingness

A failed cell is not an absent cell. It should produce an observation with failure class and whatever constraints can be determined. A truly missing cell means the experiment is incomplete and cannot be paired.

Metric absence can be legitimate—for example, reciprocal rank when the evaluator cannot decode a result—but it must be represented explicitly. Aggregators should not silently reduce the denominator. The current ragopt design already follows this principle by retaining failure outcomes and exact expected cell counts.

13. Exact paired experiments

13.1 Cell coordinates

A paired trial is indexed by:

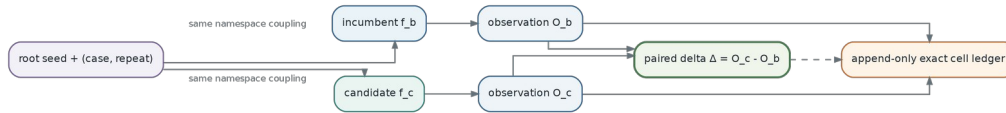
$$(run, case, repeat, role).$$

For each case and repeat there must be exactly one incumbent cell and one candidate cell. Candidate ID, system spec ID, evaluator spec ID, seed, timestamps, status, trace, resources, observation, and native output are recorded.

The ledger key excludes candidate ID because the run configuration fixes which candidate occupies each role. Reusing one run directory for a different candidate is invalid. Production run identity should include the complete campaign and release specifications.

13.2 Common randomness

The root run seed derives one cell seed from run, case, and repeat coordinates. The same seed is supplied to both roles. Component-local random namespaces then determine whether corresponding stochastic leaves are coupled.



A paired experiment uses one explicit coupling and retains both cells.

The sandbox test constructs two noisy kernels with the same random namespace and a fixed one-unit bias. Every paired difference is exactly one, demonstrating pathwise common-random-number coupling. Independent random namespaces would preserve unbiasedness under suitable assumptions but increase variance.

13.3 Coupling compatibility

Not all candidate changes support the same coupling. If the baseline makes one provider call and the candidate makes three adaptive calls, assigning call index one to the same latent draw may be reasonable while the later draws have no counterpart. If the candidate changes the provider model, shared pseudorandom seed may not imply shared latent noise at all.

A robust plugin contract should declare a `CouplingClass`, such as:

- deterministic/replayable;
- common input noise;
- common provider response artifact;
- aligned sequence by semantic call ID;
- independent only.

The paired runner then validates the requested coupling against both arms.

13.4 Order and temporal drift

Exact pairing does not require executing baseline and candidate simultaneously, but execution order can interact with remote model drift, load, caches, and mutable dependencies. A production runner should choose and record a schedule: alternating roles, randomized within pair, concurrent, or artifact replay. The schedule is operational evidence. The semantic comparison claims must state which environmental variation is controlled and which is averaged.

13.5 Resume

The JSONL ledger loads completed cells and executes only missing coordinates. Duplicate cell keys are rejected, including after reopening the file. A distributed production ledger additionally needs a lease or fencing token; the sandbox explicitly leaves cross-process write coordination outside the local file format.

Resume correctness requires that the run configuration, inputs, system specs, evaluator spec, and seed protocol remain identical. “Resume” after changing code without changing spec identity is semantic corruption.

14. Paired deltas and statistical interpretation

14.1 Direction-normalized differences

For metric m with candidate value m_c and baseline value m_b , the raw paired difference is

$$\Delta_m = m_c - m_b.$$

For a minimize metric such as latency, the beneficial difference is $m_b - m_c$. The gate layer normalizes direction so positive always means better.

The sandbox computes descriptive means and standard deviations of paired deltas. It does not claim formal confidence intervals. A production analyzer can add paired bootstrap, randomization tests, hierarchical models, or sequential methods while consuming the same immutable cells.

14.2 Why pairing matters

Query cases differ greatly in difficulty. Comparing two independent sample means leaves case difficulty in the variance. Pairing subtracts outcomes on the same case and repeat. Common random numbers can also subtract shared stochastic disturbance.

The benefit is conditional on sound pairing. Dropping a failed candidate cell while retaining its baseline counterpart creates survivorship bias. Replacing a missing candidate cell with zero changes the estimand. Exact coordinates and explicit failure prevent both.

14.3 Adaptive bias

When a proposer observes results and chooses the next candidate, naive confidence intervals over the winning candidate can be optimistic. Reusing the same holdout repeatedly leaks information through the campaign history. The semantic architecture can represent visibility but does not magically solve adaptive inference.

Recommended controls include:

- separate development, validation, and final holdout partitions;
- limited disclosure from holdout to proposer;
- preregistered hard gates;
- fresh future or temporal holdouts;
- nested evaluation for highly adaptive search;
- sequential inference methods when peeking is expected;
- human review of the complete trial history, not only the winner.

Visibility must be part of campaign state and artifact lineage.

15. Decision semantics

15.1 Three-valued verdicts

The sandbox uses Eligible, Rejected, and Undecided.

- **Eligible** means every hard rule passes and every soft rule considered so far passes.
- **Rejected** means a hard rule fails.
- **Undecided** means hard rules pass but one or more soft improvement conditions are not established.

This avoids forcing “not proven better” into “bad” and distinguishes it from a safety failure.

15.2 Ordered gates

A policy is an ordered list of rules. Evaluation stops at the first failed hard rule. A representative RAG order is:

1. authorization and disclosure constraints;
2. artifact, trace, and contract integrity;
3. complete paired coverage;
4. noninferiority on protected relevance and answer strata;
5. target improvement;
6. latency, cost, and capacity budgets;
7. tie-breaking preferences.

The order is semantic. Moving cost before security would be nonsensical. A plugin system should therefore identify the policy program, not merely the set of metrics.

15.3 Constraint rule

ConstraintAll requires that a named candidate constraint is observed and has zero failures. A production policy may add quantified variants:

- zero failures globally;
- failure rate below a bound;
- zero failures in protected groups;
- per-case invariant;
- confidence bound on a Bernoulli rate.

The rule name and quantifier belong in identity.

15.4 Noninferiority

A noninferiority rule requires beneficial paired mean delta at least $-\epsilon$:

$$\overline{\Delta}_m \geq -\epsilon.$$

The sandbox uses means directly. Production should generally apply a lower confidence bound or another declared decision statistic. Noninferiority margins should be product decisions, not inferred from candidate results.

15.5 Improvement and budget

An improvement rule requires a minimum beneficial delta. A budget rule applies an absolute upper bound to a candidate metric such as latency, provider calls, or index units. Budgets are not equivalent to comparison against the baseline; both can be useful. A candidate may improve latency while still violating an operational ceiling.

16. Partial order and Pareto fronts

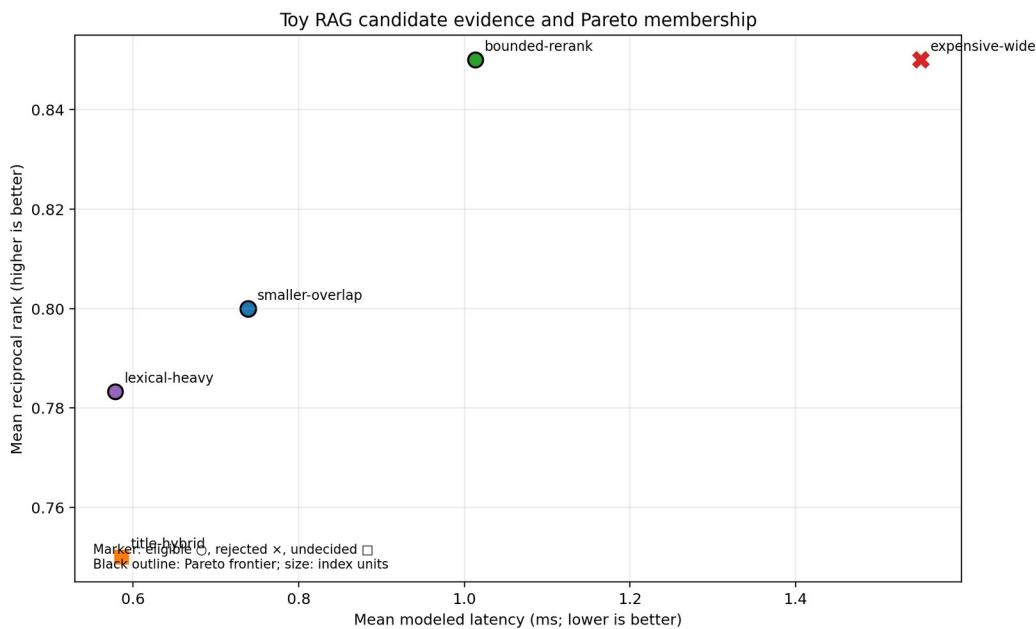
16.1 No universal scalar

Optimization libraries often demand one objective. RAG candidates trade relevance, grounding, latency, cost, freshness, index size, update amplification, disclosure, and interaction quality. A weighted sum encodes one policy and can allow large gains to hide hard regressions.

After hard eligibility gates, a Pareto preorder is often more honest. Candidate a dominates b when it is at least as good in every selected metric and strictly better in at least one, after direction normalization.

16.2 Frontier computation

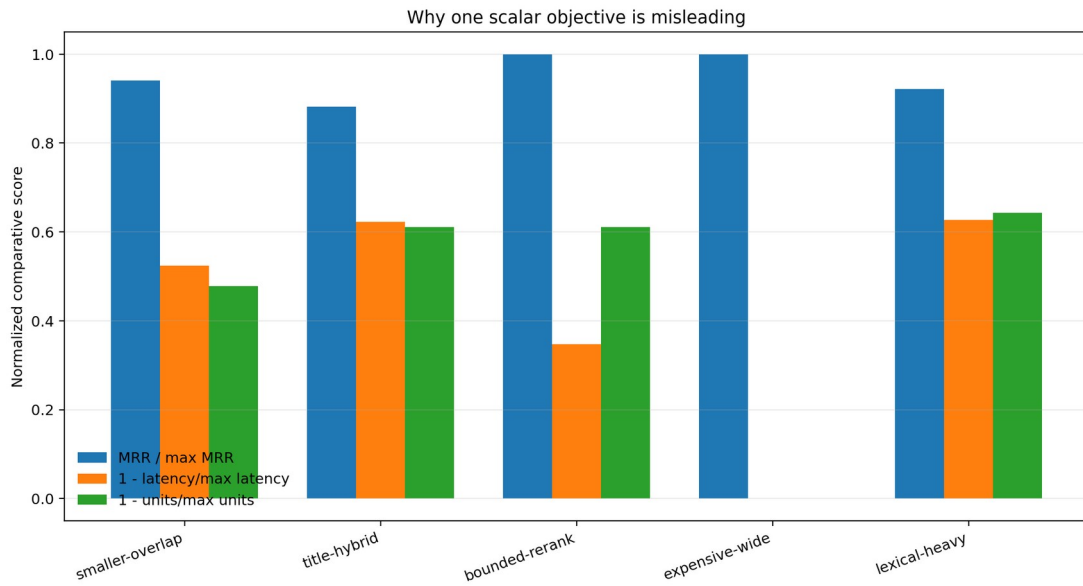
The sandbox computes the nondominated set over declared metric directions. The demonstration produces three Pareto candidates: bounded reranking, lexical-heavy retrieval, and smaller overlapping chunks. Each represents a different quality/resource trade.



The toy RAG candidates occupy a quality, latency, and index-cost frontier.

16.3 Selection remains policy

Pareto membership does not choose a winner. A selector can choose the highest MRR among eligible candidates, the cheapest candidate above a quality threshold, or require human review. The selection rule is a decision plugin with a pure spec.



Normalizing dimensions into one chart illustrates why a single score hides trade-offs.

16.4 Preorders and incomplete information

Missing metrics, wide uncertainty, or incomparable protected groups can make preference partial even beyond Pareto structure. The architecture should permit Undecided rather than inventing a total order. Search algorithms that require scalar rewards can operate on a surrogate projection, but promotion still uses the full decision semantics.

17. Adaptive campaigns as stochastic processes

17.1 Campaign state

A campaign history contains:

- initial and current incumbent;
- immutable trial records;
- visible summaries and verdicts;
- candidate provenance and parent links;
- workload/evaluator identities;
- visibility budget and holdout-use history;
- optional proposer state.

A proposer is conceptually a kernel

$$\pi: H \rightarrow D(P + \text{Stop}),$$

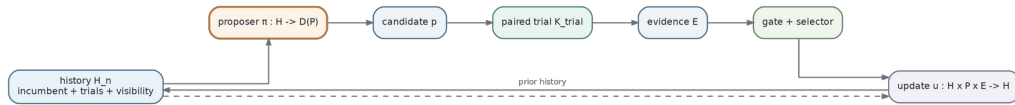
where P is the candidate parameter object. A deterministic static list is one instance. Bayesian optimization, evolutionary mutation, an LLM proposer, or a human queue are others.

17.2 One campaign step

Let $K_{\text{trial}}: H \otimes P \rightarrow D(E)$ execute and summarize a paired trial. Let $u: H \otimes P \otimes E \rightarrow H$ append the immutable evidence and update the incumbent under a selector. One adaptive step is the composite Markov kernel

$$H \xrightarrow{\Delta_H} H \otimes H \xrightarrow{id \otimes \pi} H \otimes P \xrightarrow{K_{\text{trial}}} H \otimes P \otimes E \xrightarrow{u} H.$$

Iteration yields a stochastic state process over campaign histories.



An adaptive campaign is a stochastic state transition over immutable history.

17.3 Control does not own evidence

The proposer receives a view of history; it does not mutate past cells. The selector produces a new incumbent pointer; it does not rewrite candidate evidence. This event-sourced structure permits replay with a different selector or audit of what the proposer knew when it made a choice.

17.4 Stopping

Stopping can occur because:

- the proposer returns `Stop`;
- maximum rounds or budget is reached;
- no eligible candidate is found;
- a target threshold is achieved;
- holdout visibility is exhausted;
- an operator cancels the campaign.

The stop reason is a terminal campaign event. It should not be inferred from the absence of a next row.

18. Visibility and information-flow discipline

18.1 Four visibility classes

A useful campaign partitions evidence into:

- **training/development**, visible at case level to proposers and engineers;
- **validation**, visible as controlled summaries;
- **holdout**, used for gates with limited or blinded disclosure;
- **canary/production**, observed only after offline eligibility.

The classification is not just metadata. It constrains morphisms: the proposer cannot have a data path from hidden holdout labels if the campaign claims holdout validity.

18.2 Information-flow graph

A typed implementation can model history views separately:

```
type ProposerView struct { /* permitted summaries */ }
type ReviewerView struct { /* richer diagnostics */ }
type HoldoutCustodian interface { Evaluate(candidate ID) GateArtifact }
```

The custodian can emit a gate result without exposing per-case labels. This is a software analog of controlling sufficient statistics and observations in categorical probability.

18.3 Adaptive overfitting as state

Every holdout query consumes information. A visibility ledger records which candidate, aggregate, slice, and explanation was revealed. A fresh holdout or temporal cohort can be required after a budget is exhausted.

The core cannot prove that humans did not memorize results, but it can prevent unrecorded programmatic access and make the remaining assumption explicit.

19. Statistical games and Bayesian lenses

19.1 Why these theories are relevant

Categorical work on parametrized maps and lenses separates a forward process from backward information used to update parameters. Statistical games attach objectives to statistical models and study compositional approximate inference. Open games model components relative to an environment and compose sequentially and monoidally.

Optimization architecture has a similar shape: a system produces outcomes forward; an evaluator produces evidence; a controller uses evidence to choose parameters. The resemblance is structural, not an assertion that a RAG campaign is literally one canonical open-game construction.

19.2 A deliberately conservative use

The thesis borrows three ideas:

1. parametrized processes should be first-class rather than encoded as string maps;
2. evaluative or backward structure should remain distinct from forward execution;
3. composite components can expose local fitness/evidence interfaces while participating in a larger system.

It does not implement general optics, Bayesian inversion, equilibria, or variational inference. Those are research directions once the simpler process/evidence/control split is stable.

19.3 Local evaluators and global utility

A RAG graph may have local evaluators:

- chunk stability and source coverage;
- ANN recall against an exact oracle;
- retrieval relevance;
- answer grounding;
- agent task completion;
- frontend provenance;
- production latency and cost.

These local observations compose into a structured evidence object, not necessarily an additive loss. Hard constraints and dependencies prevent arbitrary local optimization. The statistical-game perspective motivates keeping the evaluator interface compositional while the decision layer governs the final order.

20. Refinement and equivalence

20.1 Equality is too strong and too weak

Byte equality is too strong for stochastic systems and operational schedules. Final-answer equality is too weak because it erases trace, policy, cost, and failure behavior. The architecture needs named observational relations.

20.2 Exact equivalence

Exact finite denotational equivalence requires equal output distributions for every input. Instrumented exact equivalence additionally requires equal distributions over outcome, trace, resources, and status.

The finite interpreter can decide equality for small finite kernels with rational probabilities.

20.3 Pathwise equivalence

Under one coupling and root seed, two sampled implementations can be pathwise equivalent if their observable outcomes and canonical traces match for every tested input/seed. The sandbox uses this stronger check for composition reassociation. Pathwise equivalence is useful for refactors and cache/replay adapters.

20.4 Distributional equivalence

Two implementations may differ per seed but induce the same distribution. Establishing this empirically requires statistical tests and cannot be proved by the sampled reference implementation alone. A plugin certificate must state whether its law is exact, pathwise under a seed protocol, or empirically distributional.

20.5 Refinement

A candidate K' refines K under relation $\sqsubseteq$ when it preserves hard observations and improves or bounds selected dimensions. For example:

- no new disclosure events;
- same authorized evidence support;
- noninferior relevance;
- lower resource vector;
- same terminal contract.

Refinement is policy-relative. It is more useful for production substitution than unqualified “compatibility.”

Part IV. The compositional optimization backbone

21. The abstract optimization field

21.1 Definition

An optimization field over a symmetric monoidal Markov category $\mathcal{C}$ consists of the following data.

1. A parameter object P .
2. A workload-visible input object X .
3. A system-output object Y .
4. An instrument object $J(Y) = (Y + F + C) \otimes T \otimes R \otimes W$.
5. A parametrized system process

$$S : P \otimes X \rightarrow J(Y).$$
6. A case object C with a visibility projection $v : C \rightarrow X$.
7. An evaluator

$$E : C \otimes J(Y) \rightarrow O,$$
 possibly stochastic and therefore represented in $\mathcal{C}$.
8. A coupling family that lifts two instantiated systems to paired outcome processes.
9. An aggregation algebra $\alpha : O^i \rightarrow A$ over finite cell collections or streams.
10. A decision program $G : A_b \otimes A_c \otimes A_\Delta \rightarrow D$, where D contains at least eligible, rejected, and undecided outcomes.
11. A campaign-history object H .
12. A proposer $\pi : H \rightarrow P + Stop$ or stochastic $H \rightarrow D(P + Stop)$.
13. A history update process $u : H \otimes P \otimes A \otimes D \rightarrow H$.
14. Visibility and capability policies restricting which morphisms can be composed.

This definition is intentionally modular. It does not require a differentiable parameter space, a scalar loss, or an optimizer that can inspect the internals of S . It supports black-box and white-box optimization, exact and sampled evaluators, static and adaptive proposals, and product-native evidence.

21.2 One trial as composition

For candidate $p : I \rightarrow P$, the instantiated system is $S_p : X \rightarrow J(Y)$. The case projection supplies visible input. Copying the case is deterministic, so the evaluator can retain hidden information while the system receives only $v(c)$:

$$C \xrightarrow{\Delta_C} C \otimes C \xrightarrow{id_C \otimes v} C \otimes X \xrightarrow{id_C \otimes S_p} C \otimes J(Y) \xrightarrow{E} O.$$

This diagram is the semantic core of one cell. The operational runner adds identifiers, time, durable custody, and native artifact paths without changing the evaluation relation.

21.3 One paired trial

For incumbent p_b and challenger p_c , select a coupling Γ over the two system processes. The paired evaluator produces (o_b, o_c) at one case/repeat coordinate. A deterministic delta process maps them to δ . The cell ledger stores both observations before aggregation.

The architecture does not force one coupling. Independent product, common-random-number, retained-response replay, and semantic-call-aligned couplings are all possible plugins satisfying marginal laws.

21.4 One campaign step

A campaign step is a composite from history to a distribution over new history:

$$K_{campaign} = H \xrightarrow{\pi} D(P + Stop) \xrightarrow{trial} D(P \otimes A \otimes D) \xrightarrow{u} D(H).$$

The proposer is therefore one process in the field, not the field itself. This is the main architectural consequence. A static enumerator, Bayesian optimizer, or LLM proposer can be replaced without changing system execution, evaluation, or gate semantics.

21.5 Compositional system families

Suppose a RAG system is a composition

$$S = Q \circ B,$$

where $B: P_B \otimes Corpus \rightarrow Release$ builds an index release and

$Q: P_Q \otimes (Release \otimes Request) \rightarrow J(Y)$ serves queries. The composite parameter object is $P_Q \otimes P_B$. A query-only candidate fixes p_B and varies p_Q . An indexing candidate varies p_B and forces a new release. A joint candidate varies both.

This factorization directly induces invalidation and reuse. A morphism downstream of a changed parameter must be reinterpreted; unrelated upstream artifacts can remain fixed. In implementation, the dependency graph is an explicit finite presentation of this compositional structure.

21.6 Why not one optimizer interface

A universal interface such as

```
type Plugin interface {
    Run(context.Context, map[string]any) (map[string]any, error)
}
```

can encode every component but proves nothing. It erases objects, parameter factors, hidden-label boundaries, effects, and laws. The abstract field instead provides a small number of *families* of interfaces whose types correspond to distinct morphisms.

The core remains simple because it does not contain every RAG stage. It contains only process composition, evidence custody, decisions, and campaign state. The actual cases graft typed objects and factories onto these structures.

21.7 Relation to fibrations

Different parameter values, workloads, and evaluator contexts can be viewed as fibers over a base category of specifications. Reindexing corresponds to changing context or parameterization. Statistical-game work develops richer fibrational accounts of approximate inference and loss models. The present architecture does not implement a general fibration library, but the separation of a base specification from typed realizations and evaluator contexts is compatible with that direction.

21.8 Minimality criterion

A concept belongs in the core only when at least one of the following holds:

- it is required to state composition laws;
- it prevents an invalid comparison across product domains;
- it is needed by both exact and sampled interpreters;
- it supports two independent application grafts;
- it defines custody or identity shared by all campaigns.

Chunk semantics, SQL facts, judge rubrics, UI widgets, and product source roles fail this test and remain in adapters. Seed derivation, trace structure, cell coordinates, gate order, and plugin specifications pass it.

22. Denotational semantics of the backbone

22.1 Base interpretation

Let ρ be a runtime realization environment mapping pure component specifications to actual processes. The denotation of a typed plugin specification $s : X \rightarrow Y$ is

$$\llbracket s \rrbracket_\rho \in C(X, J(Y)).$$

Composition is homomorphic:

$$\llbracket s_2 \circ s_1 \rrbracket_\rho = \llbracket s_2 \rrbracket_\rho \circ \llbracket s_1 \rrbracket_\rho.$$

Parallel composition is similarly preserved. Runtime binding is valid only if the realized kernel has the same semantic specification identity as the factory.

22.2 Specification identity

A Spec includes:

- kind;
- stable name;
- schema/version;
- canonical semantic configuration;
- declared effects;
- capabilities;
- random namespace;
- content-derived ID.

The ID is a domain-separated hash of canonical encoding. Effects and capabilities are sorted; map keys are canonicalized. Endpoint URLs, credentials, timestamps, and worker counts are excluded unless they alter denotation. Provider model revision, prompt content, fallback policy, and remote-disclosure class are included.

A spec ID is not proof that the implementation is correct. It is a commitment to claimed semantics. Law certificates and runtime verification add evidence.

22.3 Outcome functor intuition

The instrumentation J acts like a combination of probability, exception/partiality, writer, and observation effects. One could seek a formal monad transformer or graded monad presentation. The sandbox avoids committing to one universal effect stack because failure, cancellation, trace, and resources have different composition rules and because Go does not express higher-kinded abstractions ergonomically.

Instead it defines executable composition directly and states laws. This is a pragmatic algebraic presentation: the semantics is explicit even if the implementation is not a generic monad library.

22.4 Resources

A resource vector is a finite map from resource name to nonnegative finite real. Sequential and parallel semantic accumulation both use addition in the reference implementation:

$$R_{\text{composite}} = R_1 + R_2.$$

For wall-clock latency, true parallel execution would use a critical-path operation rather than sum. The sandbox labels its latency as modeled resource consumption. A production resource algebra should declare per-dimension composition: additive cost, maximum parallel elapsed time, peak memory, union of disclosures, or a richer trace-derived measurement.

This motivates a future typed resource semiring or quantale. The current finite map is intentionally conservative.

22.5 Warnings and fallback

Warnings form an ordered list under sequential composition and a canonical multiset or trace under parallel composition. The implementation stores warnings in outcome order and records semantic fallback in trace events. A production schema should distinguish warnings that alter quality class from informational diagnostics.

22.6 Failure

Failure is represented as an attributable outcome rather than interpreter exception. This is close in spirit to partial Markov categories, which make observations and partiality explicit. The sandbox does not implement normalization or Bayesian update; it adopts only the principle that partiality must appear in the process algebra rather than as missing probability mass.

23. Operational semantics

23.1 Configurations

A sampled execution configuration can be written

$$\langle K, x, s, m \rangle,$$

where K is a kernel, x input, s root seed, and m operational metadata. A terminal configuration is an outcome or interpreter error.

For composite execution, internal configurations retain left/right subconfigurations and accumulated outcomes. The operational semantics refines the denotational composition but may choose scheduling.

23.2 Leaf rule

A bound leaf plugin validates context, derives component-local randomness from $(s, namespace, labels)$, performs its effect, and returns a terminal outcome. The implementation wrapper rejects nonterminal outcomes, invalid resources, completed outcomes without values, and nil bindings.

23.3 Sequential rule

For $K = g \circ f$:

1. execute f with root seed s ;
2. if f produces no value, return the same failure/cancellation with its trace plus a short-circuit event;
3. otherwise execute g on the value with the same root seed s ;
4. combine status, warnings, traces, and resources.

Passing the same root seed is essential. Leaves, not composite tree nodes, derive random streams. Reassociation does not change draws.

23.4 Parallel rule

For $K = f \otimes g$ on pair (x_1, x_2) , execute both branches under the same root seed. Because leaves have distinct namespaces, streams remain distinct. The interpreter may schedule branches sequentially or concurrently. It combines them with canonical Par trace and resource algebra.

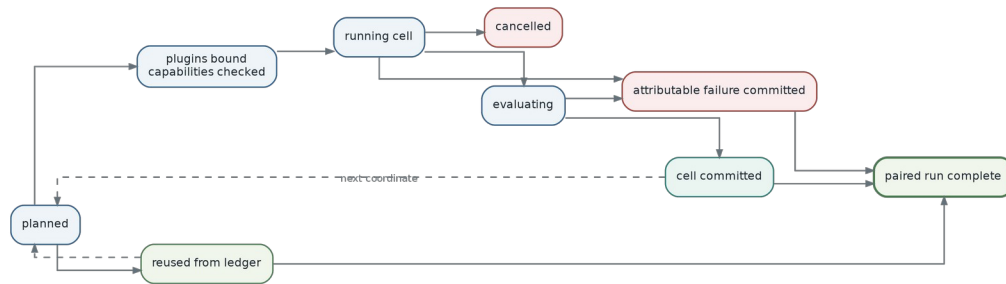
The reference implementation executes branches sequentially to stay small and deterministic. The semantic tensor does not promise parallel wall-clock execution.

23.5 Cell rule

For a planned coordinate $(case, repeat, role)$:

1. derive the coordinate seed;
2. check the ledger for an existing cell;
3. if present and valid, reuse it;
4. otherwise execute the role kernel;
5. pass the complete outcome to the evaluator;
6. serialize product output if possible;
7. append one terminal cell and synchronize the ledger.

A domain failure completes the cell. An interpreter/storage error stops the run because custody is uncertain.



Operational states for one experiment cell and paired run.

23.6 Campaign rule

A campaign round:

1. asks the proposer for challengers based on a history view;
2. builds each challenger into a kernel;
3. creates a ledger identified by the exact pair;
4. executes paired cells;
5. aggregates baseline, candidate, and paired evidence;
6. evaluates ordered gates;
7. computes Pareto membership and selection;
8. appends an immutable trial and possibly changes the incumbent pointer.

The sandbox static proposer emits a fixed list in one round. The interfaces permit adaptive proposers over multiple rounds.

23.7 Cancellation

Context cancellation produces an attributable cancelled outcome when observed inside a kernel wrapper. Cancellation before or during ledger custody can be an interpreter error depending on whether a terminal cell was durably committed. Production code should distinguish requested cancellation, provider-confirmed cancellation, and run-level abort.

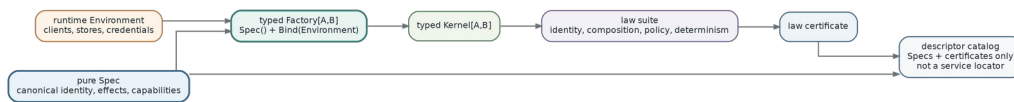
24. The plugin doctrine

24.1 Pure specification, effectful binding

The preferred system plugin interface is:

```
type Factory[A, B any] interface {
  Spec() core.Spec
  Bind(context.Context, Environment) (core.Kernel[A, B], error)
}
```

`Spec()` is pure and canonical. `Bind` resolves operational resources. The returned kernel must have the same spec ID. This prevents a factory from advertising one meaning and binding another unnoticed.



Typed plugins separate pure specifications from runtime resources and law evidence.

24.2 Why a factory rather than a singleton

A singleton plugin instance tends to capture mutable clients, caches, credentials, and environment variables before semantic validation. A factory can be inspected, identified, and policy-checked before binding. It also supports per-run isolation and test environments.

Bindings can fail because a capability is absent or an endpoint cannot realize the spec. Such failure is not a product outcome; the experiment never began under the promised semantics.

24.3 Four plugin families

The architecture recognizes four primary plugin families.

System plugins realize typed stochastic processes. Examples: chunker, representation generator, embedder, index backend, retriever, reranker, generator, agent tool.

Evaluator plugins consume cases and outcomes and emit product observations plus native artifacts. Examples: retrieval judge, grounded-answer validator, Garden session evaluator.

Decision plugins implement gate rules, Pareto dimensions, and selection policy. They are deterministic pure processes over evidence in the normal case.

Control plugins propose candidates and update search state. They can be stochastic and stateful through explicit history.

A fifth operational family—storage/scheduler adapters—may exist beneath interpreters, but these should not redefine optimization semantics.

24.4 Interface segregation

A plugin should expose the narrowest meaningful port. An index backend should not implement a generic `Run(any)`. It may implement typed full-build, delta-build, and snapshot-search interfaces, each with separate capabilities and laws. A reranker should accept authorized candidate records, not an entire product service.

Smaller ports make algebraic composition and law testing possible.

24.5 Capability declarations

Capabilities describe required or provided features, for example:

- deterministic build;
- exact scores;
- filter pushdown;
- point delete;
- remote text disclosure;
- streaming generation;
- replay from retained artifact;

- hidden-label access;
- GPU resource;
- regional provider constraint.

An effect policy can reject a graph before execution. For example, a holdout evaluator may access labels but a system plugin may not. A remote reranker can require an authorization certificate capability.

24.6 Effects

The sandbox declares coarse effects: pure, random, local I/O, remote I/O, stateful, and disclosure. These participate in spec identity. A production system can refine them into a graded effect policy carrying data class, region, idempotency, and retention.

Effects are descriptive in the core; interpreters enforce them. This mirrors typed effect systems without requiring language-level effect typing in Go.

25. Law certificates

25.1 Certificates are evidence, not proof objects

A plugin law certificate records the spec ID and results of executable laws. It can include exact finite checks, property-test seeds, conformance suites, disclosure spies, and compatibility fixtures. The sandbox catalog accepts only passing certificates.

A certificate is not a cryptographic attestation of arbitrary runtime behavior. It is durable evidence that a declared test suite passed for a particular semantic spec and implementation build.

25.2 Core law families

Relevant laws include:

- identity and composition compatibility;
- deterministic reproducibility;
- random namespace stability;
- trace preservation;
- resource finiteness;
- failure attribution;
- authorization-before-disclosure;
- filter soundness;
- index open/build equivalence;
- cache hit/fresh observational equivalence;
- evaluator hidden-label discipline;
- ledger idempotence and uniqueness;
- reducer convergence;
- candidate mutation constraints.

Not every plugin implements every law. A descriptor declares the applicable suite.

25.3 Substitutability

Two plugins are substitutable for a campaign only when:

1. their typed ports match;
2. required capabilities are present;
3. effect policy permits both;
4. semantic schemas and versions are compatible;
5. the campaign's claimed equivalence/refinement relation is supported;
6. required laws have current certificates;
7. the intervention declaration accounts for any semantic difference.

Compile-time interface satisfaction is merely the first condition.

25.4 Certificate invalidation

A certificate is invalidated by changes to plugin spec, implementation build, law suite, relevant runtime class, or dependency. A remote provider alias that changes behind the same name undermines certification. Production manifests should bind immutable provider revisions or classify the evidence as time-bound.

26. Catalogs are not service locators

26.1 Descriptor custody

The sandbox Catalog stores canonical specs and certificates. It supports introspection, reporting, policy checks, and reproducibility. It deliberately does not return executable any values by string name.

This protects typed composition. Normal Go constructors and generics wire the runtime graph. A product compiler can choose among factories while retaining concrete types.

26.2 Why service locators are dangerous here

A universal registry encourages hidden dependencies:

```
| x := registry.Resolve("reranker")
```

The caller cannot see the port type, effect, capability, lifetime, or semantic identity. Test code can install an object with compatible methods but different coupling behavior. Runtime failures occur far from composition.

A descriptor catalog is useful. An executable service locator is not the semantic backbone.

26.3 Compile-time plugin sets

For in-process components, a product can define a typed sum or configuration compiler:

```
| type RerankerChoice struct {
|     None *NoRerankFactory
|     Local *LocalCrossEncoderFactory
|     Remote *RemoteRerankFactory
| }
```

Compilation selects one branch and returns a common typed factory. This makes the supported set explicit and keeps dependency injection ordinary.

26.4 Generated registries

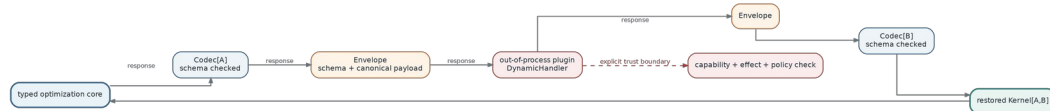
When many plugins exist, code generation can build a typed registry from descriptors. Generated code retains concrete constructors and schema checks. The generated registry is a compilation artifact, not a dynamic bag of interfaces.

27. Dynamic plugins at explicit edges

27.1 Dynamic necessity

Out-of-process workers, user-defined evaluators, polyglot services, and independently deployed providers sometimes require dynamic schemas. Refusing them entirely would make the architecture impractical. The answer is to localize schema erasure.

The sandbox defines an `Envelope{Schema, Payload}` and a `DynamicHandler`. Typed codecs encode and decode values. `AdaptDynamic` checks input/output schema names and returns a typed `Kernel[A, B]`.



Dynamic plugins cross one explicit schema and trust boundary, then return to typed composition.

27.2 Required dynamic metadata

A production dynamic request should carry:

- plugin spec ID;
- input/output schema IDs;
- operation/cell ID;
- root or derived random seed contract;
- deadline and cancellation token;
- capability grant;
- data-class and disclosure policy;
- idempotency key;
- artifact and trace return contract.

The handler cannot choose a broader capability than the grant.

27.3 Schema evolution

Schema compatibility is not semantic compatibility. Adding an optional field may alter default behavior. A dynamic plugin declares both transport schema and semantic spec version. Adapters can migrate transport while preserving or explicitly changing meaning.

Unknown fields should fail or be retained according to a declared policy. Silent field loss is particularly dangerous for authorization and expected-label data.

27.4 Trust

Loading untrusted native plugins into the host process is outside the sandbox's scope. Use process isolation, OS/container policy, authenticated artifacts, and least-privilege credentials. The semantic interface complements rather than replaces a security boundary.

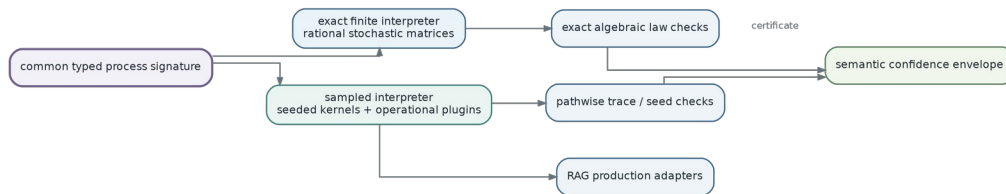
28. Exact and sampled interpreters

28.1 One signature, two realizations

The backbone uses two execution styles.

Exact finite interpretation. Processes are exact rational stochastic matrices or functions to finite distributions. Equality and laws are decidable for finite enumerated inputs.

Sampled interpretation. Processes are Go kernels executed with splittable seeds, effects, traces, resources, and failures. Laws are checked pathwise, through properties, or empirically.



The exact and sampled interpreters share process structure but provide different evidence.

28.2 Why both are needed

A sampled-only framework can accidentally treat Monte Carlo agreement as an algebraic law. An exact-only framework cannot call real providers or represent production failure. The dual approach lets exact models test the abstract spine and sampled adapters implement realistic systems.

28.3 Reference interpreters

For a production RAG query plan, one can add a deterministic reference interpreter using retained channel rankings and provider artifacts, then compare it with a concurrent optimized interpreter. This follows the same pattern as `finite` versus `core`: one is simple and law-oriented; the other is operational.

28.4 Interpreter refinement

A production interpreter refines the reference when it preserves declared outcomes and traces while improving operational dimensions. Concurrency, batching, caching, and retries should be verified as interpreter refinements where possible rather than treated as arbitrary new candidates.

28.5 Random sampling from exact distributions

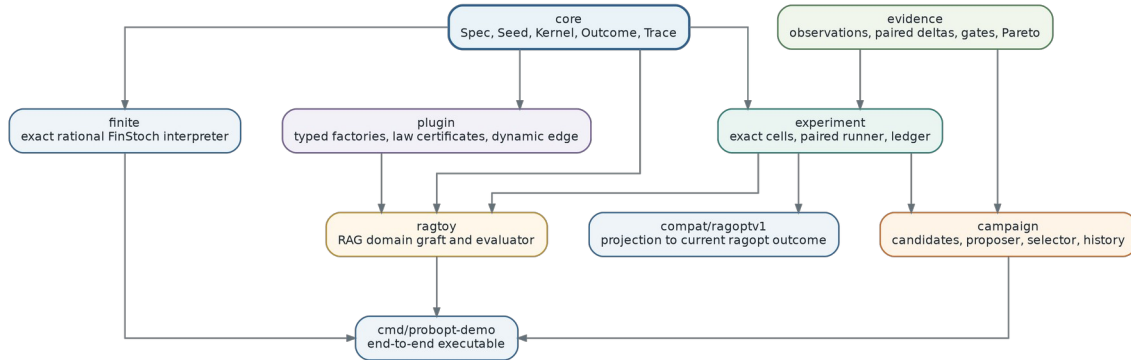
The finite distribution type requires the caller to supply a total support order when sampling. Go map iteration is nondeterministic; hiding that fact would make identical seeds produce different samples across runs. This small API detail exemplifies the thesis: operational structure not present in the denotation must be explicit in the interpreter.

Part V. Self-contained implementation

29. Repository tour

29.1 Module layout

The accompanying module is `example.com/probopt` and requires Go 1.23 or later. It has no external dependencies.



Package dependencies in the sandbox implementation.

The packages are:

Package	Responsibility
core	specifications, seeds, kernels, outcomes, traces, composition
finite	exact rational finite distributions and process laws
evidence	observations, summaries, paired deltas, gates, Pareto fronts
experiment	cases, evaluators, exact cells, paired runner, ledgers
campaign	candidates, builders, proposers, selectors, histories
plugin	typed factories, environments, laws, certificates, dynamic edges
ragtoy	complete miniature RAG domain graft
compat/ragoptv1	projection into the current ragopt outcome shape
cmd/probopt-demo	executable law and RAG campaign demonstration
examples/minimal	smallest composition example

The package graph follows the four semantic layers. `core` does not import evidence or campaigns. `experiment` imports system and evidence structures but not candidate search. `campaign` imports experiment and evidence. Product code can use only the layers it needs.

29.2 Build and verification commands

The root `Makefile` supports:

```
make test
make vet
make race
make demo
```

The reproducible demonstration is:

```
go run ./cmd/probopt-demo \
-out out \
-seed 20260809 \
-repeats 5
```

It writes a Markdown report, full JSON artifact, Pareto CSV, and one JSONL cell ledger per baseline/challenger pair.

29.3 Standard-library-only constraint

Avoiding third-party dependencies keeps the artifact self-contained and makes its semantic mechanisms visible. It is not a recommendation to reimplement mature statistics, storage, or RPC libraries in production. Production adapters should replace local pieces while preserving the interfaces and laws.

30. Canonical specifications

30.1 spec

The core specification is:

```
type Spec struct {
    Kind      string
    Name      string
    Version   string
    Config     map[string]any
    Effects    []Effect
    Capabilities []string
    RandomNamespace string
    ID        string
}
```

NewSpec trims required fields, sorts effects and capabilities, canonically encodes configuration, and computes a domain-separated SHA-256 ID. The canonical encoder rejects nonfinite numbers and unsupported nondeterministic representations.

30.2 Semantic versus execution identity

Spec identifies component meaning. A run or cell additionally identifies execution coordinates. Credentials, hostnames, and timestamps remain outside Spec; model revision, prompt content, field boosts, fallback policy, and random namespace belong inside when they alter behavior.

A production system may define separate material artifact identity and execution identity, but both should point back to semantic spec identity.

30.3 Configuration typing

The sandbox stores config as map[string]any for canonical reporting, but plugins themselves use typed parameter structs. The map is a serialization of a typed value, not the primary API. A production design can generate canonical spec projections from typed configuration to avoid missing fields.

30.4 Domain separation

Spec and trace hashes use different prefixes. Candidate IDs use another. Domain separation prevents the same bytes in different semantic roles from producing confusing identities. Versioned prefixes create explicit identity epochs when canonicalization changes.

31. Seeds, outcomes, and traces

31.1 Seed protocol

`SeedFromUint64` and `SeedFromString` create immutable roots. `Derive(labels...)` length-prefixes labels before hashing. `For(spec, labels...)` uses `RandomNamespace` when present or falls back to spec ID.

The root seed is copied, not consumed. Random generators are created only at leaves:

```
| draw := req.Seed.For(spec, "draw").Rand().Float64()
```

This design is schedule-independent but requires plugin discipline. A law can verify that a plugin uses stable semantic call labels rather than loop indices that change under refactoring.

31.2 Outcome validation

`KernelFunc.Run` validates context and terminality. A kernel returning completed status without a value is rejected. Resource vectors are validated. Interpreter errors are separated from domain outcomes.

This wrapper is the checked boundary around every plugin realization.

31.3 Trace normalization

`Seq` flattens nested sequential nodes, removes empties, and preserves order. `Par` flattens, removes empties, and sorts children by digest. `Identity` emits `EmptyTrace`, so composing an identity does not change trace.

The tests verify sequential associativity, parallel associativity, symmetry, and identity neutrality.

32. Composition combinators

32.1 Identity and lift

`Identity[A]` is a neutral kernel. `Lift` embeds a deterministic function with domain failure into the sampled process category. A lifted failure is an attributable domain failure, not a Go interpreter error.

32.2 Sequential composition

`Compose(f, g)` builds a new semantic spec from child IDs and merged effects. At runtime it executes `f`, short-circuits if no value is present, then executes `g`. Trace uses `Seq`; resources use addition; warnings concatenate.

The composite spec is useful for introspection, but `random` leaves derive from their own namespaces rather than the composite ID.

32.3 Tensor

`Tensor(f, g)` maps a typed `Pair[A, C]` to `Pair[B, D]`. Both branches receive the same root seed. Canonical child namespaces prevent random collision. The trace is `Par(left, right)`.

The reference implementation does not launch goroutines. A concurrent interpreter can be added without changing `Tensor`'s semantic meaning. Its operational artifact can record scheduling and critical-path latency separately.

32.4 Fanout

`Fanout(f, g)` first deterministically copies an input into a pair, then tensors the kernels. This makes the correlation structure visible. It does not run one stochastic kernel and duplicate its output.

32.5 MapOutcome

`MapOutcome` performs deterministic postprocessing while retaining the original trace and resources. It is useful for product adapters that normalize outputs. A postprocess failure becomes an attributable outcome.

33. Exact finite stochastic processes

33.1 Rational probabilities

`finite.Prob` wraps `big.Rat`. Constructors normalize probabilities and support exact addition and multiplication. `Dist[T]` is a finite map from comparable values to rational mass. Validation requires nonnegative mass summing exactly to one.

33.2 Kernel composition

A finite kernel is:

```
type Kernel[A, B comparable] struct {
  Name string
  Run func(A) Dist[B]
}
```

For $f : A \rightarrow D(B)$ and $g : B \rightarrow D(C)$, composition computes

$$(g \circ f)(c \mid a) = \sum_b f(b \mid a) g(c \mid b).$$

Tensor multiplies independent branch probabilities. Deterministic kernels return point distributions.

33.3 Law checker

The exact checker enumerates input values and compares distributions. Associativity can therefore be verified without sampling error for the finite example. The demonstration uses a biased binary channel and deterministic inversion.

33.4 Sampling order

Exact distributions are represented by Go maps, which have no stable iteration order. `SampleOrdered` requires an explicit support order, validates that every support value appears once, and then interprets one random draw. This prevents an implementation artifact from violating reproducibility.

34. Experiment custody

34.1 Typed arms and evaluators

The sandbox arm is a typed tuple:

```
type Arm[X, Y any] struct {
  Role      string
  CandidateID string
  Kernel    core.Kernel[X, Y]
}
```

The evaluator is independently typed:

```
type Evaluator[X, Y any] interface {
  Spec() core.Spec
  Evaluate(context.Context, EvalRequest[X, Y])
    (evidence.Observation, error)
}
```

This is a stricter internal model than current ragopt while remaining easy to adapt at JSON boundaries.

34.2 Cell artifact

A cell records API version, run/case/repeat/role, candidate and spec IDs, seed, start and finish times, status, failure, warnings, resources, semantic trace, observation, and serialized product output.

The output is optional because some values may not serialize; production evaluators should always write a native artifact reference. The sandbox keeps output inline for inspection.

34.3 Runner

RunPaired validates arms, evaluator, repeats, and case IDs. It builds the expected cell set, loads existing ledger cells, executes missing coordinates, aggregates role summaries, and constructs paired deltas only when both cells exist.

The test verifies eight cells over two cases, two repeats, and two roles; a second invocation reuses all eight.

34.4 Memory and file ledgers

MemoryLedger enforces unique cell keys. FileLedger uses append-only JSONL, validates existing lines, caches keys after loading, rejects duplicates, synchronizes every append, and supports reopening.

The file ledger is intentionally local. Distributed custody should use a transactional store with run lease and fencing. The JSONL format remains a useful interchange and audit artifact.

35. Evidence and gates

35.1 Aggregation

`evidence`. Summarize groups observations by candidate and computes descriptive metric and constraint summaries. `Pair` joins baseline and candidate observations by coordinate and computes metric deltas.

Metrics absent from one role are absent from the paired result rather than imputed. Product failures can still emit explicit metrics such as success zero or constraint false according to evaluator semantics.

35.2 Rules

The implemented rules are:

- `ConstraintAll`;
- `NonInferior`;
- `Improve`;
- `Budget`.

Each emits a `GateResult` with name, pass/fail, hard flag, and explanation. Policy evaluates in order and stops on failed hard rules.

35.3 Pareto

`ParetoFront` accepts candidate summaries and metric definitions with maximize/minimize directions. It returns nondominated candidate IDs. Missing metrics exclude a candidate from domination claims unless policy supplies another rule.

35.4 Extension points

Production rules can add confidence bounds, protected-group minima, temporal freshness, security attestations, and multiple-comparison correction. They implement the same deterministic evidence-to-verdict interface and carry pure specs.

36. Campaign package

36.1 Candidate identity

A candidate contains ID, parent ID, label, typed parameters, and hypothesis. Canonical encoding and domain-separated hashing determine ID. The sandbox allows arbitrary typed parameter changes; the current ragopt adapter can enforce exactly-one mutation.

36.2 Builder

A `Builder[P, X, Y]` maps a candidate to a typed system kernel. For RAG, this can build and open an index release, construct a query plan, or bind an existing release plus query parameters. Builder failure stops the trial because the advertised candidate cannot be realized.

36.3 Proposer

A proposer consumes history and returns candidates. `StaticProposer` returns unseen candidates from a fixed list. A production proposer can retain its own serialized state inside history or derive proposals deterministically from root seed and history digest.

36.4 Selector

The sandbox `BestMetricSelector` chooses the best eligible candidate on one metric after gate evaluation. Pareto membership is computed separately. The separation makes it clear that eligibility and preference are distinct.

36.5 History

A trial records round, incumbent, challenger, full run result, and verdict. The history stores initial and current incumbent plus ordered trials. Changing the incumbent is a new history state, not mutation of the previous candidate.

37. Plugin implementation

37.1 Environment

Environment is a minimal lookup abstraction. It is intentionally operational and untyped at the lookup boundary because resources can include arbitrary clients. Each typed factory performs its own assertion and validation. Production code can replace it with generated typed environments.

37.2 Factory validation

FactoryFunc.Bind rejects nil binders, nil kernels, and spec-ID mismatch. It does not automatically execute laws; certification normally occurs during registration/build rather than every bind.

37.3 Law API

A Law has a name and Check(context.Context) error. Certify records every result and overall pass. The catalog refuses failed certificates.

The generic interface permits domain law suites without importing product code into plugin.

37.4 Effect policy

The policy package can permit or deny effects and required capabilities. A graph compiler can merge child effects and check the composition before binding. The sandbox demonstrates the structure; production should use data-class-aware policies.

37.5 Dynamic adapter

AdaptDynamic checks handler input/output schema against codecs, encodes input, executes the handler with root seed, verifies output schema, decodes, and returns a typed success with handler trace. A production adapter should also preserve domain failure and resources rather than treating every handler error as interpreter error.

38. Compatibility with current `ragopt`

38.1 Projection

`compat/ragoptv1` projects a sandbox `Cell` into a dependency-free structure matching the current `ragopt` outcome fields: completion, contract validity, abstention, failure, metric map, provider/tool calls, tokens, duration, and native artifact.

This demonstrates migration without adding a compile-time dependency on the extracted repository.

38.2 Native artifact mapping

The sandbox stores serialized output inline. A production arm writes the complete native artifact into the current `ragopt` run directory and places a verified reference in the projected outcome. Semantic trace can be another native artifact until `ragopt` adds first-class trace fields.

38.3 Seed migration

Current `ragopt` schedules deterministic coordinates but does not expose a root seed protocol in its `Request`. The first migration can derive a seed from run configuration, case, repeat, and semantic random namespace and pass it through product adapters. Provider APIs that support explicit seed can use it; others record coupling class as uncontrolled or retained-response replay.

38.4 Typed internal, opaque external

The current suite can remain opaque JSON at the `ragopt` boundary. Each product arm decodes it into a typed case before invoking the shared kernel. This preserves `ragopt`'s domain neutrality while recovering typed semantics inside the adapter.

Part VI. Grafting retrieval-augmented systems onto the backbone

39. RAG as a parametrized open process

39.1 System boundary

A RAG system is not merely a function from query text to answer text. For optimization, a useful typed boundary includes corpus/release state, subject policy, query or conversation state, and an instrumented outcome. One direct-retrieval family can be written

$$Q: P_Q \otimes Release \otimes Subject \otimes Query \rightarrow J(Hits).$$

A retrieve-then-generate family is

$$A: P_A \otimes Release \otimes Subject \otimes Conversation \otimes Query \rightarrow J(Answer).$$

An agent family is a transition process over conversation and tool state. An indexing family is

$$B: P_B \otimes CorpusSnapshot \rightarrow J(Release).$$

The full offline system used by a campaign is the composition of a build family and a query/answer family. Production serving can instead bind an already built immutable release and vary only query parameters.

39.2 Open-system interpretation

The system is open to an environment containing corpus, providers, subject policy, workload, and runtime resources. Its plugin interfaces are ports through which it composes with that environment. Category-theoretic open-system constructions, including decorated cospans and open games, motivate the idea that a component should expose both its boundary and compositional decoration. The sandbox uses ordinary typed factories rather than implementing a general cospan category.

39.3 RAG parameter factorization

The toy implementation uses:

```
type Params struct {
  ChunkWords      int
  OverlapWords    int
  TitlePrefix     bool
  VectorDimensions int
  LexicalWeight   float64
  VectorWeight    float64
  RerankTopN      int
  RerankWeight    float64
  RerankNoise     float64
  TopK            int
}
```

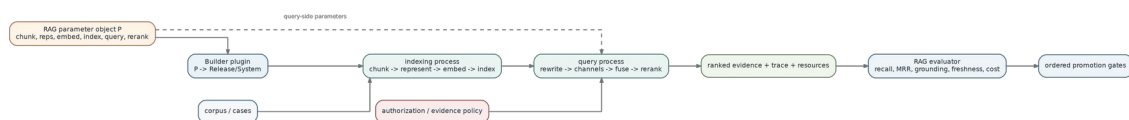
This compact struct crosses indexing and querying:

- chunk size, overlap, title prefix, and vector dimensions alter built material;
- lexical/vector weights, rerank depth/weight/noise, and top- k alter query behavior.

A production parameter object should be factored into nested typed specs with independent IDs and dependency edges. The toy struct makes the cross-phase interaction visible without implementing a full release registry.

39.4 Product-specific objects

The shared process structure does not make every RAG value generic. A Document, Query, Hit, or Answer can be shared at an appropriate domain layer. GEC scopes, TTC product fields, Garden widgets, and SQL facts retain product types. The plugin graph composes them through adapters rather than coercing them into one Evidence map.



RAG indexing and querying are one parametrized process family with product-owned policy and evaluation.

40. Indexing plugins

40.1 Chunker

A chunker plugin has a deterministic port such as:

$$Chunk: ChunkSpec \otimes DocumentRevision \rightarrow Chunk^i.$$

Its laws include deterministic output order, valid source spans, total source lineage, stable IDs, and document locality when claimed. Its spec identifies boundary algorithm, size, overlap, tokenizer/normalizer, and structural policy.

A chunker is a strong plugin candidate because multiple algorithms share a stable semantic port. The core should not know Markdown heading semantics. A `ragkit` adapter can implement the factory for fixed, Markdown-aware, and heading-aware chunkers.

40.2 Representation generator

A representation plugin maps a source chunk and representation spec to searchable derived material. Raw and breadcrumb representations can be deterministic. Summaries, questions, and entity strings are stochastic provider-backed processes unless retained artifacts make them material.

The plugin must preserve the distinction between searchable representation and authoritative evidence. A generated question can contribute to retrieval but cannot be cited as source evidence without resolving to the source chunk.

Its random namespace should be stable by semantic generation stage, prompt, model, and source chunk identity. A candidate prompt change changes spec identity and normally invalidates generated representations and downstream embeddings.

40.3 Embedder

An embedder is a stochastic or deterministic process depending on provider guarantees:

$$Embed: EmbedSpec \otimes Text \rightarrow J(Vector_d).$$

Capabilities include dimension, normalization, batch size, provider/data policy, and deterministic/replay class. A cache wrapper can turn repeated identical requests into material reuse if its key covers all semantic inputs and its hit law is certified.

The embedding plugin should not expose mutable client configuration as semantic input. Runtime endpoint and credentials bind through environment; immutable model revision and preprocessing belong in spec.

40.4 Index builder

An index backend plugin generally needs more than one method. Suggested ports are:

- full build from ordered entries;
- optional delta build from upserts and tombstones;
- open immutable snapshot;
- search under typed filters;
- optional compact;
- verify manifest and capabilities.

A monolithic `IndexPlugin.Run` would obscure which laws apply. Interface segregation permits exact backends, ANN backends, and lexical engines to implement only meaningful capabilities.

40.5 Build composition

A simplified build process is:

$$\text{Corpus} \xrightarrow{\text{Normalize}} \text{Documents} \xrightarrow{\text{Chunk}} \text{Chunks} \xrightarrow{\text{Represent}} \text{Representations} \xrightarrow{\text{Embed}} \text{Vectors} \xrightarrow{\text{Index}} \text{Release}.$$

The process is actually a DAG because lexical indexing may consume text before embedding, and representations can be generated in parallel by kind. The monoidal process structure expresses parallel branches. The build coordinator supplies batching, caching, retries, and durable events as interpreter concerns.

40.6 Invalidation from composition

If a candidate changes only lexical field weights at query time, the build branch is reused. If it changes chunking, every downstream branch depending on chunks is invalidated. If it changes vector query effort but not index construction, the index artifact is reused. The dependency graph is a finite static presentation of the process expression and parameter occurrence.

A plugin declares which parameter factors it reads. A compiler computes the transitive closure. This is stronger than naming an intervention class manually and less ambitious than automatic program dependency analysis.

41. Query plugins

41.1 Query rewriter

A query rewriter maps subject/query context to one or more retrieval queries. Deterministic synonym expansion, multi-query generation, HyDE, intent routing, and route selection have different stochastic and disclosure effects.

The plugin port should return a typed set of query variants with provenance and weights. It should not directly invoke search, because separating rewrite permits reuse and evaluation of channel contributions. A fallback to original query is an explicit outcome path.

41.2 Channel searcher

A lexical or vector channel is:

$$Search_i: Release_i \otimes AuthorizedQuery \rightarrow J(Ranking_i).$$

The input contains server-owned policy or an authorization certificate. Filter pushdown capability is declared. Output candidates include stable evidence/representation IDs, finite scores, and channel ranks.

A channel plugin must define completeness semantics. Exact lexical top- k under filters, approximate vector top- k with recall relation, and postfiltered global top- k are different contracts.

41.3 Collapse and fusion

Collapse maps multiple searchable representations to one evidence identity. Fusion combines channel rankings. These are deterministic pure kernels and good candidates for exact law testing.

Weighted reciprocal-rank fusion is identified by rank constant, channel weights, missing-channel behavior, tie order, and finite-score policy. Map iteration order must not influence the result. A reference implementation can use exact rational arithmetic or deterministic sorted loops for fixtures.

41.4 Authorization

Authorization is a product-owned deterministic process:

$$Authorize: Subject \otimes CandidateMetadata \rightarrow Decision.$$

Its output constrains which candidate text may be hydrated or sent remotely. The RAG query plan should make authorization dominate every remote text-bearing stage. An effect-policy checker rejects a graph in which a disclosure effect is reachable before authorization evidence.

The toy RAG plugin prefilters documents by role before scoring and reranking. Every observation includes an authorized constraint. This is a small executable instance of the trust-boundary law.

41.5 Reranker

A reranker accepts authorized hydrated candidates. Its spec includes model, input text composition, pool size, score/rank semantics, blend policy, timeout, fallback, and disclosure class. Random reranking noise in the toy system models provider variability and uses a stable namespace shared across paired arms where appropriate.

A reranker failure can be a failed query or a degraded fallback to fused order. The choice belongs in query policy. The trace records the path, and evaluators can gate degradation frequency.

41.6 Evidence admission

Evidence admission applies count, token/rune, diversity, authority, and source-role policies to ranked candidates. It produces an evidence session or context, not merely a truncated list. The operation should be deterministic for a fixed ranking and policy.

A product can plugin a diversification policy while retaining laws: every admitted item must come from authorized candidates; capacity is bounded; stable input prefix produces stable admitted prefix where the policy claims that property; release lineage remains uniform.

42. Answer and agent plugins

42.1 Retrieve-then-generate answer

An answer plugin consumes a context built from admitted evidence and returns a typed generated contract. Its system prompt, model, decoding, citation schema, repair policy, and abstention behavior belong in spec.

Generation is stochastic and remote. It discloses query, conversation context, and evidence according to provider policy. The trace records provider call identity and validation outcome without necessarily retaining raw sensitive text.

42.2 Grounding validator

A validator is normally deterministic:

$$\text{Validate}: \text{EvidenceSession} \otimes \text{GeneratedContract} \rightarrow \text{ValidationResult},$$

where

$$\text{ValidationResult} = \text{Valid Answer} + \text{Safe Abstention} + \text{Contract Failure}.$$

It checks citation labels, evidence membership, schema, and product rules. A model-based faithfulness judge is a separate evaluator, not a replacement for structural validation.

42.3 Agent process

An agentic RAG turn is not a single feed-forward arrow unless its internal state is made explicit. Let S be conversation/tool/evidence state. One bounded step is

$$\text{Step}: \text{AgentSpec} \otimes S \rightarrow D(S + \text{Terminal}).$$

Iterating up to a limit yields a finite state process. Tool calls are typed plugins. Search calls use the same release and evidence-session semantics as direct retrieval. The final trajectory is part of trace and native artifact.

42.4 Agent optimization

An agent policy candidate can change tool descriptions, routing, max iterations, retry, memory projection, or model. Retrieval metrics alone cannot promote it. Its required evaluator fidelity includes complete trajectories and session outcomes.

The four-layer backbone handles this without changing the core: the system process has a richer output and state; the evaluator consumes a conversation case and trajectory; gates and proposer remain the same structures.

43. RAG evaluator plugins

43.1 Retrieval evaluator

A retrieval evaluator decodes expected source spans or document IDs and computes metrics such as recall, precision, MRR, and nDCG. It also checks finite scores, authorized evidence, and target granularity.

Its spec identifies label projection, cutoff values, duplicate/collapse policy, and how missing or failed retrieval is scored. Chunking campaigns should use source-span labels or a versioned projection, not baseline chunk IDs that cease to exist.

43.2 Answer evaluator

An answer evaluator can combine:

- structural contract validity;
- citation support;
- claim completeness;
- contradiction and authority selection;
- abstention appropriateness;
- product rubric dimensions;
- latency, calls, and tokens.

Model judges are stochastic evaluator plugins with their own repeats and coupling. Judge prompt/model identity must be locked. Native artifacts retain explanations and transcripts.

43.3 Session evaluator

A session evaluator drives multi-turn interactions under one release/evidence-epoch policy. It can assert choices, source kinds, terminal settling, tool limits, widget provenance, and user outcome. Garden calibration fits this interface.

The evaluation unit is one conversation, not one prompt. Repeated turns inside a conversation are dependent and should remain grouped in statistical analysis.

43.4 Operational evaluator

A load or refresh evaluator consumes runtime traces and source-change trajectories. It measures freshness, build work amplification, tail latency, failure, cost, and release consistency. Such an evaluator may execute a simulator or shadow deployment rather than a pure local kernel.

It is still product-owned and produces shared observations under the same experiment custody model.

44. The toy RAG domain

44.1 Corpus and cases

The sandbox corpus contains a small set of tree-care documents with public and staff roles. Ten query cases specify query text, subject role, expected document IDs, and tags. The evaluator uses expected IDs while the search kernel sees only query and role.

The corpus is deliberately small enough for source inspection. It is not a benchmark of real retrieval quality.

44.2 Chunking

Documents are tokenized into word chunks with configurable size and overlap. Optional title prefix changes representation text. Chunk IDs are canonical digests of source and content fields. Index cost is modeled from chunk count and vector dimension.

44.3 Lexical and vector channels

Lexical scoring uses normalized token overlap and simple inverse-frequency-like weighting. Vector scoring uses a deterministic hashed embedding with configurable dimension and cosine similarity. This is not intended to rival production BM25 or embeddings; it creates two composable channels with controlled behavior.

44.4 Hybrid ranking

Lexical and vector scores are combined with configurable weights. Scores are validated as finite. Candidates are role-filtered before scoring. Top candidates can be stochastically reranked with bounded noise and blend weight. Results include per-channel contributions, rank, source, chunk ID, title, and snippet.

44.5 Resources and trace

The kernel emits resources:

- `index_units;`
- `modeled_latency_ms;`
- `scored_chunks;`
- `rerank_items.`

The semantic trace includes authorization prefilter, channel scoring, optional rerank, and collapse/emit events. The paired seed protocol makes rerank noise comparable between baseline and candidate.

44.6 Evaluator

The evaluator computes recall, precision, and MRR against expected document IDs. It projects index units and latency. Constraints require authorization, finite scores, and contract validity. Tags are retained.

This is enough to exercise every shared layer: system plugin, evaluator, paired experiment, gate policy, Pareto, proposer, selector, ledger, and report.

45. Demonstration campaign

45.1 Baseline

The baseline uses 26-word chunks without overlap or title prefix, 24 vector dimensions, lexical/vector weights 0.72/0.28, no reranker, and top-3 output.

45.2 Candidates

Five challengers are evaluated:

- **smaller-overlap**, using 14-word chunks and three-word overlap;
- **title-hybrid**, adding title prefixes and adjusting channel behavior;
- **lexical-heavy**, increasing lexical weight;
- **bounded-rerank**, adding a small stochastic rerank stage;
- **expensive-wide**, increasing work enough to violate the latency budget.

Each candidate is canonically identified and linked to the baseline parent.

45.3 Gate policy

The demonstration gate requires:

1. all `authorized` constraints pass;
2. all `finite_scores` constraints pass;
3. recall is noninferior with zero margin;
4. MRR is noninferior within 0.03;
5. mean modeled latency is at most 1.3 ms;
6. mean index units are at most 1,200;
7. an optional MRR improvement of at least 0.005.

The first six are hard. The final rule determines eligible versus undecided but does not reject.

45.4 Results

With root seed 20260809, ten cases, and five repeats:

Candidate	Verdict	Recall	MRR	Precision	Latency ms	Index units
bounded-rerank	eligible	0.900	0.850	0.333	1.013	688
expensive-wide	rejected	0.900	0.850	0.333	1.551	1,768
lexical-heavy	eligible	0.900	0.783	0.333	0.579	631
smaller-overlap	eligible	0.900	0.800	0.333	0.739	922
title-hybrid	undecided	0.900	0.750	0.333	0.586	688

The bounded-rerank candidate is selected by the configured highest-MRR selector. The expensive-wide candidate achieves the same MRR but fails the latency budget before later rules are considered. Title-hybrid passes hard constraints but establishes no target improvement and is therefore undecided.

45.5 Interpretation

The result illustrates why the decision layer is separate. A scalar combining MRR and latency could choose expensive-wide under one weight and lexical-heavy under another. The policy instead rules expensive-wide ineligible, retains several Pareto candidates, and applies an explicit selector among eligible options.

No claim is made that bounded reranking is generally best. The demonstration validates the architecture and reproducibility of one finite toy campaign.

46. Indexing-querying joint optimization

46.1 Shared candidate identity

A candidate that changes chunking must produce a new built system before query evaluation. A query-only candidate can reuse the baseline index. The builder plugin is responsible for interpreting parameter dependency and returning a kernel whose spec commits to both built material and query policy.

The toy builder rebuilds in memory for every candidate. A production builder would cache content-addressed artifacts and return a release reference.

46.2 Reuse as a semantic theorem

Artifact reuse is sound when unchanged upstream spec and input identities imply equal downstream inputs at the reuse boundary. For a deterministic derivation F , a cache key commits to all inputs and F 's spec. A cache hit is valid if the stored output verifies and is observationally equivalent to fresh F .

This is more precise than “same filename” or “same candidate label.” The dependency graph and canonical specs provide the premises for reuse.

46.3 Multi-fidelity evaluation

A joint campaign can stage evaluation:

1. build-law and lineage checks;
2. exact/approximate index oracle tests;
3. retrieval cases;
4. answer cases;
5. session cases;
6. refresh/load simulation;
7. shadow/canary.

A candidate is promoted to the next fidelity only after required gates. The campaign history records which evidence exists. A proposer can use low-fidelity observations while holdout gates remain hidden.

46.4 Shared randomness across build and query

Generated representations can be coupled across candidates when source chunks and generation semantics align. Query reranker noise can be coupled separately. The root seed derives stage- and item-specific streams, so adding an unrelated deterministic stage does not shift draws.

When chunking changes item identity, generated outputs usually cannot be coupled one-to-one. The campaign artifact should state the coupling breakdown rather than pretending every stochastic source is paired.

47. Production optimization as open-loop and closed-loop control

47.1 Open-loop campaigns

Offline campaigns operate on frozen corpus releases and workloads. They are easier to reproduce and diagnose. Their proposer sees experiment evidence and emits candidate releases. Promotion remains external.

47.2 Closed-loop production

A production canary introduces feedback from live traffic. Campaign state includes cohort routing, observed SLOs, user signals, and rollback events. The control process can stop or revert based on hard gates.

This is a stochastic control system over release and traffic state. It should not be represented as a background callback inside the query service. The release manager executes activation; the campaign consumes immutable operational evidence.

47.3 Separation from deployment authority

The selector can recommend a candidate. An activation authority verifies gate report, expected current release, actor, cohort, and rollback plan. This maintains the earlier ragopt principle that optimization reports do not deploy directly.

47.4 Feedback and concept drift

Live outcomes change as corpus, users, providers, and competing releases change. A campaign comparing over time must include release and environment identity. Adaptive controllers should detect when historical evidence is no longer exchangeable with current traffic.

The Markov-process model can represent changing state, but sound inference requires domain assumptions beyond category laws. The architecture makes those assumptions visible rather than solving them automatically.

Part VII. Mapping to the present repositories

48. `ragopt`: preserve custody, add semantics around it

48.1 Keep the generic core domain-neutral

The current `ragopt` package has a good architectural center: immutable candidate bundles, exact one-mutation validation, opaque product cases, exact paired cells, run custody, resume, comparison, ordered gates, and reports. It should not import RAG types or become the owner of chunkers, releases, query traces, or frontend sessions.

The probabilistic backbone can be introduced in one of two ways:

1. a separate common module such as `probopt` or `experimentkit`, with a thin `ragopt` adapter;
2. narrowly scoped packages inside `ragopt` that remain domain-neutral and do not break current JSON APIs.

The first is cleaner while the theory stabilizes. The sandbox uses a separate module and a compatibility projection.

48.2 Evolve `Arm` through adapters

Current product arms can remain:

```
type Arm interface {
    Name() string
    Run(context.Context, Request) (Outcome, error)
}
```

A typed adapter decodes `Request.Case.Input`, constructs a root seed from run coordinates, invokes `core.Kernel[X, Y]`, runs the typed evaluator or returns product output, writes native artifacts, and projects to `ragopt.Outcome`.

This gives immediate seed, trace, and composition semantics without changing the public runner. Later, `ragopt` can add optional fields for spec IDs, seed/coupling identity, trace artifact, and structured failure.

48.3 Candidate mutation and parameter objects

The exactly-one-mutation rule should remain available as a campaign policy. A typed parameter object can serialize each factor as an asset. A candidate manifest declares which factor changed. The dependency compiler computes invalidation closure and verifies locked factors.

Some campaigns legitimately change a coherent bundle of low-level fields through one high-level parameterization. The mutation unit should then be the versioned reparametrization asset, not arbitrary exception to the rule.

48.4 Outcome schema

The current compact outcome is a projection. It should not absorb every trace event. Suggested optional additions are:

```
type SemanticRef struct {
    SystemSpecID string
    EvaluatorSpecID string
    CouplingID string
    Seed string
    TraceArtifact ArtifactRef
}
```

Metrics can remain maps while a policy/suite artifact supplies metric schemas. This avoids a disruptive generic-type migration in `ragopt` itself.

48.5 Run config

Run semantic identity should include:

- system/evaluator specs for both roles;
- coupling and seed protocol;
- visibility policy;
- metric/gate schemas;
- workload partition identity;
- candidate dependency closure;
- interpreter version.

Execution details such as worker count remain run metadata unless they can alter outcome semantics.

48.6 Proposer and campaign packages

The current supplied ragopt snapshot focuses on one paired run. Adaptive campaigns can be a separate package consuming immutable completed runs and reports. It should not mutate active runs. A campaign ledger links candidate proposal, pair run, comparison, gate, selector, and next incumbent.

This separation keeps the proven runstore simple and allows static users to ignore adaptive control.

49. `ragkit`: make components realizations of typed process specs

49.1 Existing common mechanisms

`ragkit` already has deterministic document/chunk/representation identities, lexical and vector backends, immutable index bundles, retrieval and fusion kernels, answering, context construction, contract validation, flow execution, caches, retries, budgets, and evaluation metrics. These are natural realizations of the process interfaces.

The migration should not wrap every function in a runtime plugin object. Pure kernels can remain ordinary functions and implement adapters only at architectural boundaries.

49.2 Suggested interfaces

The following are suitable plugin ports after their semantics are stable across at least two applications:

- `corpus.Snapshotter` and `Normalizer`;
- `chunking.Factory` producing deterministic chunk kernels;
- `representations.Factory` by representation kind;
- `embedding.Factory` with provider and cache policy;
- `index.Builder` and `index.Opener` capabilities;
- `query.Rewriter`, `Channel`, `Fusion`, `Reranker`, and `Admission`;
- `answer.Generator` and `Validator`;
- `eval.Evaluator` adapters.

Each factory returns or compiles a typed process spec. Existing package-level constructors can implement the factory without invasive internal changes.

49.3 `flow` as an interpreter mechanism

`ragkit/flow` manages bounded concurrency, retry, caching, budgets, and failures within stages. It should remain an operational interpreter mechanism. Its effects and policies become part of the bound runtime or execution spec. It is not the mathematical process category itself.

A reference sequential interpreter can use the same component specs and provide differential fixtures. Production flow then demonstrates refinement.

49.4 Index bundle and release

An index bundle is one output artifact of a build process. A behavior-complete release additionally binds query policy, prompts, reranker, structured stores, and presentation. Optimization candidates should identify the release or system spec, not only the bundle path.

The backbone's builder can return a release reference whose query kernel binds immutable resources.

49.5 Evaluation

Existing retrieval metrics can populate `evidence.Observation`. The evaluator spec identifies target granularity and cutoffs. Answer and session evaluators remain product-owned. `ragkit` can supply shared evaluator helpers and law suites without defining one universal score.

50. RAG-TTC: proving the composite architecture

50.1 Why RAG-TTC is the main integration target

RAG-TTC contains the broadest span of relevant behavior: complete index builds, multiple generated representations, embedding caches, exact and ANN vector search, workspace Git snapshots, connected retrieval, product catalogs, model-invoked search tools, evidence ledgers, answer/tool evaluation, and persistent chat. It can exercise nearly every process and evaluator family.

It also contains a copied common RAG substrate that should be removed. One semantic kernel cannot govern two implementations that diverge by repository copy.

50.2 First step: cut to `ragkit`

Before adding the new optimization backbone, migrate copied common packages to published `ragkit` imports with differential fixtures. This establishes one owner for chunking, indexing, retrieval, and answer laws. Product-specific packages remain in RAG-TTC.

The exact/sampled law approach can help: compare deterministic fixtures and retained traces before and after the cut.

50.3 Build family

The current index command can become a `Builder[Params, CorpusSnapshot, Release]`. Its parameter object includes chunker, representation kinds/prompts, embedding, lexical/vector backend, and bundle settings. Existing caches are interpreter resources with semantic keys.

A production candidate changes one factor, computes impact closure, and returns a verified release artifact. The experiment runner evaluates the resulting query kernel.

50.4 Query family

Workspace search, ask, and `ttcrag.SearchTool` can share one compiled retrieval process but use separate interpreters:

- direct result interpreter;
- answer interpreter;
- agent-tool interpreter with evidence state.

Routes, product filters, connected retrieval, and product catalog remain typed TTC plugins. Their effects and capabilities become visible in the system spec.

50.5 ANN campaign

The existing HNSW bakeoff becomes an evaluator/plugin campaign over index backend parameters. The exact backend supplies an oracle evaluator. Update, filter, compaction, and downstream answer evaluators can be added as higher fidelities. The decision policy keeps recall/authorization hard and trades latency, memory, and build cost on a Pareto frontier.

50.6 Tool-loop campaign

The existing `ragopt` adapter for TTC tool evaluation already follows the right dependency direction. It can wrap a typed agent kernel and evaluator, emit a semantic trace artifact, and use common random namespaces for aligned provider calls when supported. Product transcript and judge artifacts remain authoritative.

51. GEC: authorization and administrative evaluation

51.1 Product-owned policy

GEC owns access scopes, source roles, synonym groups, reranker document composition, tool schemas, judges, and administrative chat behavior. These remain product adapters around shared process ports.

The backbone makes their order and effects explicit. In particular, authorization is a deterministic policy process whose certificate must precede any remote reranker disclosure.

51.2 Current query composition

A GEC query family can be represented as:

```
raw query
-> lexical rewrite
-> authorized lexical/vector channels
-> collapse and weighted fusion
-> authorized hydration
-> optional remote rerank
-> rerank/fused blend
-> evidence admission
-> admin tool projection
```

The current implementation's post-retrieval filtering should be replaced by policy-constrained candidate generation or local pre-rerank filtering. A law certificate includes a spy test proving unauthorized marker text never reaches the remote provider.

51.3 Synonym and reranker specs

Synonym content, expansion semantics, reranker model, text composer, pool size, blend, timeout, and fallback become pure specs within the behavior-complete release. Runtime endpoint and credentials bind separately.

This removes the ambiguity in which two servers share one bundle ID but answer differently due to environment configuration.

51.4 Fusion campaign

The current GEC RRF sweep can be retained as a low-fidelity evaluator over cached channel rankings. In the new field it is a query-only parameter subspace. Higher-fidelity arms include reranking, answer judgment, authorization strata, and latency/failure tests.

The shared campaign retains exact one-mutation candidates and paired cells; GEC supplies labels and native artifacts.

51.5 Administrative evidence

GEC's run-scoped evidence ledger maps naturally to an evidence-session plugin bound to one release. Stable labels, bounded capacity, and citation validity become laws. The optimization evaluator can test evidence behavior without moving admin-specific display schemas into the core.

52. Garden: interaction and presentation as evaluable processes

52.1 Structured and unstructured evidence

Garden combines source chunks, structured product facts, intent routing, connected retrieval, and evidence-bound widgets. The shared system output is therefore a typed sum of evidence and presentation events, not a list of text passages.

Structured fact adapters retain database/query/item lineage. Source chunks retain document revision and span. The product projection decides how they combine.

52.2 Conversation process

A Garden conversation is a stateful stochastic process. Each turn can acquire a release lease or continue within an explicit conversation evidence epoch. Search and structured tools update evidence state. Agent output emits answer, choices, and widget events.

Optimization candidates can change intent policy, search route, tool description, answer prompt, widget policy, or release factors. Required evaluator fidelity follows the dependency graph.

52.3 Calibration evaluator

The existing calibration runner is a session evaluator plugin. It creates isolated sessions, submits turns or choices with deterministic idempotency coordinates, waits for stable terminal state, and records normalized snapshots. It can emit shared observations for task completion, source kinds, widget provenance, latency, and tool count while retaining full native turns.

52.4 Presentation gates

Hard Garden constraints include:

- no widget field without admitted provenance;
- no cross-release evidence mixing;
- no invalid choice continuation;
- no hidden developer data in customer projection;
- no conflicting fact silently selected.

These cannot be inferred from retrieval metrics. The backbone handles them as product constraints at the session/presentation level.

53. Plugin placement rules

53.1 Good plugin boundaries

A boundary is a good plugin interface when:

- multiple implementations exist or are expected;
- input/output semantics are stable;
- effects and capabilities can be declared;
- laws can be tested independently;
- the component can be composed without global service knowledge;
- substitution is a meaningful intervention.

Chunkers, embedders, index backends, query rewriters, channels, rerankers, evaluators, gates, and proposers often satisfy these conditions.

53.2 Poor plugin boundaries

A boundary is usually poor when:

- it is only a helper function;
- implementations require arbitrary access to the full application;
- its semantics are not stable;
- it exists solely to mock code;
- the only implementation is trivial and unlikely to vary;
- it would force pervasive `any` or reflection;
- lifecycle and ownership cannot be stated.

Do not make every rank comparator, struct mapper, or error formatter a plugin.

53.3 Product facades

A product facade can compose many shared plugins and product functions into one typed arm. This is often the right granularity for `ragopt`: the generic experiment runner sees one product arm, while the arm is internally compiled from the compositional backbone.

53.4 Interface versioning

A port has both transport/API version and semantic contract version. A backwards-compatible Go method addition can still change semantics. Law suites and spec IDs must track contract changes.

53.5 Lifecycle

Factories declare binding scope:

- process-global immutable;
- release-scoped;
- run-scoped;
- case-scoped;
- turn/session-scoped.

A singleton service locator cannot express these cleanly. Typed compilers and environments can.

54. The dependency graph as a free presentation

54.1 From category expression to graph

A composite process expression can be presented as a directed acyclic graph whose nodes are plugin specs and edges are typed artifacts or values. The graph is not the semantics itself; it is a finite syntax interpreted into the process category.

A build/query graph might contain nodes for normalize, chunk, represent, embed, lexical index, vector index, rewrite, channel search, fusion, rerank, and evaluate. Node specs commit to parameters. Edge schemas commit to artifact meaning.

54.2 Free construction intuition

The graph can be viewed as a term in the free symmetric monoidal category generated by a signature of operations, quotiented by declared laws. An interpreter maps generators to actual kernels and composition/tensor to runtime combinators.

This provides a clean plugin story: plugins realize generators; the core owns composition. A plugin does not own the graph scheduler or the meaning of other nodes.

54.3 Dependency closure

Each parameter field belongs to one or more node specs. A candidate diff changes node IDs. Downstream node identities change because their input artifact/spec IDs change. The transitive closure determines rebuild and reevaluation.

This can be computed structurally without understanding implementation code. It is an important practical benefit of canonical pure specs.

54.4 Common subexpressions

If baseline and candidate graphs contain nodes with equal semantic spec and equal input artifact identities, the corresponding outputs can be reused under a certified cache law. This is categorical common-subexpression elimination over process terms, constrained by effects.

Random/stateful nodes require retained material or compatible coupling. Remote calls cannot be elided merely because names match.

54.5 Graph compiler

A product graph compiler should:

1. validate typed ports;
2. canonicalize specs;
3. compute IDs and dependency closure;
4. check effects/capabilities/visibility;
5. select factories and bind environments;
6. construct exact/reference and sampled/production interpreters;
7. emit graph, law, and release manifests.

It should not become a generic visual workflow editor. The domain signature remains code and versioned schemas.

55. Migration plan

55.1 Phase 0: freeze evidence

Capture current ragopt paired runs, candidate validation, gate reports, and representative product-native artifacts. Add deterministic fixtures for RAG-TTC, GEC, and Garden query paths. Record current failure behavior and identity epochs.

55.2 Phase 1: introduce core specs and traces

Add canonical component specs and semantic trace artifacts behind existing product arms. Do not change runner APIs. Add root seed derivation and stable component namespaces for controllable stochastic stages.

Acceptance requires pathwise-stable refactors and no dropped failures.

55.3 Phase 2: typed product adapters

Inside each arm, decode opaque JSON into typed cases and invoke typed kernels/evaluators. Project back to current outcomes. Keep native artifacts unchanged or enriched.

This phase produces immediate compile-time composition without changing ragopt domain neutrality.

55.4 Phase 3: plugin factories and laws

Wrap genuinely variable components in typed factories. Introduce pure spec/runtime binding separation, effect/capability policy, and law certificates. Keep constructors for fixed helpers.

Start with chunker, embedding provider, index backend, reranker, and evaluator interfaces where current applications already vary implementations.

55.5 Phase 4: dependency-aware RAG candidates

Represent RAG build/query parameters as typed factors. Compile candidate diffs into dependency closures. Reuse equal artifacts under certified keys. Extend run reports with system/evaluator/coupling IDs.

55.6 Phase 5: decision and Pareto semantics

Migrate ad hoc winner selection to ordered gates and explicit selectors. Add metric schemas, directions, protected strata, and nondominated fronts. Preserve current gate outputs through adapters during transition.

55.7 Phase 6: adaptive campaign custody

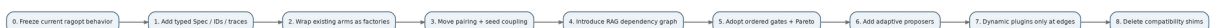
Add immutable campaign histories and proposer interfaces above completed paired runs. Initially use static proposers. Introduce adaptive search only after visibility and holdout policies are represented.

55.8 Phase 7: dynamic edges

Add out-of-process plugins only where operationally necessary. Require schema codecs, capability grants, effect policy, idempotency, trace return, and law certification. Do not retrofit a universal runtime registry.

55.9 Phase 8: cleanup

Remove compatibility shims, old seed paths, duplicate metric selection, and copied RAG substrate. Major-version public interfaces only after at least two product adopters exercise the laws.



Dependency-ordered migration from current experiment custody to the full compositional backbone.

Part VIII. Correctness, assurance, and research program

56. Law hierarchy

56.1 Algebraic laws

Algebraic laws concern the structure of composition independent of a particular workload:

- identity neutrality;
- sequential associativity;
- tensor associativity and symmetry;
- trace normalization;
- discard naturality;
- deterministic-copy naturality;
- resource-composition laws;
- parameter substitution and reparametrization coherence.

The finite exact interpreter is the preferred test bed. Sampled pathwise tests supplement it for seed and trace behavior.

56.2 Component laws

Component laws describe domain semantics:

- chunk spans reconstruct exact source slices;
- representation resolves to source evidence;
- embedding dimension and normalization match spec;
- index filters are sound;
- deletion/tombstone removes logical evidence;
- fusion is deterministic;
- reranker sees only authorized text;
- context contains only admitted evidence;
- citations resolve to context;
- frontend widgets resolve to admitted evidence.

These are plugin certification suites.

56.3 Experiment laws

Experiment laws include:

- one cell per exact coordinate;
- baseline and candidate share case/repeat/coupling identity;
- no silent cell omission;
- resume is equivalent to uninterrupted execution;
- product failure remains a completed cell;
- hidden expected data never reaches the system kernel;
- aggregation is invariant under cell order;
- decision results are reproducible from cells and policy.

56.4 Campaign laws

Campaign laws include:

- history is append-only;
- proposer receives only permitted view;
- selected incumbent is eligible under the recorded policy;
- candidate parent exists in history or declared external base;
- trial identity is stable;
- changing selector can replay history without changing cells;
- stop reason is terminal and explicit.

57. Property-based and differential testing

57.1 Process generators

Generate small exact kernels over finite sets and check category/Markov laws. Generate deterministic maps separately for copy naturality. Include zero-probability support, degenerate distributions, and equal-score outcomes.

The sandbox uses hand-written examples to avoid dependencies, but production can use Go fuzzing or a property library.

57.2 Seed properties

For random root seeds, namespaces, labels, and composition trees:

- equal inputs produce equal derived seeds;
- distinct length-delimited labels do not collide in generated samples;
- reassociation preserves leaf draws;
- tensor branch order preserves the canonical result under symmetry mapping;
- adding an unrelated component with a distinct namespace does not shift existing draws;
- baseline/candidate shared namespace produces expected coupling.

No finite test proves cryptographic collision resistance. The property is operational consistency.

57.3 Trace properties

Generate trace trees and check:

$$Seq(Seq(a, b), c) = Seq(a, Seq(b, c)),$$

$$Par(Par(a, b), c) = Par(a, Par(b, c)),$$

$$Par(a, b) = Par(b, a),$$

with empty identities. Verify canonical JSON and digest are stable under map insertion order.

57.4 RAG differential fixtures

For each applied system, run current and target paths on retained local fixtures. Compare at several projections:

- exact artifact identity where expected;
- ranked evidence and tie order;
- trace events and disclosure;
- context/evidence admission;
- answer contract;
- product tool or widget projection;
- metric and gate projection.

Classify intentional differences, especially authorization-before-reranking and identity epochs.

57.5 Cache laws

For generated semantic inputs, verify that every input field claimed material changes the key. A hit and fresh computation should be equivalent under the declared relation. Corrupt or mismatched cache values fail closed.

58. State-machine testing

58.1 Paired-run state

A run can be modeled with states planned, active, cancelling, complete, failed, and cancelled, plus a set of committed coordinates. Commands include start, append cell, resume, cancel, and complete.

Invariants include:

- committed coordinates are unique;
- terminal run is immutable;
- complete means exactly expected coordinates are present;
- resume never changes run identity;
- a failed storage append cannot be reported as a completed cell.

58.2 Distributed ledger extension

A production store adds lease and fencing token. Model two workers acquiring, losing, and renewing leases. A stale worker cannot commit after a newer fence. Content-addressed provider output may be retained, but run state advances only under the current fence.

58.3 Campaign state

Model concurrent reviewer, proposer, and selector actions. A proposer can operate on a history revision and its candidate remains linked to that revision. If the incumbent changes before evaluation, the campaign either evaluates against the declared parent or rejects/rebases explicitly. There is no implicit “latest.”

58.4 Release and production state

When campaigns connect to deployment, include release registered, verified, staged, active, draining, retired, and revoked states. Activation is compare-and-swap. A campaign report can request activation but cannot bypass state invariants.

58.5 Model checking

TLA+ or an explicit-state checker is appropriate for run custody, leases, activation, and campaign parentage. Category laws do not detect race conditions; state-machine models do. Conversely, state-machine tests do not establish process associativity. Assurance is layered.

59. Security and plugin trust

59.1 Interface semantics are not isolation

A typed factory can still execute malicious code. Law certificates can be forged. The architecture supports review and policy but does not create a secure sandbox. Untrusted plugins require process/container isolation, signed artifacts, least privilege, network policy, and resource limits.

59.2 Capability grants

A bound plugin should receive a capability object rather than ambient access to the whole environment. A reranker gets an authorized provider client and disclosure certificate validator, not the corpus store. An evaluator gets hidden labels only in its process. A proposer receives a restricted history projection.

59.3 Data-flow effects

A refined effect type can carry:

```
RemoteDisclosure{
  data_class,
  provider,
  region,
  retention,
  purpose
}
```

Composition joins effects. A policy decides whether the graph is admissible for a subject/workload. Trace proves which effects occurred.

59.4 Dynamic plugin authentication

Out-of-process handlers should authenticate spec and implementation build, negotiate schemas, and sign result artifacts where custody requires it. The host checks that returned trace/effects are compatible with the grant. Sensitive values should be referenced by authorized artifact handles rather than embedded in generic envelopes.

59.5 Evaluator integrity

An adaptive proposer has an incentive to exploit evaluator artifacts. Hidden labels and judge prompts should be access-controlled. Product-native artifacts can be available to reviewers while the proposer receives summaries. Automated proposers should not choose evaluator versions or remove failed cases.

60. Statistical validity under adaptivity

60.1 Descriptive core, pluggable inference

The sandbox core computes deterministic descriptive summaries. It does not bake one inference method into the semantic kernel. This is deliberate because appropriate statistics depend on case sampling, clustering, stochastic repeats, adaptive selection, and metric distribution.

A statistical-analysis plugin consumes immutable cells and emits an evidence artifact with method, assumptions, confidence bounds, and multiplicity handling.

60.2 Repeats

Repeats estimate stochastic variation only when they represent independent or intentionally coupled samples from the claimed process. Reusing a deterministic cache does not create new samples. Repeated model calls can be temporally correlated. The evaluator artifact should classify each repeat source.

60.3 Case sampling

Cases may be a fixed conformance suite rather than a random sample. In that case, exact suite outcomes can be reported without population inference. Generalization claims require a sampling model. The architecture keeps case IDs and groups so the report can state the estimand honestly.

60.4 Multiple candidates

Selecting the best of many noisy candidates creates winner's curse. Controls include validation/holdout splits, nested selection, empirical Bayes or hierarchical models, sequential corrections, and a final fresh evaluation of the selected candidate.

The campaign history exposes the number and lineage of tried candidates; a report should not present the final pair as though it were the only comparison.

60.5 Paired bootstrap and cluster structure

For RAG cases grouped by source, user, or conversation, resample at the appropriate cluster level. Within each cluster retain paired baseline/candidate outcomes. Repeats can be nested. A statistical plugin specifies the resampling unit.

60.6 Online canaries

Canary monitoring often involves repeated looks and nonstationary traffic. Sequential confidence sequences, alpha spending, or Bayesian decision rules can be plugins. Hard security and integrity invariants remain exact and do not need probabilistic thresholds.

61. Limits of the reference implementation

61.1 Probability model

`finite` handles only finite comparable values and exact rational distributions. `core` returns one pseudorandom sample and does not expose density, conditioning, or automatic integration. Continuous provider behavior is represented operationally, not measure-theoretically.

61.2 Effects

Effects are declared strings and enforced by simple policy. They do not form a compiler-checked graded monad or capability-safe language. Runtime adapters can still misdeclare behavior.

61.3 Resource algebra

Resource dimensions are untyped strings with additive accumulation. Real latency, memory, peak concurrency, privacy loss, and monetary cost require different composition operations. The model should evolve to typed resource algebras.

61.4 Statistics

The demonstration uses means and standard deviations, no confidence intervals, and a fixed toy corpus. Results validate mechanics only.

61.5 Durability

JSONL `append` and `fsync` provide local durability but no cross-process linearizability, transactional artifact commit, fencing, or corruption recovery beyond parse validation.

61.6 Plugin loading

The catalog does not dynamically load Go plugins. The dynamic adapter is an interface and schema pattern, not an RPC implementation. This is intentional.

61.7 RAG realism

The toy lexical/vector functions are simplistic. The model omits generation, corpus evolution, release activation, frontend events, and true ANN structures. The thesis explains how those graft onto the same backbone; the code focuses on the minimum complete optimization field.

62. Research directions

62.1 Representable and partial Markov categories

Representable Markov categories provide abstract distribution objects and comparison of statistical experiments. They may supply a stronger account of candidate evidence and Blackwell informativeness. Partial Markov categories may model failed observations, conditioning, and decision problems more naturally than the current outcome sum.

A future formalization could express failure/cancellation and hidden-label observation through a partial stochastic process calculus while retaining the executable Go projection.

62.2 Bayesian lenses and statistical games

Bayesian lenses provide compositional forward and inverse statistical structure. Statistical games attach losses to models. This may support local evaluator composition, approximate inference plugins, and optimizer updates as bidirectional structure.

The architectural challenge is to preserve hard constraints and product-native diagnostics rather than reducing every local game to additive loss.

62.3 Cybernetic systems

Categorical cybernetics studies processes interacting with environments and controllers. The four-layer model can be viewed as a concrete software-oriented fragment: forward system, observation/evaluation, controller, and state. Further work could model online RAG maintenance and adaptive routing as open cybernetic systems.

62.4 Open games and multi-agent optimization

Agentic RAG may involve multiple tool policies, reviewers, users, and deployment authorities with different incentives. Open games offer a compositional language for strategic interaction. This is relevant when optimization cannot be represented as one cooperative objective.

The immediate architecture should not require equilibrium computation. It should retain boundaries and cointeraction/evidence flows so richer models remain possible.

62.5 Fibrations of evaluation contexts

Different workloads and evaluator fidelities form contexts over system specifications. Fibrational structure may formalize reindexing a candidate into retrieval, answer, session, and production evaluators and classify which approximations compose.

The current typed dependency graph is a finite operational approximation. Formal work could prove when local evaluation sections are coherent.

62.6 Quantaes and resource semantics

Latency bounds, cost accumulation, disclosure sets, and reliability can be modeled in ordered monoids or quantaes. Enriched categories may give a uniform account of approximate/refinement distance and compositional budgets.

A practical next step is typed resource dimensions with per-dimension serial/parallel operations and monotone gate semantics.

62.7 Causal intervention

The parameterized-process model states interventions but does not by itself identify causal effects under changing environments. Structural causal models could represent corpus, provider, and traffic confounding. Candidate pairing and randomization protocols can then be derived from a causal graph.

This is particularly important for online RAG optimization, where releases are not evaluated under identical user populations or source worlds.

62.8 Proof assistants

The finite laws, trace algebra, gate monotonicity, and campaign state invariants are small enough for mechanization in Lean, Coq, or Agda. A verified core could generate test vectors or schemas for the Go implementation. Full provider behavior remains outside proof.

62.9 Probabilistic programming interpreters

A richer sampled interpreter could target a probabilistic programming language, enabling inference over parameter/posterior objects and exact conditioning for models that support it. The typed process signature and plugin specs would remain the external architecture.

63. Conclusion

The optimization field should not be organized around one `Optimize` method. Its stable backbone consists of four different structures:

- a compositional process theory for the system being changed;
- an evidence theory for how behavior is observed under exact workloads and couplings;
- a decision theory for constraints and preference;
- a control theory for proposing the next intervention.

Markov categories supply the right process intuition because they distinguish deterministic copying from stochastic composition and support sequential and parallel wiring. Parametrized maps supply the right candidate intuition because a system family is a process with an explicit parameter object. Statistical games and lenses motivate a separate evaluator/backward structure. Stochastic state transitions supply the right campaign intuition.

The software consequence is a small kernel rather than a universal framework. Typed kernels compose. Pure specs identify meaning. Seeds derive at stochastic leaves. Traces retain intensional behavior. Evaluators remain product-owned. Paired cells have exact coordinates. Gates are ordered. Pareto fronts preserve incomparability. Proposers operate over immutable histories. Plugins expose typed factories and law evidence; dynamic schemas are confined to edges.

The self-contained implementation demonstrates that these ideas can be made ordinary Go. It certifies exact finite laws, preserves sampled associativity pathwise, runs a cross-phase RAG campaign, resumes durable cells, and selects under constraints. Its limitations are explicit, which is part of the semantic discipline.

For the present repositories, the design does not require moving RAG into `ragopt`. `ragopt` remains experiment custody. `ragkit` becomes the owner of shared RAG process semantics and plugin realizations. Product applications provide authorization, evaluators, agents, and presentation. A thin RAG optimization adapter composes them.

The central standard for future work is therefore not “can this component be registered?” It is:

Can the component be given typed ports, a pure semantic specification, explicit stochastic/effect behavior, a composition-preserving realization, and executable substitution laws?

When the answer is yes, a plugin interface is useful. When it is no, registration merely hides architecture. The proposed backbone keeps the core simple precisely by demanding strong semantics at the graft points.

Appendix A. Formal definitions and laws

A.1 Symmetric monoidal process category

Let $(C, \otimes, I, \alpha, \lambda, \rho, \sigma)$ be a symmetric monoidal category. For readability, coherence isomorphisms are usually suppressed. Objects represent typed value spaces or interfaces. Morphisms represent processes.

A **Markov category** additionally equips every object X with morphisms

$$\text{copy}_X : X \rightarrow X \otimes X, \text{discard}_X : X \rightarrow I,$$

forming a commutative comonoid compatible with tensor. A morphism $f : X \rightarrow Y$ is **deterministic** when

$$\text{copy}_Y \circ f = (f \otimes f) \circ \text{copy}_X.$$

In a causal Markov category,

$$\text{discard}_Y \circ f = \text{discard}_X$$

for every morphism. The exact finite implementation represents causal finite kernels.

A.2 Parametrized process bicategory

For a monoidal category C , a parametrized morphism from X to Y consists of an object P and a morphism

$$f : P \otimes X \rightarrow Y. \text{ Composition of } (P, f) : X \rightarrow Y \text{ and } (Q, g) : Y \rightarrow Z \text{ is}$$

$$(Q \otimes P, Q \otimes P \otimes X \xrightarrow{id_Q \otimes f} Q \otimes Y \xrightarrow{g} Z).$$

A reparametrization $r : P' \rightarrow P$ maps (P, f) to $(P', f \circ (r \otimes id))$. Keeping reparametrizations as 2-cells yields a bicategorical account. The Go sandbox represents the object-level family and immutable candidate values but does not implement 2-cells explicitly.

A.3 Instrumented stochastic process

Let:

- Y be successful output;
- F attributable failures;
- C cancellations;
- T structured semantic traces;
- R resource summaries;
- W warnings or degradation markers.

Define the outcome carrier

$$J(Y) = (Y + F + C) \otimes T \otimes R \otimes W.$$

In a category of stochastic kernels, an instrumented process is $X \rightarrow J(Y)$. The implementation presents one sample through `Outcome[Y]`.

The trace object has operations:

$$\epsilon_T, \text{seq} : T \otimes T \rightarrow T, \text{par} : T \otimes T \rightarrow T.$$

`seq` is associative. `par` is associative and commutative. Both share an empty unit in the reference presentation. No general interchange equation is assumed.

The resource object has an additive operation in the sandbox. Production resource dimensions can define a typed

$$\text{algebra } R = (R_i, \oplus_i^{seq}, \oplus_i^{par}).$$

A.4 Failure-aware composition

For outcomes $o_1 \in J(Y)$ and process $g: Y \rightarrow J(Z)$, sequential composition applies g only when o_1 is successful.

Failure or cancellation is propagated with accumulated trace and resources. This is a short-circuiting bind over the success summand.

Associativity requires:

- associative trace/resource accumulation;
- stable status/failure propagation;
- leaf random streams independent of composite parenthesization;
- no wrapper that drops warnings or native evidence.

The sandbox tests pathwise associativity for sampled kernels with stable random namespaces.

A.5 Exact finite category

For finite sets X, Y , a morphism in $FinStoch$ is a matrix $K(y \mid x) \in Q_{\geq 0}$ with each row summing to one.

Composition is

$$(L \circ K)(z \mid x) = \sum_{y \in Y} K(y \mid x) L(z \mid y).$$

Tensor is

$$(K \otimes L)((y, w) \mid (x, z)) = K(y \mid x) L(w \mid z).$$

Identity is the Kronecker delta. Copy is $\Delta_X((x_1, x_2) \mid x) = 1$ exactly when $x_1 = x_2 = x$. Discard maps every input to the singleton with probability one.

A.6 Case visibility

Let C be the case object, X visible system input, and Z hidden labels. A typical case is a product $C = X \otimes Z \otimes G$ with projection $v: C \rightarrow X$. The system process factors through v ; there is no morphism from Z to the system under the declared graph.

A trial evaluator is

$$Trial_p = E \circ (id_C \otimes S_p) \circ (id_C \otimes v) \circ \Delta_C.$$

Information-flow validation checks that no alternative edge leaks Z to S_p .

A.7 Coupled pair

For two outcome kernels $K_b: X \rightarrow D(O_b)$ and $K_c: X \rightarrow D(O_c)$, a coupling is $\Gamma: X \rightarrow D(O_b \otimes O_c)$ satisfying marginal equations:

$$(id_{O_b} \otimes !_{O_c}) \circ \Gamma = K_b,$$

$$(!_{{O_b}} \otimes id_{O_c}) \circ \Gamma = K_c.$$

The product coupling samples independently. Common random numbers construct both outputs from one latent seed or disturbance. Retained-response replay can define a deterministic conditional coupling.

A.8 Observation algebra

Let an observation contain finite metrics m_i , propositions b_j , tags, and diagnostics. The summary of a multiset of observations is computed by a commutative aggregation algebra. For a numeric metric, the mergeable state is

$$(n, s, q, l, u)$$

with count, sum, squared sum, minimum, and maximum. Merge is componentwise addition with min/max. Mean and sample variance are derived. This algebra is insensitive to cell order.

The aggregate is descriptive. Statistical inference is a separate analyzer over retained cells and sampling structure.

A.9 Gate algebra

Let verdicts be $D = \{Eligible, Undecided, Rejected\}$. A rule returns pass/fail, hard/soft, and evidence.

Policy evaluation is left-to-right:

- failed hard rule yields **Rejected** and stops;
- failed soft rule yields at least **Undecided** but evaluation continues;
- all passing rules yield **Eligible**.

This is not a commutative fold. Rule order is part of policy identity.

A.10 Pareto preorder

Given metrics M with directions, normalize each value so larger is better. Candidate a weakly dominates b when

$$\forall m \in M, a_m \geq b_m,$$

and strictly dominates when at least one inequality is strict. The Pareto front contains candidates not strictly dominated by another admissible candidate.

Hard-gate rejection precedes Pareto comparison. An unsafe candidate cannot dominate a safe one through quality metrics.

A.11 Campaign coalgebra

Let history H include incumbent and immutable trials. A proposer $\pi : H \rightarrow D(P + Stop)$, experiment kernel $K : H \otimes P \rightarrow D(A)$, and update $u : H \otimes P \otimes A \rightarrow H$ induce one campaign step $H \rightarrow D(H + Terminal)$.

A deterministic static campaign is a special case. Adaptive stochastic search changes only π and optional proposer state, not the meaning of K or completed cells.

A.12 Plugin realization

Let Σ be a typed signature of operation generators and equations. A plugin set provides a realization functor

$$\rho : FreeSMC(\Sigma) / E \rightarrow C,$$

mapping operation symbols to kernels and preserving identities, composition, tensor, and declared equations. In software, a factory realizes one generator. A graph compiler interprets syntax through factories. Law certificates provide finite evidence that the realization respects selected equations.

A.13 Refinement relation

For a projection $\pi : J(Y) \rightarrow V$ and preorder $\leq_V$, define $K' \sqsubseteq K$ when for every allowed input the distribution or coupled observations of K' satisfy the required relation to K . Examples include exact equality, pathwise equality, stochastic dominance, noninterference, noninferiority, or resource improvement.

Every substitution or optimization claim names its projection and relation. “Compatible” without them is insufficient.

Appendix B. Structural operational semantics

B.1 Runtime judgments

Write

$$\rho \vdash \langle K, x, s, m \rangle \Downarrow o$$

when realization environment ρ executes kernel K on input x , root seed s , and metadata m , producing terminal outcome o . Write $\Uparrow e$ for interpreter failure.

A leaf specification k binds to implementation $\rho(k)$. Binding fails when capabilities/effects are not admitted or when bound spec identity differs.

B.2 Identity

$$\frac{\square}{\rho \vdash \langle id_x, x, s, m \rangle \Downarrow Success(x, \epsilon_T, 0_R)}$$

Identity contributes no semantic trace event or resources.

B.3 Deterministic lift

For pure function f :

$$\frac{f(x) = y}{\rho \vdash \langle lift(f), x, s, m \rangle \Downarrow Success(y, event(f), 0_R)}$$

$$\frac{f(x) = error(e)}{\rho \vdash \langle lift(f), x, s, m \rangle \Downarrow Failure(domain, e, event(fail_f), 0_R)}.$$

B.4 Sequential success

$$\frac{\rho \vdash \langle f, x, s, m \rangle \Downarrow o_1 = Success(y, t_1, r_1) \quad \rho \vdash \langle g, y, s, m \rangle \Downarrow o_2}{\rho \vdash \langle g \circ f, x, s, m \rangle \Downarrow combine_{seq}(o_1, o_2)}.$$

The same root seed s is passed. Leaves derive namespaces.

B.5 Sequential short circuit

$$\frac{\rho \vdash \langle f, x, s, m \rangle \Downarrow o_1 \quad value(o_1) = \emptyset}{\rho \vdash \langle g \circ f, x, s, m \rangle \Downarrow short(o_1, g \circ f)}.$$

No execution of g occurs. The trace records short circuit without erasing t_1 .

B.6 Tensor

$$\frac{\rho \vdash \langle f, x_1, s, m \rangle \Downarrow o_1 \quad \rho \vdash \langle g, x_2, s, m \rangle \Downarrow o_2}{\rho \vdash \langle f \otimes g, (x_1, x_2), s, m \rangle \Downarrow combine_{par}(o_1, o_2)}.$$

Premises may be scheduled in either order or concurrently. Canonical trace and leaf seed derivation make semantic output schedule-independent under the plugin laws.

B.7 Context cancellation

$$\frac{cancelled(ctx)}{\rho \vdash \{K, x, s, m\} \Downarrow Cancelled(ctxErr, event(cancel, K))}.$$

A plugin that has already crossed an external effect boundary may add an event indicating cancellation was requested but not confirmed.

B.8 Cell plan

A cell configuration is

$$\langle R, c, n, a, L \rangle,$$

where R is run config, c case, n repeat, a arm, and L ledger.

If key (c, n, a) exists and validates against R , the transition is $Reused(cell)$. Otherwise execute arm and evaluator, then append. Append success produces $Committed(cell)$. Append failure produces run-level interpreter failure; the cell is not assumed durable.

B.9 Paired completion

A paired run completes only when the ledger contains exactly two valid cells for every case/repeat coordinate. A failure outcome satisfies the coordinate; a missing line does not.

Paired aggregation joins by coordinate. Duplicate or unmatched coordinates are invalid.

B.10 Campaign transition

A campaign configuration is

$$\langle h, budget, visibility \rangle.$$

Rules include:

- **Propose**: proposer emits unseen candidate linked to a history revision;
- **Build**: builder realizes candidate or records build failure according to campaign policy;
- **Evaluate**: execute paired run;
- **Decide**: apply gate and selector;
- **Advance**: append trial and update incumbent pointer;
- **Stop**: record reason when proposer stops or budget/visibility limit is reached.

Historical trials are never rewritten.

B.11 Dynamic handler transition

A dynamic adapter encodes typed input under schema S_A , sends envelope with capability grant, receives envelope S_B , verifies schema and trace, decodes, and returns typed output. Any schema mismatch or decode error is interpreter failure. Product failure must be encoded in the declared output schema, not smuggled through transport failure.

B.12 Operational refinement obligations

An optimized interpreter can reorder/batch/cache effects only when:

- semantic leaf identities remain the same;
- random namespaces and coupling remain valid;
- trace projection preserves declared events;
- cache values verify;
- failure/fallback relation is preserved;
- resource relation satisfies claimed refinement;
- effect/capability policy remains admitted.

Appendix C. Core Go API blueprint

C.1 Process core

```
package core

type Effect string

type Spec struct {
    Kind      string
    Name      string
    Version   string
    Config     map[string]any
    Effects    []Effect
    Capabilities []string
    RandomNamespace string
    ID        string
}

type Seed [32]byte
func (s Seed) Derive(labels ...string) Seed
func (s Seed) For(spec Spec, labels ...string) Seed

type Request[A any] struct {
    Input    A
    Seed     Seed
    Metadata map[string]string
}

type Kernel[A, B any] interface {
    Spec() Spec
    Run(context.Context, Request[A]) (Outcome[B], error)
}

func Identity[A any](name string) Kernel[A, A]
func Lift[A, B any](Spec, func(A) (B, error)) Kernel[A, B]
func Compose[A, B, C any](Kernel[A, B], Kernel[B, C]) Kernel[A, C]
func Tensor[A, B, C, D any](Kernel[A, B], Kernel[C, D])
    Kernel[Pair[A, C], Pair[B, D]]
func Fanout[A, B, C any](Kernel[A, B], Kernel[A, C])
    Kernel[A, Pair[B, C]]
```

C.2 Outcome and trace

```
type Status string
const (
    StatusCompleted Status = "completed"
    StatusFailed    Status = "failed"
    StatusCancelled Status = "cancelled"
)

type Outcome[T any] struct {
    Value    *T
    Status   Status
    Failure  *Failure
    Warnings []string
    Trace    Trace
    Resources Resources
}

type Trace struct {
    Kind      TraceKind
    Label     string
    SpecID    string
    Fields    map[string]string
    Children  []Trace
}

func Event(label, specID string, fields map[string]string) Trace
func Seq(...Trace) Trace
func Par(...Trace) Trace
```

C.3 Exact finite interpreter

```
package finite

type Prob struct { /* exact rational */ }
type Dist[T comparable] map[T]Prob

type Kernel[A, B comparable] struct {
    Name string
}
```

```

Run func(A) Dist[B]
}

func Identity[T comparable]() Kernel[T,T]
func Deterministic[A,B comparable](string, func(A)B) Kernel[A,B]
func Compose[A,B,C comparable](Kernel[A,B], Kernel[B,C]) Kernel[A,C]
func Tensor[A,B,C,D comparable](Kernel[A,B], Kernel[C,D])
  Kernel[Pair[A,C],Pair[B,D]]
func Copy[T comparable]() Kernel[T,Pair[T,T]]
func Discard[T comparable]() Kernel[T,Unit]

```

C.4 Evidence

```

package evidence

type Direction string
const (
  Maximize Direction = "maximize"
  Minimize Direction = "minimize"
)

type Observation struct {
  Metrics      map[string]float64
  Constraints  map[string]bool
  Tags         []string
  Diagnostics  map[string]string
}

type Summary struct {
  CandidateID string
  Metrics      map[string]Stats
  Constraints  map[string]ConstraintStats
  Observations int
}

type PairedSummary struct {
  Metrics map[string]Stats
  Pairs   int
}

type Rule interface {
  Name() string
  Evaluate(baseline, candidate Summary, paired PairedSummary) GateResult
}

type Policy struct { Rules []Rule }
func (Policy) Evaluate(Summary, Summary, PairedSummary) Verdict

```

C.5 Experiment

```

package experiment

type Case[X any] struct {
  ID       string
  Input    X
  Expected json.RawMessage
  Tags     []string
}

type Arm[X,Y any] struct {
  Role          string
  CandidateID   string
  Kernel        core.Kernel[X,Y]
}

type Evaluator[X,Y any] interface {
  Spec() core.Spec
  Evaluate(context.Context, EvalRequest[X,Y])
    (evidence.Observation, error)
}

type Ledger interface {
  Existing(context.Context) (map[string]Cell, error)
  Append(context.Context, Cell) error
}

func RunPaired[X,Y any](
  context.Context,
  RunConfig,
  []Case[X],
  Arm[X,Y],
  Arm[X,Y],
  Evaluator[X,Y],
  Ledger,
) (RunResult, error)

```

C.6 Campaign

```
package campaign

type Candidate[P any] struct {
    ID          string
    ParentID    string
    Label       string
    Params      P
    Hypothesis  string
}

type Builder[P,X,Y any] interface {
    Spec() core.Spec
    Build(context.Context, Candidate[P]) (core.Kernel[X,Y],error)
}

type Proposer[P any] interface {
    Spec() core.Spec
    Propose(context.Context, History[P]) ([]Candidate[P],error)
}

type Selector[P any] interface {
    Spec() core.Spec
    Select(History[P], []Trial[P]) (Candidate[P],bool,error)
}
```

C.7 Plugin

```
package plugin

type Environment interface { Lookup(string)(any,bool) }

type Factory[A,B any] interface {
    Spec() core.Spec
    Bind(context.Context,Environment) (core.Kernel[A,B],error)
}

type Law interface {
    Name() string
    Check(context.Context) error
}

type Certificate struct {
    SpecID  string
    Results []LawResult
    Passed  bool
}
```

C.8 Suggested production additions

The reference API should grow only through demonstrated needs. Likely additions are:

- typed resource dimensions and serial/parallel algebras;
- explicit coupling descriptors;
- native artifact references in cells;
- visibility-class views;
- metric schemas and statistical analyzer plugins;
- distributed ledger lease/fence;
- release/reference types;
- graph compiler and impact closure;
- data-class-aware effect grants;
- typed dynamic outcome envelope.

Appendix D. Plugin law catalog

This catalog is intended for product law suites and certification manifests. A plugin lists applicable laws, test interpreter, evidence artifact, implementation build, and validity envelope.

D.1 Universal factory laws

Law	Requirement	Suggested check
Spec purity	repeated Spec() calls are equal and effect-free	direct equality and side-effect spy
Canonical ID	semantic config reorder does not change ID	canonicalization property
Material sensitivity	every declared semantic field can affect ID	field mutation test
Bind identity	bound kernel spec ID equals factory spec ID	direct assertion
Capability honesty	missing required capability makes bind fail	environment matrix
No ambient fallback	bind does not silently use undeclared global resource	isolated environment test
Lifecycle	bound instance closes or is scoped as declared	leak/lifecycle test
Trace totality	every terminal outcome has a valid trace	generated inputs
Resource validity	all reported resources are finite and valid	generated/failure inputs
Cancellation	context cancellation reaches declared boundary	cancellation injection

D.2 Deterministic-process laws

A deterministic plugin should satisfy:

- repeated execution under different root seeds yields equal output and semantic trace;
- copy naturality holds in finite/reference models;
- no random effect is declared or observed;
- equal semantic input yields equal result regardless of scheduling;
- caches are optional refinements, not sources of new behavior.

A plugin using current time, locale, mutable environment, or unordered map iteration is not deterministic until those inputs are explicit or normalized.

D.3 Stochastic-process laws

A stochastic plugin should satisfy:

- all randomness derives from declared namespace or is classified as provider-uncontrolled;
- same root seed and environment gives pathwise repeatability when claimed;
- random namespace does not collide with sibling stages;
- reparenthesization of composites does not shift draws;
- coupling class is declared;
- retries and extra calls are visible in trace;
- samples are finite and outputs validate.

For remote providers without seed control, the law certificate records only interface and trace properties plus empirical distribution checks.

D.4 Chunker laws

- each chunk belongs to exactly one source revision;
- ranges are valid and source text matches the recorded slice or canonical extraction;
- output ordering is total and deterministic;
- IDs are unique within the release;
- duplicate execution yields identical chunks;
- unchanged documents do not alter chunks for unrelated documents when locality is claimed;
- overlap and boundary constraints hold;
- no access metadata is lost;
- a deletion removes all derived chunk lineage in the maintained view.

D.5 Representation laws

- every representation resolves to one source evidence item or explicit structured source;
- generated representation never acquires source authority by itself;
- kind and prompt/model identity are material;
- deterministic kinds are seed-invariant;
- stochastic kinds retain output artifact and coupling class;

- empty or malformed generation is attributable;
- representation text and metadata satisfy size and data-policy constraints.

D.6 Embedder laws

- output dimension equals spec;
- values are finite;
- normalization matches declared metric;
- batch and single-item modes are equivalent under declared tolerance;
- item order does not alter item vectors;
- cache hit verifies semantic key;
- provider data policy permits payload;
- partial batch failure does not misassociate vectors;
- retained vector artifact identifies model revision and preprocessing.

D.7 Index-builder laws

- every indexed entry resolves to one representation/source lineage;
- manifest counts and digests verify;
- duplicate entry IDs fail or follow declared replacement semantics;
- open after build yields declared snapshot;
- exact backend matches a pure oracle on finite generated data;
- filter soundness holds;
- delete/tombstone semantics hold;
- full and incremental maintenance are equivalent at a barrier;
- compaction preserves logical view;
- approximate backend meets declared oracle relation;
- build determinism class is honest;
- crash before publish cannot produce an active partial index.

D.8 Query rewriter laws

- original query lineage retained;
- server-owned subject/policy cannot be overridden by model text;
- deterministic rewrite is seed-invariant and ordered;
- stochastic rewrite identifies provider/prompt/model and fallback;
- empty rewrite follows declared policy;
- variants are bounded;
- protected data is not disclosed to an unapproved provider;
- cache key covers release, subject partition, query, and rewrite spec.

D.9 Channel-search laws

- results belong to opened release and requested channel;
- scores are finite;
- total tie order is deterministic;
- filter soundness holds;
- completeness/approximation contract is reported;
- $\text{top-}k$ is a prefix of $\text{top-}k'$ for exact search when claimed;
- no unauthorized text is hydrated or disclosed;
- deadline/fallback is explicit;
- result cache includes release and policy partition.

D.10 Collapse and fusion laws

- collapse emits at most one result per logical evidence identity;
- every contribution references a real input channel/rank;
- input map iteration does not alter output;
- weights and constants are finite and valid;
- ties use declared stable total order;
- missing channels follow policy;
- fusion is deterministic;
- output ranks are contiguous and unique.

D.11 Reranker laws

- input candidate set is authorized before text disclosure;
- candidate IDs and returned scores align exactly;
- duplicate/missing/unknown IDs fail or follow declared policy;
- nonfinite scores are rejected;
- text composer identity is material;
- fallback/degradation is trace-visible;
- pool size and output depth are bounded;
- provider call is idempotent or retry semantics are explicit;
- common-random-number compatibility is declared;
- reranker cache key covers ordered candidates and composed text.

D.12 Evidence-admission laws

- every admitted item comes from authorized candidates;
- evidence session and release IDs are uniform;
- capacity and token/rune budgets hold;
- stable labels are unique;
- repeated admission does not silently relabel existing evidence;
- diversification reason is retained;
- presentation-only material is not promoted to answer evidence;
- citation resolution is total for emitted labels.

D.13 Generator/answer laws

- context includes only admitted evidence;
- provider disclosure policy is satisfied;
- final contract is validated before terminal success;
- unsupported citations produce failure/repair/abstention as declared;
- streamed partial output cannot become authoritative terminal output without validation;
- retry and repair attempts are trace-visible;
- cancellation produces one terminal path;
- token and provider usage are finite and attributable.

D.14 Agent laws

- tool call IDs are idempotent under replay;
- tool capability cannot be escalated by model arguments;
- each tool result belongs to the turn release/evidence epoch;
- loop is bounded or has a liveness argument;
- zero-tool completion is distinguishable from tool failure;
- repeated evidence preserves stable labels;
- final output validates against accumulated evidence;
- cancellation closes the turn and lease;
- trajectory artifact is complete enough for evaluator replay.

D.15 Evaluator laws

- system receives no hidden expected data through evaluator wiring;
- evaluator spec identifies labels/rubric/judge/aggregation;
- all terminal system statuses map to an observation or explicit evaluator failure;
- native artifact verifies and remains linked to cell;
- metric names have schemas and finite values;
- constraints are not silently converted to missing metrics;
- repeated stochastic judging uses declared namespace/coupling;
- arm labels are blinded when required;
- tag/group projection is deterministic;
- evaluator cannot mutate candidate or workload.

D.16 Gate laws

- rule result is deterministic from evidence and policy;
- failed hard rule cannot be overridden by later rules;
- rule order is retained in report;
- missing required metric fails closed or yields declared undecided state;
- metric direction is consistent;
- noninferiority margin and improvement threshold are finite;
- protected strata are not omitted from aggregate decision;
- replay from cells produces same verdict;
- a policy change changes policy identity.

D.17 Proposer laws

- proposal IDs are canonical and parented to the history revision observed;
- proposer sees only permitted history view;
- duplicate proposals are detected;
- parameter values satisfy domain constraints;
- stochastic proposals use declared seed namespace;
- stopping is explicit;
- proposer cannot rewrite prior evidence;
- model-generated candidate text is validated before becoming an executable spec;
- hidden holdout access is absent or recorded under policy.

D.18 Selector laws

- selector chooses only among eligible candidates unless policy explicitly permits undecided review;
- deterministic tie-break is total;
- selected candidate is present in trial set;
- Pareto-only selector chooses a nondominated candidate under declared dimensions;
- replay is deterministic;
- selector does not change evidence or verdicts;
- human decision is recorded as an external signed event rather than fabricated deterministic output.

D.19 Ledger laws

- append is one terminal cell per key;
- read after append returns the same cell;
- duplicate key is rejected;
- corrupt line/artifact fails closed;
- resume executes only missing cells;
- order of valid cell lines does not alter summary;
- terminal run is immutable;
- distributed implementation fences stale writers;
- artifact and cell commit protocol has a documented atomicity boundary.

Appendix E. RAG parameter, dependency, and fidelity catalog

E.1 Parameter schema

A production RAG parameter object should be nested and versioned. One possible shape is:

```
type RAGSpec struct {
  Source          SourceSpec
  Normalize       NormalizeSpec
  Chunk           ChunkSpec
  Represent        []RepresentationSpec
  Embed           EmbedSpec
  LexicalIndex    LexicalIndexSpec
  VectorIndex     VectorIndexSpec
  QueryRewrite    RewriteSpec
  Channels        []ChannelSpec
  Collapse        CollapseSpec
  Authorization   AuthorizationSpec
  Fusion          FusionSpec
  Rerank          *RerankSpec
  Admission       AdmissionSpec
  Answer          *AnswerSpec
  Agent           *AgentSpec
  Serving         ServingSpec
  Presentation    PresentationSpec
}
```

Each child has its own semantic ID. The release root commits to the composition.

E.2 Dependency table

Changed factor	Directly invalidated	Downstream evidence requiring repeat
source snapshot	normalized documents and affected descendants	all levels for changed content; freshness
admission policy	admitted documents, indexes/filters	security, retrieval, answer, session
normalizer	affected documents onward	retrieval through presentation
chunker	chunks, reps, embeddings, indexes	retrieval through presentation
representation prompt/model	representation kind onward	retrieval through presentation
embedding model	vectors/vector index	retrieval through presentation
lexical analyzer/build fields	lexical index	retrieval through presentation
ANN construction	vector index	oracle, retrieval, answer, load
ANN query effort	query observations only	oracle, retrieval, answer, load
synonyms/rewrite	query observations only	retrieval through presentation
channel depth	query observations only	retrieval through presentation
authorization implementation	query/policy trace	security, retrieval, answer, session
fusion	fused rank onward	retrieval through presentation
reranker	rerank onward	security, retrieval, answer, load
context admission	context onward	answer, agent, presentation
answer prompt/model	answer onward	answer, session, presentation
agent policy/tool description	trajectory onward	session, presentation
deadline/fallback	operational and possibly outcome	failure/load plus affected outcome levels
frontend reducer/widget policy	presentation	session/frontend only

E.3 Semantic classes

Operational-preserving. Claimed to preserve protected system distribution/trace projection while changing execution resources. Example: batching with exact result order.

Approximation-changing. Replaces exact computation with a declared relation. Example: ANN search.

Relevance-changing. Changes evidence ranking but not source knowledge or policy. Example: fusion weights.

Knowledge-projection-changing. Changes what source material is searchable or how it is represented. Example: chunking or generated questions.

Policy/security-changing. Changes admission, authorization, disclosure, or retention. Normally requires independent review and hard gates.

Interaction-changing. Changes agent/tool trajectory, choices, or conversation semantics.

Presentation-changing. Changes customer-visible projection without necessarily changing evidence or answer.

An intervention can belong to several classes.

E.4 Minimum fidelity table

Intervention	Static laws	Retrieval	Answer	Session/UI	Load/refresh	Canary
pure fusion weight	yes	required	usually	optional	latency	recommended
lexical synonym set	yes	required	required	optional	optional	recommended
chunking	yes	required	required	product-dependent	refresh required	recommended
generated representation prompt	yes	required	required	product-dependent	cost/refresh	recommended
embedding model	yes	required	required	product-dependent	build/load	required for major change
ANN query effort	yes	oracle required	spot check	no	load required	recommended
ANN construction/backend	yes	oracle required	required	no	build/update/load	required
remote reranker	security laws	required	required	product-dependent	failure/load	required
context budget	yes	diagnostic	required	product-dependent	latency/cost	recommended
answer model/prompt	contract	retrieval fixed	required	required if interactive	load/cost	required
agent/tool policy	capability laws	diagnostic	partial	required	load/cost	required
frontend reducer	reducer laws	no	no	required	reconnect/load	canary
refresh/compaction policy	maintenance laws	temporal	temporal	temporal	required	staged production

E.5 Coupling table

Component change	Preferred coupling	Caveat
deterministic pure stage	exact replay	no stochastic difference
same model/prompt, parameter tweak downstream	shared retained output	strongest reuse
same stochastic component semantics	common random namespace	semantic call IDs must align
changed prompt but same provider	shared root seed at stage	provider may not honor seed comparably
changed provider/model	independent or artifact-based	shared integer seed is not shared latent noise
changed chunking	source/case pairing only	item-level generated calls no longer align
changed agent policy	shared user case and selected environment noise	tool sequence can diverge
online canary	randomized traffic assignment	environment not pathwise coupled

E.6 Resource algebra table

Resource	Serial composition	Parallel composition	Order
monetary cost	sum	sum	minimize
provider calls	sum	sum	minimize/budget
input/output tokens	sum	sum	minimize/budget
CPU work	sum	sum	minimize
wall-clock latency	sum on critical chain	maximum plus overhead	minimize/budget
peak memory	maximum or declared combination	sum/max by isolation	minimize/budget

Resource	Serial composition	Parallel composition	Order
index bytes	material union	material union	minimize/budget
disclosure set	union	union	subset constraint
privacy loss	mechanism-specific composition	mechanism-specific	budget
freshness lag	terminal/source difference	maximum relevant lag	minimize/budget
failure severity	join in severity lattice	join	constraint

The sandbox uses additive maps for simplicity; production code should encode these operations by resource type.

E.7 Example candidate declaration

```
api_version: rag-intervention/v1
candidate_id: gec-rerank-bge-v3
parent_release: rel_...
parameter_patch:
  rerank:
    model: bge-reranker-v3
    pool: 24
    blend:
      retrieval: 0.35
      reranker: 0.65
semantic_classes:
  - relevance-changing
  - disclosure-changing
  - operational-changing
direct_targets:
  - query.rerank
dependency_closure:
  - query.rerank
  - query.admission
  - answer
  - session
claimed_invariants:
  - authorization-before-disclosure
  - release-pinned_evidence
required_fidelities:
  - security-spy
  - retrieval
  - answer
  - load
  - canary
coupling:
  class: common-case-independent-provider
hypothesis: >
A stronger reranker improves authoritative first-hit rank while
preserving recall and remaining inside latency and disclosure policy.
```

E.8 Example gate program

```
api_version: gate-policy/v1
rules:
  - kind: constraint_all
    name: authorized_remote_disclosure
    hard: true
  - kind: constraint_all
    name: release_coherent
    hard: true
  - kind: complete_pairing
    hard: true
  - kind: noninferior
    metric: recall_at_5
    direction: maximize
    margin: 0.0
    statistic: paired_lower_confidence_bound
    hard: true
  - kind: noninferior_by_group
    metric: groundedness
    groups: [restricted, procedure, troubleshooting]
    margin: 0.02
    hard: true
  - kind: budget
    metric: latency_p95_ms
    maximum: 900
    hard: true
  - kind: budget
    metric: remote_disclosed_bytes
    maximum: 180000
    hard: true
  - kind: improve
    metric: mrr
    minimum: 0.01
    hard: false
selector:
  kind: pareto_then_review
```

```
dimensions:  
  mrr: maximize  
  latency_p95_ms: minimize  
  provider_cost: minimize
```

Appendix F. Sandbox execution, extension, and verification

F.1 Purpose of the sandbox

The accompanying sandbox is a constructive model of the thesis rather than a production framework. It demonstrates that a small typed core can support exact categorical laws, reproducible stochastic execution, plugin binding, exact paired evidence custody, ordered gates, Pareto comparison, adaptive candidate selection, and a nontrivial RAG graft without importing RAG vocabulary into the core.

The implementation deliberately omits distributed scheduling, remote artifact stores, production authentication, durable fencing across processes, provider SDKs, deployment activation, and large-scale statistical libraries. Those omissions keep the semantic boundary inspectable. They also prevent the sandbox from being mistaken for a drop-in replacement for the existing ragopt runtime.

The repository is self-contained, uses only the Go standard library, and requires Go 1.23 or later. The module path is `example.com/probopt`.

F.2 Reproduction commands

From the sandbox root:

```
make verify
make demo
```

The equivalent explicit commands are:

```
go test ./...
go vet ./...
go test -race ./...
go run ./cmd/probopt-demo \
  -out out \
  -seed 20260809 \
  -repeats 5
```

The demo writes:

```
out/
  report.md
  report.json
  pareto.csv
  runs/
    <candidate-run>/
      cells.jsonl
```

`report.json` is the complete machine-readable campaign history. `report.md` is a human projection. `pareto.csv` is a compact decision projection. Each `cells.jsonl` file contains the exact case/repeat/arm coordinates and observations for one candidate comparison. Re-executing the command against the same output directory validates the existing cell ledger and skips completed coordinates.

The root seed is not used as a mutable global random stream. Every stochastic leaf derives a seed from the root, its semantic random namespace, and stable call labels. This makes execution reproducible and permits common-random-number coupling where semantically valid.

F.3 Package inventory

Package	Public semantic role	Deliberate exclusions
core	canonical specs, effects, capabilities, seeds, outcomes, traces, typed sampled kernels, serial and parallel composition	domain metrics, candidates, storage policy
finite	exact rational distributions and finite stochastic kernels	large state spaces, continuous distributions
plugin	pure specification/runtime binding, effect policy, law certificates, typed factory contracts, dynamic boundary codec	reflective service location, domain registries
evidence	metric directions, observations, paired summaries, constraints, gates, Pareto relation	system execution, candidate proposal

Package	Public semantic role	Deliberate exclusions
experiment	cases, evaluators, exact cells, paired runner, coupling, resumable ledgers	metric meaning, deployment
campaign	candidate values, builders, proposers, selectors, history and incumbent evolution	product evaluation semantics
ragtoy	example RAG parameters, component plugins, index build, authorized hybrid retrieval, stochastic reranking, evaluator	production search quality
compat/ragoptv1	JSON-compatible projection to the current ragopt outcome boundary	compile-time dependency on supplied repository
cmd/probopt-demo	executable law certificate and RAG campaign	reusable API

The package graph is intentionally acyclic in semantic authority. The core package does not import evidence, experiment, campaign, or ragtoy. The optimizer observes a system through typed outcomes and evaluators; the system does not know which optimizer will inspect it.

F.4 Exact finite interpreter

`finite` implements distributions as finite maps from values to exact rational probabilities. A finite stochastic kernel is a function from each input value to such a distribution. Composition is matrix multiplication in rational arithmetic:

$$(g \circ f)(c \mid a) = \sum_b f(b \mid a) g(c \mid b).$$

The law suite constructs finite domains and checks equality exactly. The demo emits the following certificate:

Law	Result
left identity	true
right identity	true
associativity	true
discard naturality	true
deterministic-copy naturality	true
copy commutativity	true

The copy laws distinguish one shared sample from two independent samples. For a stochastic value x , the Markov-category copy map is $x \mapsto (x, x)$ after one sample. It is not equivalent to executing the producer twice. This distinction becomes operationally relevant in paired experiments, shared evidence, and fan-out stages.

`SampleOrdered` requires an explicit support order. Go map iteration order is intentionally unspecified, so deriving a sample from map iteration would make replay depend on runtime accident rather than semantic identity. The test suite includes a regression case for this requirement.

F.5 Sampled interpreter

The runtime kernel type is conceptually:

```
type Kernel[A, B any] struct {
  Spec Spec
  Run func(context.Context, Seed, A) (Outcome[B], error)
}
```

An outcome is either a successful value, a product failure, or cancellation, together with a normalized trace, resource vector, warnings, and metadata. Product failure is a value because it belongs in paired evidence and denominators. A Go error is reserved for failure of the interpreter contract itself: corrupt storage, invalid binding, impossible state, or context infrastructure failure.

Sequential composition executes the first kernel, short-circuits product failure or cancellation, and otherwise passes the value to the second kernel. It combines traces with ordered `Seq`, combines resources by the declared sandbox algebra, and retains warnings. Parallel composition uses `Par`, which is normalized as a commutative node by child digest. Operational goroutine completion order is not part of semantic trace identity.

A central implementation correction was required for associativity. A naive composite that repeatedly splits a mutable random stream can make

$$h \circ (g \circ f)$$

sample differently from

$$(h \circ g) \circ f.$$

The sandbox instead passes one root seed through the composition tree. Stochastic leaves independently derive their draw from the root and their stable semantic namespace. Re-parenthesization therefore preserves leaf draws and normalized trace, provided leaf specifications and call labels are unchanged. A test asserts this pathwise property.

This is stronger than equality in distribution and is useful for differential testing. It should not be elevated to a universal law of all stochastic implementations. External model providers may not expose comparable latent randomness, and changing a prompt or model can make shared integer seeds scientifically meaningless. The experiment layer therefore records the coupling class explicitly.

F.6 Structured traces

The trace algebra contains:

- **Empty**, the identity trace;
- **Event**, a versioned semantic observation;
- **Seq**, ordered composition;
- **Par**, commutative composition.

Constructors normalize nested sequence and parallel nodes. Empty children are removed. Parallel children are sorted by semantic digest. The trace intentionally excludes wall-clock timestamps, machine names, goroutine IDs, retry sleep, and other deployment accidents from its semantic digest. Such data can be retained as operational annotations outside the equality relation.

A production implementation should distinguish at least three projections:

1. **semantic trace**, used for identity and equivalence;
2. **audit trace**, retaining policy, provenance, disclosure, and exact artifacts;
3. **telemetry trace**, retaining timing, host, queue, and provider details.

The sandbox combines a small subset for readability. The thesis architecture keeps the distinction explicit.

F.7 Plugin binding

The principal typed plugin interface is:

```
type Factory[A, B any] interface {
  Spec() core.Spec
  Bind(context.Context, plugin.Environment) (core.Kernel[A, B], error)
}
```

Spec is pure, canonical, and behaviorally material. It includes plugin kind and version, configuration, random namespace, declared effects, required capabilities, and schemas. Bind resolves runtime clients, credentials, local stores, worker pools, and process resources. A binding can fail if capabilities are absent. It cannot silently rewrite the semantic spec to fit the host.

This split has five consequences.

First, a candidate can be identified and compared before expensive resources exist. Second, plugin law certification can target a stable spec. Third, release manifests can include semantic plugin configuration without embedding credentials. Fourth, deployment changes such as worker count need not contaminate experiment identity unless they can alter protected outcomes. Fifth, the same spec can be interpreted by exact, simulated, local, or production binders.

The `plugin.Catalog` stores descriptors and law certificates. It does not return untyped services. Ordinary in-process composition uses typed Go fields and constructors. The catalog answers introspective questions such as what plugin kinds exist, which schema version they implement, which capabilities they require, and which law certificate accompanies them.

This is an important negative design decision. A universal `Get(name) any` registry would erase port types at precisely the layer intended to provide strong semantics. It would also make dependency graphs and effect policies runtime conventions rather than compile-time structure.

F.8 Law certificates

The sandbox certificate is a compact value consisting of suite identity, plugin/spec identity, named law results, and diagnostics. It is not a cryptographic attestation. A production certificate should additionally bind:

- exact source and binary identity of the plugin;
- fixture and generated-test artifacts;

- property-test seeds and shrink results;
- environment and dependency versions;
- effect-policy decision;
- validity period or release scope;
- signer or CI authority;
- links to native reports.

Certification is an admission mechanism, not proof that the plugin is universally correct. Laws should be selected according to semantic claims. A deterministic chunker can claim repeatability and document locality. A stochastic reranker can claim bounded finite scores, namespace reproducibility, and authorization-before-disclosure. An ANN backend claims a workload-conditioned approximation relation, not equality with exact search.

F.9 Evidence and exact cells

An experiment case contains public input and evaluator-only expected material. An arm executes the system. An evaluator maps the native outcome and hidden expected material to an observation. The runner, rather than the arm, assigns the exact coordinate:

```
| (run, case, repeat, arm)
```

Baseline and challenger receive the same case/repeat root seed under the declared common-random-number coupling. They derive stage draws from namespaces. This creates a coupling between two marginal systems without placing the coupling inside either system's denotation.

The runner appends one terminal cell per coordinate. Missing metrics remain missing; failed outcomes remain cells; an evaluator cannot drop an inconvenient case. The file ledger rejects duplicate keys, including after reopening the process. It is suitable for a local demonstration but not a distributed writer set: production storage requires fencing and an atomic artifact/cell commit protocol.

F.10 Decision algebra

An observation contains finite typed metrics, constraints, diagnostics, outcome class, and native artifact references. Metrics have explicit direction. Paired summaries compute candidate-minus-baseline differences and orient them so positive values are beneficial where needed.

The gate program is ordered. In the demonstration it evaluates:

1. all authorization constraints;
2. all finite-score constraints;
3. recall noninferiority;
4. MRR noninferiority;
5. latency budget;
6. index-size budget;
7. target MRR improvement.

A failed hard rule stops eligibility. A failed soft target yields undecided rather than silently treating the candidate as eligible or rejected. This distinction supports review and later evidence collection.

Pareto comparison is applied after eligibility. It does not replace gates. Hard security or integrity constraints are not dimensions that can be compensated by lower cost. Among eligible candidates, Pareto preserves real incomparability among quality, latency, and resource use.

F.11 Adaptive campaign

A campaign contains an incumbent, a history, a proposer, a builder, an experiment runner, a gate policy, and a selector. One round is a state transition:

$$(H_t, p_t) \rightarrow (H_{t+1}, p_{t+1}).$$

The proposer receives a bounded view of immutable history and returns candidate parameter values. The builder creates a typed arm for each candidate. The runner obtains cells. The decision layer determines eligibility and comparison. The selector chooses an eligible successor or retains the incumbent.

This decomposition allows grid search, Bayesian optimization, model-generated proposals, human design, or evolutionary search to share trial semantics. An adaptive proposer cannot alter prior cells, hidden labels, evaluator policy, or gate definitions. Changes to those assets begin a new campaign identity.

The sandbox proposer is static because the purpose is to exercise the boundary, not to implement a sophisticated search algorithm. A production Bayesian or bandit proposer would fit the same interface while carrying posterior/control state in its own typed history projection.

F.12 Toy RAG graft

ragtoy supplies a small but complete domain adapter. Its parameter value includes chunk size and overlap, representation strategy, lexical/vector weights, candidate depth, rerank controls, and resource estimates. The builder composes:

1. corpus chunking;
2. optional title-aware representations;
3. hashed vector embedding;
4. lexical and vector index construction;
5. server-owned authorization prefilter;
6. hybrid retrieval;
7. optional stochastic reranking;
8. bounded result projection.

The query kernel never receives hidden relevance labels. The independent evaluator checks expected documents, authorization, and finite scores and emits recall, MRR, precision, latency, and index-unit observations.

The toy retriever is intentionally simplistic. Its architectural value is that the generic core never imports Document, Chunk, Embedding, RRF, or Rerank. Those concepts appear only in typed domain plugins and the builder. The same generic experiment, evidence, and campaign packages could host a SQL optimizer, compiler pass tuner, control policy, prompt optimizer, or approximate numerical backend.

F.13 Demonstration results

The reference run uses ten cases and five paired repeats, producing fifty paired coordinates per candidate. Its candidate summary is:

Candidate	Verdict	Recall	MRR	Precision	Latency ms	Index units	Pareto
bounded rerank	eligible	0.900	0.850	0.333	1.013	688	yes
expensive wide	rejected	0.900	0.850	0.333	1.551	1,768	no
lexical heavy	eligible	0.900	0.783	0.333	0.579	631	yes
smaller overlap	eligible	0.900	0.800	0.333	0.739	922	yes
title hybrid	undecided	0.900	0.750	0.333	0.586	688	no

The selected incumbent is bounded-rerank. expensive-wide attains the same retrieval measurements but violates the hard latency budget. title-hybrid passes all hard constraints but does not meet the soft target-improvement rule. The remaining eligible candidates lie on different quality/resource trade-offs.

These numbers are properties of the toy workload, not recommendations for the supplied RAG products. They demonstrate that the evidence and decision layers distinguish equal relevance with different cost, hard rejection from lack of positive evidence, and a Pareto frontier from a single weighted score.

F.14 Adding a new domain

A new domain requires five decisions.

F.14.1 Define typed ports

Choose input X , output Y , and parameter P types. Use domain values, not `map[string]any`. An outcome can carry native artifacts by reference.

```
type Params struct { /* semantic candidate values */ }
type Request struct { /* public system input */ }
type Result struct { /* native system output */ }
```

F.14.2 Define a builder or factory

A factory binds one component. A campaign builder instantiates a complete arm from a candidate parameter value. Component composition can remain ordinary Go when a reified graph provides no additional analysis value.

```
type Builder struct { /* locked assets and factories */ }

func (b Builder) Build(
    ctx context.Context,
    c campaign.Candidate[Params],
) (experiment.Arm[Request, Result], error)
```

F.14.3 Define an independent evaluator

The evaluator receives the native outcome and evaluator-only case material. It emits typed observations and keeps a native artifact authoritative.

```
type Evaluator struct{
func (Evaluator) Evaluate(
    ctx context.Context,
    c experiment.Case[Request, Expected],
    out core.Outcome[Result],
) (evidence.Observation, error)
```

F.14.4 Define gate and comparison policy

Declare hard constraints, noninferiority, target improvements, resource budgets, protected groups, and Pareto dimensions. Version the policy and keep it fixed for the campaign.

F.14.5 Certify plugin laws

Write ordinary tests and property tests for domain claims. A plugin without a law suite can still be bound under a permissive development policy, but it should not enter a production release or promotion campaign that requires certification.

F.15 Adapting current `ragopt`

The supplied `ragopt` package already provides mature operational custody. The lowest-risk integration is not a replacement. A product arm can:

1. construct its system through `core`, `plugin`, and RAG-domain factories;
2. run one typed case with a stable root seed and release/candidate spec;
3. retain the full structured trace and product-native artifact;
4. project metric, failure, usage, and artifact references into the current `ragopt` outcome structure;
5. let the current `ragopt` runner own exact cells, resume, comparison, gates, and reports.

The sandbox `compat/ragoptv1` package models this JSON boundary without importing the original module. Longer-term extraction can happen incrementally: canonical process specs and random namespaces first; typed evidence directions and constraints second; plugin binding and certificates third; adaptive control and RAG parameter spaces later.

The present operational run store should remain authoritative until a new interpreter proves equivalent under interruption, duplicate, corruption, and resume tests. Mathematical elegance is not a reason to discard working evidence custody.

F.16 Verification record

The delivered sandbox was checked with:

```
go test ./... PASS
go vet ./... PASS
go test -race ./... PASS
```

Tests cover exact finite laws, sampled associativity under namespaced randomness, canonical specifications, trace normalization, plugin effect policy, law certification, finite metric validation, gate and Pareto behavior, exact paired cells, resume, duplicate ledger rejection after reopen, campaign selection, RAG authorization, and toy retrieval behavior.

The race detector does not prove distributed safety or absence of all races. It verifies the exercised in-process code paths under Go's race instrumentation. Likewise, exact finite laws certify the exact interpreter and selected constructions, not arbitrary third-party plugins.

F.17 Known limitations

The sampled distribution is represented by one reproducible execution, not an enumerable probability law. Statistical estimation occurs through repeated cells. Continuous kernels, inference objects, posterior distributions, and differentiable parameter updates are not implemented.

Resource composition uses additive numeric maps in the sandbox. Production resources need typed algebras: latency follows critical paths, peak memory may use maxima or interference models, disclosure is set union, failure severity is a lattice, and privacy loss follows mechanism-specific composition.

The local JSONL ledger is not safe for multiple processes. It has no distributed lease, fencing token, transaction with external artifact stores, or compaction. The dynamic plugin boundary validates schema tags but is not a secure code-loading mechanism. Untrusted plugins require process isolation, capability enforcement, signatures, and data-policy controls.

The toy RAG index is memory-resident and built for every candidate. It does not model incremental corpus changes, ANN approximation, generation, agent loops, or frontend projection. Those cases are formalized in the thesis and fit the same interfaces, but are not all reproduced in this compact implementation.

Appendix G. Empirical source map and current-to-target relation

G.1 Repository scale

The supplied archive contains five principal scopes used by the study:

Scope	Go files	Nonblank Go lines	Go test functions
ragopt	45	5,925	42
ragkit	173	17,743	273
RAG-TTC	515	76,705	906
GEC RAG	200	28,668	252
TTC Garden backend	70	8,485	108
Total	1,003	137,526	1,581

The count is descriptive. Architectural conclusions derive from contracts, state transitions, tests, and dependency direction rather than size.

G.2 ragopt source findings

The current package has a disciplined experimental spine. Candidate creation copies exact assets and enforces one declared mutation. Suites and cases are immutable inputs. Evaluation coordinates include case, repeat, and arm. Run custody retains failures, supports resume, and separates reports from application. Comparison requires exact pairs, and gates are ordered rather than a universal weighted objective.

These properties should remain. They correspond to the evidence and decision layers of this thesis.

The package does not yet expose a typed semantic process $X \rightarrow D(Y)$, a process specification independent of runtime binding, structured trace equivalence, declared effects and capabilities, named random sources, explicit coupling, typed parameter dependency closure, evaluator objects independent of arms, or adaptive proposer state. Product arms therefore hide most system semantics behind callbacks.

The proposed backbone fills those gaps beneath and beside current custody rather than merging all concerns into the run store.

G.3 ragkit source findings

ragkit already contains many deterministic and effectful morphisms suitable for domain plugins:

- document, chunk, representation, and evidence values;
- deterministic chunking and representation projections;
- provider-backed generated representations and embeddings;
- lexical and vector search interfaces;
- collapse, fusion, reranking, hydration, and context construction;
- grounded answer validation;
- immutable index bundles and verified opening;
- bounded execution, retries, cache keys, rate and budget controls;
- retrieval metrics and evaluation helpers.

The target relation is to give these components pure specs, effect/capability declarations, stable random namespaces, law suites, and typed composition plans where static analysis is useful. RAG semantics stay in ragkit; the probabilistic optimization core remains domain-neutral.

A chunker factory can claim determinism, locality, and exact lineage. An embedding factory declares remote text disclosure, model identity, dimensions, and cache semantics. A reranker declares candidate/evidence ports, provider policy, random namespace, and fallback. A query plan composes them into one release-specific process. The optimizer receives a parametrized family of such plans.

G.4 RAG-TTC source findings

RAG-TTC combines a copied common RAG substrate with product and research functionality. Relevant optimization cases include representation families, complete-corpus builds, provider caches, committed-Git snapshots, workspace artifacts, exact vector search, an HNSW candidate and bakeoff, route-dependent lexical/vector retrieval, connected augmentation, model-invoked search, turn-scoped evidence, answer/tool evaluation, and persistent chat state.

This repository is the strongest test bed for the proposed architecture because it exercises both build-time and query-time parameter changes. Its copied common substrate should still be removed first. Probabilistic abstractions should not become a reason to preserve two implementations of chunking, representation, retrieval, or bundle semantics.

After the cutover, product cases can be grafted as:

- Git snapshot and corpus admission factories;
- chunking/representation/embedding/index parameter nodes;
- exact and ANN backend interpreters with distinct law certificates;
- route, channel, fusion, and rerank query factories;
- connected retrieval as an effectful plugin with data-policy capabilities;
- tool-loop evaluator as a product-native statistical game;
- session calibration as a higher-fidelity evaluator;
- static, Bayesian, or model-driven proposers over typed parameter subspaces.

The existing ANN bakeoff maps to an approximation-refinement game: an exact kernel is the reference, ANN is the candidate, workload queries define cases, recall/latency/resource observations form evidence, and the gate expresses an approximation tolerance plus operational budgets.

G.5 GEC source findings

GEC directly consumes ragkit for immutable search bundles and adds product policy: access scopes, source roles, curated lexical synonyms, weighted hybrid fusion, optional cross-encoder reranking, fail-open fallback, answer evaluation, parameter sweeps, run-scoped evidence, structured tools, and an administrative chat/frontend.

GEC's RRF sweep is a clean example of a restricted parameterized process. It freezes channel rankings and changes only fusion parameters. The current method is efficient because it reuses sufficient intermediate evidence. Its semantic limitation is scope: it cannot support claims about chunking, embedding, index recall, reranker, answer behavior, or production failure.

The proposed architecture lets each campaign declare which process cut is frozen. A fusion-only evaluator can consume retained channel rankings as sufficient statistics. A reranker campaign must rerun candidate disclosure and ranking. A chunking campaign must rebuild downstream artifacts. A query-service campaign may require answer and frontend evaluation.

GEC's authorization order remains a critical product law: access policy must dominate any remote text-bearing stage. A reranker plugin cannot obtain a production law certificate if unauthorized text can reach its input. This turns a product security rule into a compositional precondition.

G.6 Garden source findings

The Garden assistant adds intent-specific routes, structured product facts, connected retrieval, per-conversation evidence, evidence-bound widgets, field-level provenance, conflict suppression, customer/developer projections, and multi-turn calibration.

These capabilities show why the optimization output type cannot be fixed to a ranked list or answer string. Garden evaluation is a statistical game over trajectories and typed presentation state. A candidate can improve retrieval while breaking source-card grouping, widget eligibility, fact coherence, or choice behavior.

The correct graft keeps Garden schemas and product validators in Garden. Shared process/evidence layers provide release-pinned evidence references, effects, traces, exact cases/repeats, and decision custody. A Garden evaluator plugin maps the native session artifact to task completion, provenance, conflict, interaction, latency, and cost observations.

G.7 Current-to-target map

Current construct	Formal role	Target relation
ragopt candidate bundle	global parameter element plus locked assets	retain; add typed parameter/spec projection
ragopt arm callback	opaque sampled system plus evaluator	split into typed system arm and independent evaluator
case/repeat/arm key	experimental coordinate	retain exactly
run store	operational experiment interpreter	retain; add coupling/trace/spec fields

Current construct	Formal role	Target relation
current metric map	observation projection	add metric schemas, directions, finite validation
ordered gates	decision algebra	retain and strengthen with typed rules
native artifact	intensional/product evidence	retain as authority
ragkit component interface	domain morphism	add pure spec, effects, capabilities, laws
flow/cache/retry	operational interpreter	keep below denotational process identity
RAG parameter sweep	finite proposer over one cut	express as candidate space and dependency closure
ANN bakeoff	refinement/approximation game	certify against exact interpreter and workload
GEC judge/eval	product evaluator	keep product-owned; project common evidence
Garden calibration	trajectory evaluator	keep product-owned; use paired campaign custody
plugin registry temptation	untyped service locator	replace with typed wiring plus descriptor catalog

G.8 Proposed extraction sequence

1. Introduce canonical component/process specs and semantic IDs without changing current execution.
2. Add random namespaces and exact case/repeat/stage seed derivation.
3. Add structured trace references to native artifacts and current outcomes.
4. Extract typed metric direction, constraint, and finite-value schemas.
5. Define component factory interfaces in `ragkit` around existing implementations.
6. Add law suites for deterministic ordering, lineage, authorization, cache soundness, and backend refinement.
7. Introduce independent evaluator interfaces in product adapters while retaining current `ragopt` cells.
8. Add typed RAG parameter spaces and dependency closure.
9. Add proposer/selector interfaces only after evidence semantics are stable.
10. Migrate campaigns incrementally; do not replace current run custody until differential and interruption tests pass.

The order is intentionally conservative. A categorical surface without stable identity, evidence, and laws would add vocabulary without increasing assurance.

G.9 Empirical limitations

The source study uses the supplied archive, which may differ from active deployment branches. The GEC snapshot references an `internal/knowledgebuild` package absent from the extracted files, so build-specific claims rely on call sites and design records rather than direct implementation inspection.

The original repositories target a newer Go toolchain than was available offline in the analysis environment. They were inspected statically rather than rebuilt as one system. The accompanying ProbOpt sandbox is independent and was compiled, tested, vetted, race-tested, and executed in the available Go environment.

No production traffic, private corpus, provider account, or latency telemetry was used. Demonstration metrics are synthetic. The thesis proposes statistical and operational protocols; it does not claim measured production gains.

Appendix H. Selected bibliography

The bibliography emphasizes primary sources that motivate the structures used in the thesis. The implementation does not depend on these works at runtime.

H.1 Probability and Markov categories

Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239.

This work develops Markov categories as symmetric monoidal categories with copying and discarding, providing the central process language used here. The thesis uses only a practical finite and sampled fragment and does not claim to implement the full theory.

Fritz, Tobias, Tomáš Gonda, Paolo Perrone, and Eigil Fjeldgren Rischel. 2023. “Representable Markov Categories and Comparison of Statistical Experiments in Categorical Probability.” *Theoretical Computer Science* 961: 113896.

The comparison of statistical experiments informs the distinction between a system’s marginal behavior, couplings used for evidence, and decision-relative informativeness.

Di Lavore, Elena, and Mario Román. 2023. “Evidential Decision Theory via Partial Markov Categories.” In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science*.

Partial Markov categories motivate treating failure, nontermination, or undefined behavior explicitly rather than conditioning them away. The sandbox models product failure and cancellation as outcomes while reserving interpreter errors for broken execution contracts.

Giry, Michèle. 1982. “A Categorical Approach to Probability Theory.” In *Categorical Aspects of Topology and Analysis*, Lecture Notes in Mathematics 915. Springer.

The Giry monad is a foundational model for measurable stochastic maps. The sandbox avoids measure-theoretic implementation and uses finite exact distributions plus sampled kernels.

H.2 Parametrized processes and learning

Cruttwell, G. S. H., Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. 2022. “Categorical Foundations of Gradient-Based Learning.” In *Programming Languages and Systems: ESOP 2022*.

The Para construction and compositional learning perspective motivate treating candidates as parameter values of typed processes rather than arbitrary mutable configuration files.

Fong, Brendan, David Spivak, and Rémy Tuyéras. 2019. “Backprop as Functor: A Compositional Perspective on Supervised Learning.” In *Proceedings of LICS 2019*.

This line of work illustrates how learning algorithms can be separated from forward system composition. The present architecture generalizes that separation beyond differentiable learning to black-box and mixed discrete optimization.

Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55–92.

Monadic semantics motivates the Kleisli-style composition of effectful or stochastic computations. The thesis uses Markov-category language when copying/discarding and probability are central.

Hughes, John. 2000. “Generalising Monads to Arrows.” *Science of Computer Programming* 37 (1–3): 67–111.

Arrows motivate typed composition for computations whose structure should remain statically inspectable rather than hidden behind unrestricted monadic binding.

McBride, Conor, and Ross Paterson. 2008. “Applicative Programming with Effects.” *Journal of Functional Programming* 18 (1): 1–13.

Applicative structure informs the distinction between statically inspectable parallel composition and dynamic data-dependent control.

H.3 Lenses, statistical games, and cybernetics

St. Clere Smithe, Toby. 2021. “Bayesian Lenses.” arXiv:2109.04461.

Bayesian lenses motivate compositional interfaces that pair forward stochastic behavior with backward inference or evaluation. The thesis adopts the separation of system and evaluator without requiring every product evaluator to be Bayesian.

St. Clere Smithe, Toby. 2023. “Approximate Inference via Fibrations of Statistical Games.” arXiv:2306.17009.

Statistical games motivate treating evaluation as structured additional semantics over systems and contexts, rather than as a metric function embedded in the system plugin.

Ghani, Neil, Jules Hedges, Viktor Winschel, and Philipp Zahn. 2018. “Compositional Game Theory.” In *Proceedings of LICS 2018*.

Open games motivate explicit interfaces between local processes, contexts, utility/evaluation, and composition. Optimization campaigns in this thesis are not open games in the formal sense, but the architectural separation is analogous.

Capucci, Matteo, Bruno Gavranović, Jules Hedges, and Eigil Fjeldgren Rischel. 2022. “Towards Foundations of Categorical Cybernetics.” *Electronic Proceedings in Theoretical Computer Science* 372.

Categorical cybernetics motivates viewing learners, controllers, and adaptive systems as compositional feedback structures. The adaptive campaign layer is a deliberately small engineering interpretation.

H.4 Open systems and compositional architecture

Fong, Brendan. 2015. “Decorated Cospans.” *Theory and Applications of Categories* 30: 1096–1120.

Baez, John C., and Kenny Courser. 2020. “Structured Cospans.” *Theory and Applications of Categories* 35: 839–897.

Fong, Brendan, and David I. Spivak. 2019. *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press.

These works motivate open-system descriptions with explicit boundaries and compositional wiring. The thesis uses typed ports and plugin factories as an engineering realization rather than implementing a cospan library.

H.5 Operational semantics and state

Plotkin, Gordon D. 1981/2004. “A Structural Approach to Operational Semantics.” *The Journal of Logic and Algebraic Programming* 60–61: 17–139.

Small-step rules in the thesis make cell execution, failure, ledger commit, campaign transition, and plugin binding explicit. They complement denotational stochastic arrows rather than replacing them.

Kahn, Gilles. 1974. “The Semantics of a Simple Language for Parallel Programming.” In *Information Processing* 74.

Kahn-style process semantics motivate deterministic stream/network interpretations and the separation of semantic order from scheduler order.

Lamport, Leslie. 1994. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16 (3): 872–923.

Lamport, Leslie. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

TLA/TLA+ motivates finite protocol models for experiment custody, build/activation state machines, and adaptive campaign histories.

H.6 Testing and verification

Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of ICFP 2000*.

Property-based testing is the practical bridge from algebraic laws to executable assurance. The sandbox uses ordinary generated and table-driven Go tests; production adoption should add richer generators and shrinking.

H.7 Retrieval and RAG

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, et al. 2020. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems* 33.

Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. “Dense Passage Retrieval for Open-Domain Question Answering.” In *Proceedings of EMNLP 2020*.

Cormack, Gordon V., Charles L. A. Clarke, and Stefan Büttcher. 2009. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods.” In *Proceedings of SIGIR 2009*.

Järvelin, Kalervo, and Jaana Kekäläinen. 2002. “Cumulated Gain-Based Evaluation of IR Techniques.” *ACM Transactions on Information Systems* 20 (4): 422–446.

Malkov, Yu. A., and D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–836.

Thakur, Nandan, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. “BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models.” In *NeurIPS Datasets and Benchmarks*.

These works motivate the concrete RAG stages, fusion, approximation, and retrieval metrics used by the domain graft. The thesis adds process identity, coupling, failure, effects, resource semantics, adaptive campaigns, and production constraints.

H.8 Empirical system under study

Supplied repository snapshot. 2026. ragkit, ragopt, RAG-TTC, GEC RAG Chat, TTC Garden Assistant, associated tests, design records, and frontend code, provided for this architecture study.

Appendix I. Glossary

Adaptive campaign. A stateful sequence of proposed candidates, paired experiments, decisions, and incumbent updates. Adaptation changes future allocation but not the meaning of completed evidence.

Arm. One executable system alternative in an experiment cell. The arm produces native outcomes; it does not own hidden labels or final promotion policy.

Binding. Resolution of a pure plugin specification into an executable kernel using runtime capabilities and resources.

Candidate. A selected parameter value and immutable metadata—identity, parent, hypothesis, and locked assets—proposed for comparison with a baseline.

Capability. A named facility required to bind or execute a plugin, such as a model provider, artifact store, GPU class, network policy, or index backend.

Categorical law. An equality or refinement required by the chosen process algebra, such as identity, associativity, or copy/discard behavior.

Cell. One terminal observation at an exact experiment coordinate, normally case $\times$ repeat $\times$ arm, with native artifact and status retained.

Common random numbers. A variance-reduction coupling in which comparable stochastic components derive draws from the same root/namespace coordinate.

Composition. Construction of a larger typed process from smaller processes through serial, parallel, choice, feedback, or domain-specific wiring.

Constraint. A Boolean or structured safety/integrity observation that normally enters a hard ordered gate and cannot be compensated by metric improvement.

Control semantics. The meaning of candidate proposal, allocation, stopping, selection, and incumbent evolution over experiment history.

Coupling. A joint distribution over two or more systems' outcomes with prescribed marginals. Pairing cases is not by itself a stochastic coupling; shared or correlated environmental randomness must be declared.

Decision semantics. The rules that turn evidence into eligible, rejected, undecided, selected, or review states.

Denotational semantics. The mathematical function, stochastic kernel, or relation assigned to a system independent of one particular runtime schedule.

Descriptor catalog. An introspective collection of plugin metadata, schemas, effects, capabilities, and certificates. It is not an untyped service locator.

Deterministic morphism. A stochastic process whose output distribution is a point mass for every input. In a Markov category, deterministic maps interact naturally with copy.

Dynamic edge. A process boundary where compile-time Go types cannot cross, usually an out-of-process plugin. Schema-tagged bytes are decoded and validated, then immediately restored to typed values.

Effect. An observable interaction or requirement beyond pure input/output computation, such as network access, remote text disclosure, storage mutation, nondeterminism, or clock use.

Evaluator. A product-owned process that maps native system outcomes and hidden case material into observations. It is separate from the system arm and proposer.

Evidence semantics. The meaning of cases, repeats, couplings, observations, artifacts, aggregation, uncertainty, and durable experiment custody.

Experiment. A controlled construction of evidence comparing systems under declared cases, repeats, coupling, evaluator, and policy.

Factory. A typed plugin object with a pure Spec and a runtime Bind operation.

Fidelity. The behavioral level and cost of evaluation, such as static laws, retrieval, answer, session, load, shadow, or canary.

Global parameter element. A concrete candidate value used to instantiate a parametrized process.

Gate. An ordered policy of constraints, noninferiority tests, improvement requirements, and budgets used to determine eligibility.

Instrumented kernel. A stochastic process that returns not only a value but also status, trace, resources, warnings, and artifact references.

Intensional trace. Structured evidence of how an outcome was produced. It distinguishes executions with the same extensional result but different provenance, disclosure, fallback, or resource behavior.

Interpreter. An implementation assigning executable behavior to process specifications. The thesis uses exact finite, sampled local, experiment, and product/runtime interpreters.

Law certificate. A versioned artifact recording which law suite was executed against which plugin/spec and with what result. It is evidence, not universal proof.

Markov category. A symmetric monoidal category of stochastic processes equipped with copying and discarding operations satisfying appropriate laws.

Metric direction. Whether larger or smaller finite metric values are preferable. Direction is part of the metric schema, not inferred from a name.

Native artifact. The full domain-specific record of one execution or evaluation. Shared metrics are a projection; the native artifact remains authoritative for diagnosis.

Noninferiority. A decision that a candidate's beneficial paired difference is not below a declared negative margin on a protected metric.

Observation. The evaluator's typed projection of one native outcome: metrics, constraints, status, diagnostics, groups, resources, and artifact references.

Operational semantics. A transition relation describing runtime states and steps such as binding, execution, evaluation, commit, resume, gate, and proposal.

Parameter space. The typed domain of candidate values. It may have constraints, dependency closure, priors, topology, or differentiable structure.

Parameterized process. A family $P \otimes X \rightarrow Y$ whose behavior is instantiated by a parameter value $p : I \rightarrow P$.

Pareto frontier. The eligible candidates not dominated on all declared preference dimensions. It preserves incomparability rather than hiding it in a scalar.

Partial stochastic process. A process whose mass may include failure, nontermination, or undefined behavior. The sandbox represents failure and cancellation explicitly in outcomes.

Plugin. A replaceable implementation grafted onto a typed semantic port, with pure spec, runtime binding, declared effects/capabilities, and applicable laws.

Process semantics. The typed deterministic or stochastic meaning of the systems being optimized.

Product failure. A valid terminal outcome such as provider failure, invalid answer, or cancellation that belongs in evidence. It differs from a broken experiment interpreter.

Proposer. A control plugin that maps an authorized view of campaign history to new candidate values or a stop decision.

Random namespace. A stable semantic name used with the root seed and call labels to derive a stochastic component's draw.

Refinement. A declared relation by which one process implements or approximates another while preserving protected observations or satisfying tolerances.

Resource algebra. Rules for composing cost, latency, calls, tokens, memory, disclosure, freshness, and other operational quantities under serial and parallel process composition.

Selector. A decision/control component that chooses an eligible candidate or retains the incumbent according to declared policy.

Semantic identity. A canonical digest of all values capable of changing the protected denotation or trace of a component/process.

Service locator. A runtime registry returning implementations by name, often through erased types. The architecture avoids it in the semantic core.

Statistical game. A structured pairing of system, context, observations, and decision/evaluation meaning. The thesis uses this notion to keep evaluators and contexts separate from forward systems.

Stochastic kernel. A mapping from each input to a probability distribution over outputs.

Structured trace. A normalized serial/parallel tree of typed semantic events rather than an unstructured log line sequence.

Sufficient statistic. Retained intermediate evidence adequate for a restricted downstream comparison. Frozen channel rankings can be sufficient for a fusion-only sweep but not for a chunking or reranker campaign.

System plugin. A typed component contributing to the forward process, as distinct from evaluator, decision, or control plugins.

Tensor. Symmetric monoidal parallel composition of independent ports/processes. Randomness may still be coupled through explicit copy or environment semantics.

Trace equivalence. Equality or refinement after a declared trace projection. Final-output equality alone is generally weaker.

Typed port. A compile-time input/output boundary expressing domain values and preventing accidental incompatible composition.